스마트 제조_1일차

류영표 강사
ryp1662@gmail.com

Youngpyo Ryu. All rights reserved.
Unauthorized reproduction or distribution is prohibited.

류 영 표

AI & Applied Mathematics Researcher · Lecturer

ryp1662@gmail.com

학력

- 동국대학교 일반대학원 응용수학 석사 수료 (2016-2019)
- 동국대학교 수학과 졸업 (2010-2016)

심사 및 전문가 자문 활동

- 인공지능 학습용 데이터 구축 사업 품질검증 전문가 자문위원 (2023)
- 2023 신뢰할 수 있는 인공지능 개발 안내서(안)_고도화 전문위원
- 2023 자율주행/의료/공공사회 분야 신뢰할 수 있는 인공지능 개발 안내서(안) 자문위원

연구 및 프로젝트

- 부산대학교 병원: 개인 맞춤형 당뇨병 예방·관리 인공지능 시스템 외주 프로젝트 (2022)
- 연세대학교 병원: 영상의학과 FLASK 고도화 연구 (2023, 2024)
- 산업수학 스터디 그룹 (국가수리과학연구소): 피부암/유방암 분류, 유전자 정보 분석
- 폐플라스틱 이미지 객체 검출 경진대회 3위 (2021)
- 동국대학교 수학과 연구원(2025.10 ~ / 양자 AI, XAI 연구 진행 중)

논문

- Quantum computing for the optimization of CT image reconstruction, IEEE ICTC 2022

㈜ 헬로월드랩스 전담교육팀 과장

강의 및 교육 활동

대학 강의

- 고려대학교, 인천대학교, 동양미래대학교, 목포대학교
- 이젠 아카데미: 파이썬/ADSP 과정

기업 교육

- 현대자동차 연구원
- 한화, 하나금융, 한전 KDN, IBK 기업은행, 아가방앤컴퍼니, IM증권

공공/국비 과정

- [Est Camp] AI 퀀트 캠프 1기
- 보건·바이오 AI 활용 전문가 양성과정 (1-3기)
- 국제인공지능올림피아드 한국 대표 멘토
- 충청 ICT 취창업 역량강화 프로그램 (SQLD)

아카데미 & 부트캠프

- 모두의연구소 Aiffel 퍼실리테이터 (1기)
- Upstage AI Boostcamp 멘토 (1-6기)
- 패스트캠퍼스/멀티캠퍼스 조교 및 멘토
- 공개SW개발자대회 멘토

제조업의 발전 과정

제조업은 기술의 발전과 함께
지속적으로 변화해 왔습니다.

1차 산업혁명
기계화 (Mechanization)

시기: 18세기 후반

핵심 기술
- 증기기관
- 기계화

핵심 변화

사람의 노동력을 기계가 보조

수작업으로 생산하던 방식에서 증기기관과 기계장치가 도입되어 사람의 노동을 기계가 대신하기 시작했습니다.

2차 산업혁명
대량생산 (Mass Production)

시기: 19세기 후반 ~ 20세기 초

핵심 기술
- 전기
- 생산라인

핵심 변화

개별 생산 → 표준화된 대량생산

전기가 보급되고 생산라이 발전하면서 동일한 제품을 대형으로 생산하는 방식이 확산되었습니다.

3차 산업혁명
자동화 (Automation)

시기: 20세기 후반

핵심 기술
- 컴퓨터
- PLC
- 산업용 로봇

핵심 변화

작업자의 반복 작업 → 컴퓨터와 기계에 의한 자동화

컴퓨터, PLC, 산업용 로봇 동이 도입되면서 반복적인 생산 작업을 자동으로 수행할 수 있게 되었습니다.

4차 산업혁명
지능화 (Intelligent Manufacturing)

시기: 21세기

핵심 기술
- IoT
- 빅데이터
- AI
- 디지털 트윈

핵심 변화

정해진 작업의 자동화 → 데이터를 활용한 분석·예측·최적화

자동화된 설비에 센서와 네트워크를 연결하고, 수집한 데이터를 분석하여 생산 공정을 개선하는 지능형 제조로 발전되었습니다.

구분	1차 산업혁명	2차 산업혁명	3차 산업혁명	4차 산업혁명
시기	18세기 후반	19세기 후반 ~ 20세기 초	20세기 후반	21세기
핵심 기술	증기기관, 기계화	전기, 대량생산	컴퓨터, PLC, 산업용 로봇	IoT, 빅데이터, AI, 디지털 트윈
제조업의 변화	사람의 노동력을 기계가 보조	생산라인을 이용한 대량생산	생산 공정의 자동화	데이터를 활용한 지능형 제조

전통적인 제조업과 디지털 제조업은 무엇이 다른가?

자동차 부품을 생산하는 공장
하루에 1,000개의 제품을 생산하고,
그중 50개가 불량품이라고 가정합니다.

불량률

$$\text{불량률} = \frac{\text{불량품 수}}{\text{전체 생산량}} \times 100 = \frac{50}{1{,}000} \times 100 = 5\%$$

공장의 목표
불량률을 낮추는 것

① 전통적인 제조 방식

- 자동차 부품 생산
- 작업자 품질 검사
- 불량품 발견
- 불량 수량 기록
- 설비 및 작업 상태 점검
- 문제 원인 추정
- 설비 조정 및 공정 개선

기존 방식의 한계

- 불량품이 생산된 정확한 시각 확인 어려움
- 해당 시점의 설비 온도나 압력 기록이 없음
- 설비 상태를 작업자의 기억에 의존
- 문제 발생 이후에야 원인 조사

기존 제조업에서도 통계적 품질관리와 공정 개선 기법을 활용했지만, 데이터를 수작업으로 수집하거나 분석하는 경우가 많았습니다.

② 디지털 기술을 도입한 제조 방식

- 자동차 부품 생산
- 설비 센서 데이터 수집
- 온도·압력·진동 기록
- 설시간 설비 상태 모니터링
- 정상 범위 이탈 감지
- 설비 점검 및 공정 조정
- 생산 결과와 데이터 비교
- 품질 개선

설제 수집한 데이터 예시 (교육용 가상 데이터)

생산 시각	온도(℃)	압력(kPa)	진동(mm/s)	불량 여부
09:00	65	120	0.12	정상
09:10	68	122	0.13	정상
09:20	71	124	0.15	정상
09:30	82	135	0.34	불량
09:40	85	138	0.38	불량

불량이 발생한 구간에서 온도, 압력, 진동이 함께 중가하는 것을 확인할 수 있습니다. 이를 통해 블량 발생 시점의 설비 상태를 데이터로 확인할 수 있다는 점이 기존 방식과의 큰 차이입니다.

1 생산정보의 디지털화 (Digitization)

종이나 수기로 관리하던 정보를
컴퓨터가 처리할 수 있는 디지털 데이터로 변환하는 단계입니다.

기존 방식 — 종이 점검표에 수기로 기록

설비 점검표

시간	온도(℃)	압력(kPa)	작업자
09:00	65	120	홍길동
10:00	68	125	홍길동
11:00	72	128	홍길동
12:00	70	123	이영회
13:00	71	126	이영회

문제점

- 여러 날짜의 데이터를 비교하기 어려움
- 수작업으로 인한 기록 오류 가능
- 실시간 설비 상태 확인 불가능
- 설비 온도나 압력의 정확한 시점 확인이 어려움
- 통계 분석이나 자동 집계가 어려움

디지털 기술 도입 — Excel 또는 데이터베이스에 저장

시간	온도(℃)	압력(kPa)	작업자
09:00	65	120	홍길동
10:00	68	125	홍길동
11:00	72	128	홍길동
12:00	70	123	이영회
13:00	71	126	이영회
...	...	...	...

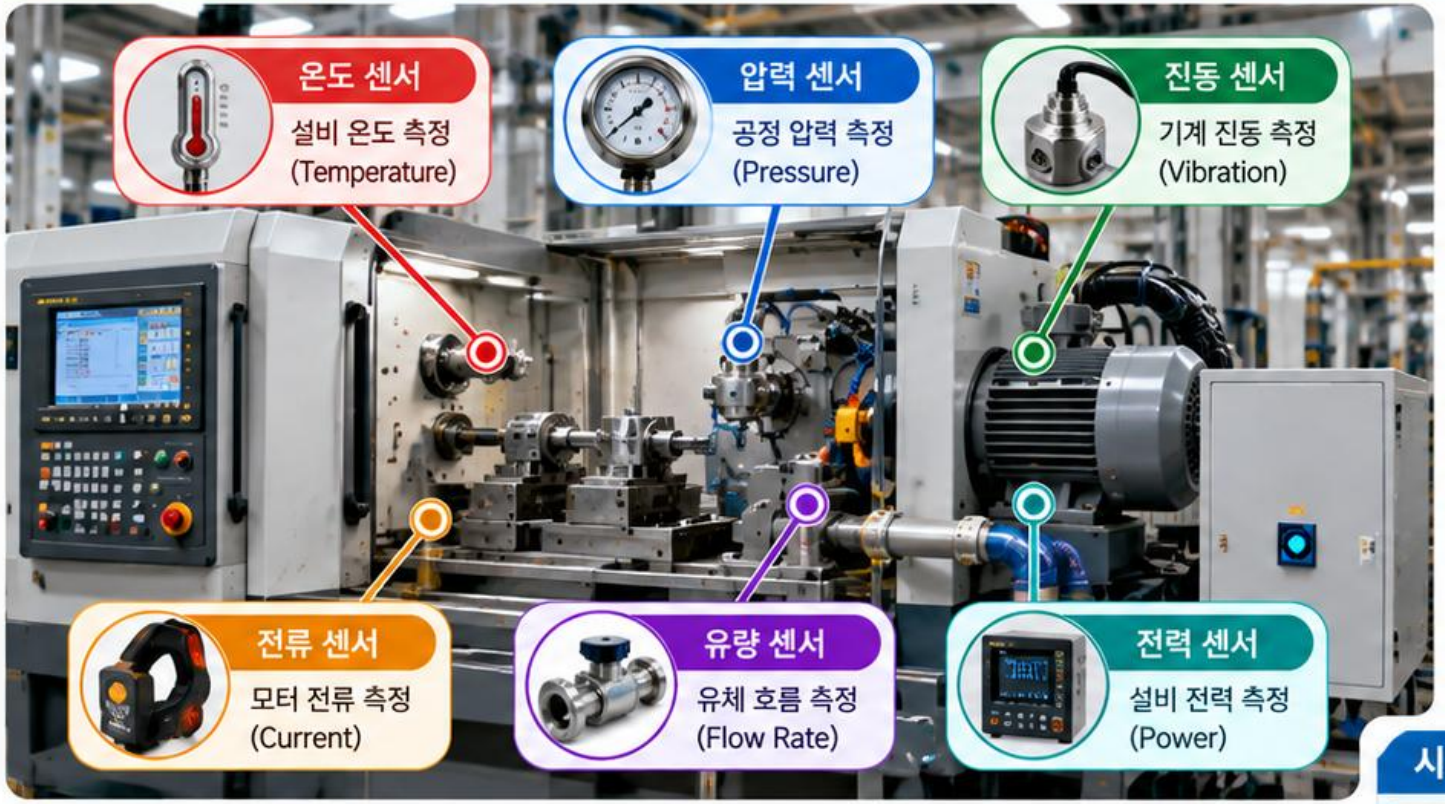
도입 효과

- 날짜별 생산량 비교
- 설비 온도 변화 분석
- 불량품 수량 집계
- 월별 생산 실적 분석
- 데이터 검색 및 공유
- 통계 분석 자동화

핵심 포인트 | 아날로그 정보를 컴퓨터가 처리할 수 있는 **디지털 데이터로 바꾸는** 것이 첫 번째 단계입니다.

2 센서를 활용한 자동 데이터 수집

제조 설비에 센서를 설치하여
온도, 압력, 진동 등 다양한 데이터를 자동으로 수집합니다.



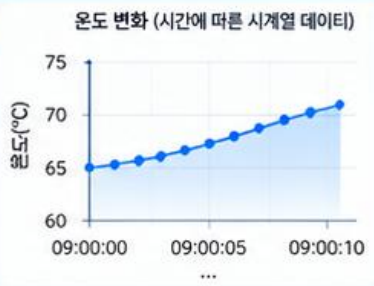
주요 센서와 수집 데이터

센서 종류	수집 데이터	확인 가능한 정보	예시 이미지
온도 센서	Temperature (°C)	설비 온도 변화 공정 온도 관리	
압력 센서	Pressure (kPa)	공정 압력 변화 유체 압력 상태	
진동 센서	Vibration (mm/s)	모터 및 기계의 진동 설비 이상 징후	
전류 센서	Current (A)	모터의 전류 소비 부하 상태 분석	
유량 센서	Flow Rate (L/min)	유체의 흐름 유량 변화 분석	
전력 센서	Power (kW)	설비 전력 사용량 에너지 소비 분석	

데이터 수집 과정

시계열 데이터 예시 (온도 센서)

측정 시각	온도(°C)
09:00:00	65.1
09:00:01	65.2
09:00:02	65.3
09:00:03	65.5
09:00:04	65.7
...	...



시간 순서대로 수집하는 데이터는 **시계열 데이터(Time Series Data)**이며, 설비 상태의 변화와 이상 징후를 파악하는 데 활용할 수 있습니다.

사용되는 디지털 기술

- **IoT** — 설비와 센서를 네트워크로 연결
- **PLC** — 센서 데이터 수집 및 제어
- **데이터 수집 장치** — 센서 데이터를 저장 및 전송
- **산업용 네트워크** — 설비 간 데이터 통신
- **SCADA / HMI** — 실시간 설비 상태 모니터링

3 네트워크를 통한 설비 연결 (Connectivity)

여러 설비를 네트워크로 연결하고
데이터를 통합하여 공장 전체의 상태를
한눈에 파악할 수 있습니다.

기존 방식

각 설비의 데이터를 개별적으로 저장

설비 A

개별 데이터 저장
- 온도 데이터
- 압력 데이터
- 생산 수량 등

설비 B

개별 데이터 저장
- 온도 데이터
- 압력 데이터
- 생산 수량 등

설비 C

개별 데이터 저장
- 온도 데이터
- 압력 데이터
- 생산 수량 등

- ☑ 각 설비의 데이터가 분리되어 있어 전체 생산라인의 상태를 한눈에 파악하기 어려움
- ☑ 설비 간 데이터 비교나 통합 분석이 어려움
- ☑ 설시간으로 공장 전체를 모니터링하기 어려움

디지털 기술 도입

설비를 네트워크로 연결하여 데이터 통합

설비 A

설비 B

설비 C

산업용 네트워크
(Ethernet, OPC UA 등)

데이터 수집 서버

통합 데이터베이스

통합 모니터링 (대시보드)

여러 설비의 데이터를 한 화면에서 확인

주요 네트워크 기술

Ethernet
공장 내 유선 네트워크

OPC UA
설비 간 데이터 교환
표준 프로토콜

산업용 통신 프로토콜
PROFINET, Modbus,
EtherNet/IP 등

핵심 포인트 | 개별 설비의 데이터를 하나의 시스템에서 확인할 수 있도록 연결하는 것이 핵심입니다.

4 실시간 모니터링 (Monitoring)

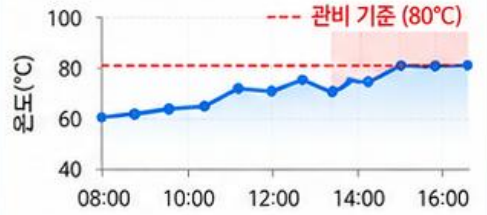
수집되고 연결된 데이터를 모니터를 통해 실시간으로 확인하여
설비 상태와 생산 현황을 관리하는 단계입니다.

대시보드 예시

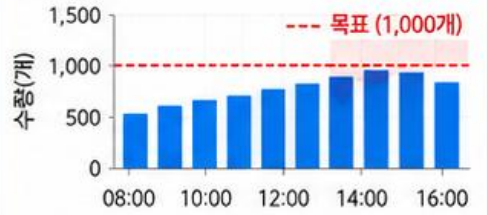
관리 항목	현재 상태	관리 기준 (예시)
설비 온도	75℃	80℃ 이하
설비 압력	125kPa	140kPa 이하
설비 진동	0.18mm/s	0.30mm/s 이하
생산 수량	850개	목표 1,000개
불량 수량	25개	목표 30개 이하

※ 관리 기준은 설명을 위한 가상 설정이며 실제 설비의 안전 기준이 아닙니다.

설비 온도 추이



생산 수량 추이



실시간 모니터링의 장점

1 설비 이상을 빠르게 확인

온도, 압력, 진동 동이 설정 범위를
벗어나면 경고를 표시하여
이상 징후를 신속하게 확인할 수 있습니다.

2 여러 설비를 동시에 관리

하나의 화면에서 여러 생산라인과
설비의 상태를 동시에 확인하여
공장 전체를 효율적으로 관리할 수 있습니다.

3 과거 데이터와 현재 데이터 비교

최근 데이터의 변화 추이를 분석하여
설비 상태를 평가하고
생산 공정을 개선할 수 있습니다.

주요 사용 기술

- **SCADA** — 공장 전체 설비 모니터링
- **HMI** — 설비 상태 시각화
- **MES** — 생산 관리 및 데이터 수집
- **데이터 시각화 시스템** — 대시보드 구성 및 분석

5 데이터 분석을 통한 생산 공정 개선

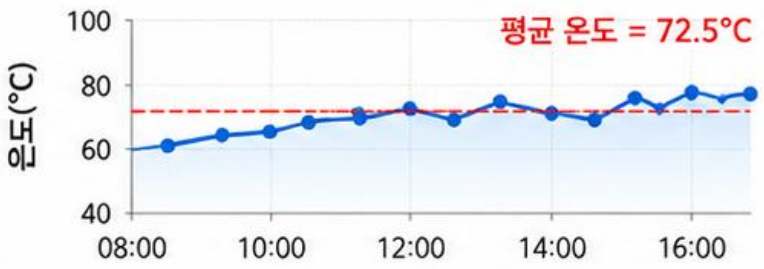
수집된 데이터를 분석하여 설비 상태를 평가하고
문제를 찾은 후, 생산 공정을 개선하는 단계입니다.

1 평균 온도

설비가 전반적으로 어느 정도의 온도에서
작동하는지 확인할 수 있습니다.

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

설비 온도 시계열 데이터 예시

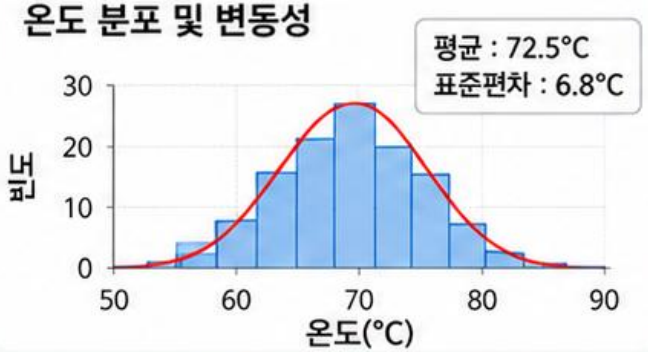


2 표준편차

온도 데이터가 얼마나 변동하는지
확인하는 데 사용합니다.

$$s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n} (x_i - \bar{x})^2}$$

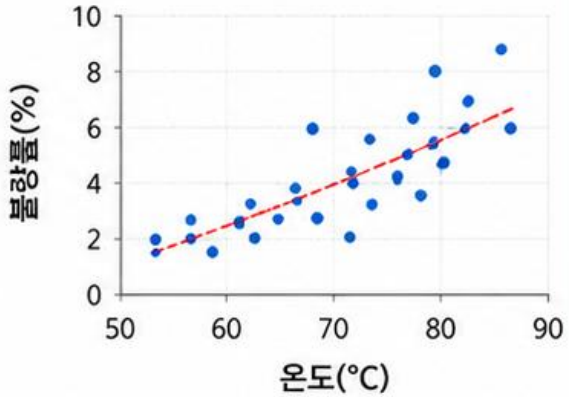
온도 분포 및 변동성



3 변수 간 관계

온도와 불량률의 관계를 분석하여
온도가 높은 구간에서 불량률이 높게 나타나는지 확인합니다.

설비 온도와 불량률의 관계 (예시)



- 온도가 높을수록 불량률이 증가하는 경향이 확인됨
- 다만, 온도만으로 불량의 원인이라고 단정할 수 없으며, 압력, 생산 속도, 설비 종류 등 다른 변수를 함께 고려해야 합니다.

데이터 분석 도구

Python
데이터 분석
통계 분석

Excel
데이터 정리
기본 분석

Power BI
데이터 시각화
대시보드 구축

주요 분석 항목과 활용 예시

분석 대상	분석 방법	활용 가능성	예시 그래프
설비 온도	평균, 표준편차	온도 변동 관리	시계열 그래프
설비 압력	시계열 분석	압력 변화 관찰	시계열 그래프
제품 불량	불량률 분석	품질 문제 탐색	막대 그래프
설비 진동	이상치 탐지	설비 이상 징후 확인	산점도
생산 시간	공정별 시간 비교	병목 공정 탐색	막대 그래프

핵심 포인트 | 이 단계부터 제조데이터가 단순한 기록을 넘어 **의사결정을 지원하는 정보**로 활용됩니다.

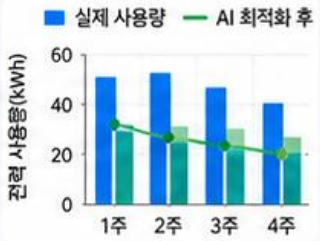
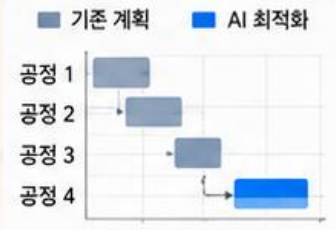
6 AI를 활용한 예측과 최적화 (AI for Prediction & Optimization)

수집된 데이터를 기반으로 미래의 상태를 예측하고 최적의 의사결정을 지원하는 단계입니다.

AI의 역할

기존 데이터 분석이 현재 상태와 과거의 문제를 이해하는 데 집중했다면, AI는 **미래의 상태를 예측하고 의사결정을 지원하는 데 활용**할 수 있습니다.

제조 현장의 문제와 AI 활용 예시

불량품이 많이 발생함	설비가 갑자기 고장 남	생산량이 일정하지 않음	전력 사용량이 높음	생산 공정에 대기 시간이 많음
AI 활용 방식 불량 발생 가능성 예측	AI 활용 방식 설비 고장 징후 탐지	AI 활용 방식 생산량 예측	AI 활용 방식 에너지 소비량 예측 및 최적화	AI 활용 방식 공정 병목 분석 및 생산계획 최적화
				
이미지 분석으로 불량품 자동 판별	센서 데이터 기반 이상 징후 조기 탐지	수요 예측을 통한 생산 계획 수립	설비 운영 조건 최적화로 에너지 비용 절감	공정 순서 및 자원 배분 최적화로 생산 효율 향상

AI 활용 과정

데이터 수집 및 전처리
- 센서 데이터 수집
- 데이터 정제
- 이상치 처리

AI 모델 학습
- 과거 데이터 학습
- 불량, 고장, 생산량 등 예측 모델 구축

예측 및 분석
- 미래 상태 예측
- 이상 징후 탐지
- 최적 공정 조건 제안

현장 적용 및 지속 개선
- 실시간 예측
- 의사결정 지원
- 모델 성능 지속 향상

데이터 품질의 중요성

AI를 도입하기 위해서는 **신뢰할 수 있는 데이터**가 확보되어야 합니다.

- 데이터 품질이 낮거나 센서값이 잘못 측정되면 AI 모델도 잘못된 결과를 출력할 수 있습니다.
- 따라서 데이터 관리, 센서 · 제어 시스템 간 통합, AI 시스템의 신뢰성이 매우 중요합니다.

NIST 미국 국립표준기술연구소 (NIST, 2026)

NIST의 2026년 스마트제조 AI 로드맵에서도 **데이터 관리, 센서 · 제어 시스템 간 통합, AI 시스템의 신뢰성을** 중요한 과제로 제시하고 있습니다.

핵심 포인트

AI는 제조데이터를 기반으로 **미래를 예측하고 더 나은 의사결정을 지원**하여 생산성을 높이는 핵심 기술입니다.

실제 제조업에서는 디지털전환을 어떻게 진행하고 있을까?

제조업은 다양한 산업에서 디지털 기술을 도입하여 생산 효율을 높이고 품질을 개선하고 있습니다.

1 자동차 산업 — 생산라인과 품질 관리의 디지털화

자동차 제조업에서는 차체 조립, 용접, 도장, 최종 검사 등 다양한 공정에 디지털 기술이 활용됩니다.

차체 용접 공정의 주요 데이터

- 용접 전류
- 용접 시간
- 설비 상태
- 용접 위치
- 품질 검사 결과

데이터 수집 및 분석

수집한 데이터를 분석하면 생산 과정의 문제를 추적할 수 있습니다.

생산 이력 관리

생산설비와 품질 검사 시스템을 연결하여 제품별 생산 이력을 관리할 수 있습니다.

Audi

Audi는 소프트웨어 정의 공장, 산업용 AI, 가상화 기술을 제조 공정에 도입하고 있으며, AI 기반 실시간 품질 검사도 추진하고 있습니다.

2 반도체 산업 — 정밀 공정의 데이터 기반 관리

반도체 제조업은 온도, 압력, 가스 유량 등 미세한 공정 조건을 정밀하게 관리해야 합니다.

데이터	활용 목적
챔버 온도	공정 온도 관리
챔버 압력	공정 조건 유지
가스 유량	공정 가스 공급 상태 분석
장비 전류	설비 상태 분석
웨이퍼 검사 결과	품질 및 수율 분석

데이터 통합 및 분석

다양한 공정 데이터를 통합하면 제품의 생산 이력을 추적하고 품질 변동을 분석하는 데 도움이 됩니다.

3 지멘스 (Siemens) — 디지털 트윈을 활용한 공장 설계

디지털전환은 이미 운영 중인 공장뿐 아니라 새로운 공장을 설계할 때도 활용됩니다.

가상 공장 설계 (디지털 트윈)

실제 공장 구축

디지털 트윈 활용 과정

1. 공장 모델링 (가상환경 구축)
2. 설비 배치 및 공정 설계
3. 시뮬레이션 및 사전 검증
4. 실제 공장 구축 및 운영 최적화

주요 효과 (Siemens, 2022)

생산성 20% 향상 (기존 시설 대비)

- 설비 배치, 자재 이동, 생산 공정을 사전에 검토 가능
- 공장 구축 전 문제점 발견 및 개선
- 더 효율적인 공장 설계 및 운영

※ 이는 디지털 트윈만의 독립적인 효과가 아니라 종합적인 디지털화의 성과입니다.

제조업 디지털전환의 핵심은 무엇일까?

데이터를 기반으로 제조 현장을 연결하고,
분석하며, AI를 활용하여 더 효율적이고
지능적인 공장으로 변화시키는 것입니다.

제조업 디지털전환의 6단계

지금까지 살펴본 내용을 하나의 흐름으로 정리하면 다음과 같습니다.

1 디지털화

종이 기록을
Excel · 데이터베이스로
변환

주요 기술

Excel, 데이터베이스

2 데이터 자동 수집

센서를 활용하여
제조 현장의 데이터를
자동으로 수집

주요 기술

센서, PLC, IoT,
데이터 수집 장치

3 설비 및 시스템 연결

여러 설비를 네트워크로
연결하고 데이터를
통합

주요 기술

산업용 네트워크,
OPC UA, 데이터 통합

4 실시간 모니터링

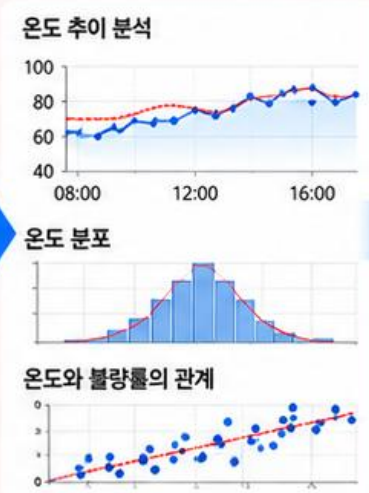
대시보드를 통해
설비 상태와 생산 현황을
실시간으로 확인

주요 기술

HMI, SCADA,
MES, 대시보드

5 데이터 분석

수집된 데이터를 분석하여
문제를 찾고
개선 방향을 도출



주요 기술

통계 분석, 시각화,
이상 탐지

6 AI 기반 예측 및 최적화

AI를 활용하여
미래의 상태를 예측하고
최적의 의사결정을 지원

- 불량품 예측
- 설비 고장 예측
- 생산량 예측
- 에너지 최적화
- 공정 최적화

주요 기술

머신러닝, 딥러닝,
예측 모델, 최적화

핵심 포인트

위의 6단계는 디지털전환의 발전 과정을 이해하기 위한 대표적인 설명이며,
실제 기업이 반드시 이 순서대로 모든 기술을 도입해야 하는 것은 아닙니다.

기업은 해결하려는 문제와
기존 설비 환경에 맞게 필요한 기술부터 도입할 수 있습니다.

디지털전환 전후 비교

디지털전환은 단순히 자동화를 넘어서,
공장 곳곳의 데이터를 연결하고 분석하여 생산과 의사결정에 활용하는 것입니다.

디지털전환 이전 (As-Is)

분리된 정보, 제한된 분석, 경험 중심의 관리

구분	디지털전환 이전	디지털전환 이후
생산 기록	종이 및 수기 기록	디지털 자동 기록
설비 상태 확인	작업자의 정기 점검	센서 기반 지속적 모니터링
품질 관리	생산 후 검사 중심	생산 중 데이터 분석과 검사 병행
설비 고장 관리	고장 후 수리, 정기 점검	상태 기반 점검 및 고장 예측
데이터 분석	수작업 및 제한적인 분석	통계 분석 및 자동화된 데이터 처리
생산관리	경험과 기존 계획 중심	실시간 데이터 기반 의사결정 지원
공정 개선	문제 발생 후 원인 조사	지속적인 모니터링과 개선

디지털전환 이후 (To-Be)

연결된 데이터, 실시간 분석, 데이터 기반 의사결정

다음 점을 꼭 기억해야 합니다.

디지털전환 이전에도 자동화와 통계적 품질관리는 존재했습니다.
따라서 디지털전환을 단순히 '사람이 하던 일을 기계가 대신하는 것'이라고 설명하는 것은 정확하지 않습니다.

디지털전환의 핵심

공장 곳곳에서 생성되는 **데이터를 연결하고 분석**하여 **생산과 의사결정에 활용**하는 것입니다.

1 스마트팩토리란?

제조 설비, 센서, 네트워크, 데이터 관리 시스템, 분석 기술 등을 연결하여 생산 공정을 효율적으로 운영하고 개선하는 지능형 제조 시스템입니다.

스마트팩토리의 정의

스마트팩토리(Smart Factory)는 제조 설비, 센서, 네트워크, 데이터 관리 시스템, 분석 기술 등을 연결하여 생산 공정을 효율적으로 운영하고 개선하는 **지능형 제조 시스템**입니다.

스마트팩토리의 핵심 3가지

1 데이터 수집 (Data Collection)

- 센서와 제어 시스템으로부터 생산 데이터 수집
- 온도, 압력, 진동, 전류, 생산량 등

2 데이터 연결 및 분석 (Data Integration & Analytics)

- 여러 설비의 데이터를 통합
- 통계적 분석과 시각화
- 공정 상태 파악

3 데이터 기반 의사결정 (Data-driven Decision Making)

- 생산관리, 품질 검사, 설비 점검, 공정 개선에 활용
- 필요 시 AI를 활용하여 불량 예측, 설비 이상 예측 등 지원

스마트팩토리의 기본 구조

제조 설비 (생산 공정)

- CNC, 로봇, 컨베이어, 검사기 등
- 제품의 가공, 조립, 검사 과정

데이터 수집 (센서 · PLC)

온도 센서 · 압력 센서 · 진동 센서

PLC
(설비 제어 및 데이터 수집)

데이터 분석 (통합 · 저장 · 분석)

데이터 서버
(통합 데이터베이스)

통계 분석
및 시각화

AI 모델
(이상 탐지 · 예측)

의사결정 (생산 · 품질 · 설비 관리)

- 생산관리
- 품질 검사
- 설비 검검
- 공정 개선

제조 경쟁력 향상

- 생산 효율 향상
- 품질 개선
- 설비 가동률 중가
- 비용 절감

중요한 점

스마트팩토리는 반드시 AI를 사용해야 하는 것은 아닙니다.

예를 들어 온도 센서로 설비 상태를 모니터링하고, 설정 온도를 초과하면 경고를 발생시키는 시스템도 스마트 제조 환경의 중요한 구성 요소입니다.
AI는 스마트팩토리의 기능을 더욱 고도화하는 기술 중 하나입니다.

2 일반적인 제조공장과 스마트팩토리의 차이

같은 제조공장이라도 데이터를 어떻게 수집하고,
연결하고, 활용하노냐에 따라 공장의 운영 방식이 달라집니다.

자동차 부품 생산 공장 예시

이 공장에서는 하루에 1,000개의 부품을 생산합니다.
다양한 설비가 연결되어 하나의 공정을 이루고 있습니다.

CNC 가공 설비	산업용 로봇	컨베이어벨트	검사 장비
부품 가공	부품 조립	제품 이동	품질 검사

제조 방식 비교

구분	일반적인 제조 방식	스마트팩토리
데이터 수집	수기 기록 및 개별 장비 확인	센서와 시스템을 통한 자동 수집
설비 관리	정기 점검 및 고장 후 수리	상태 모니터링과 예지보전
품질 검사	사람 중심의 검사	머신비전 및 AI 검사 지원
생산 관리	경험과 계획 중심	실시간 생산정보와 데이터 기반 관리
이상 감지	작업자가 직접 확인	센서 분석 및 자동 경고
의사결정	담당자의 경험 중심	현장 경험과 데이터 분석 결과 결합

사례: CNC 가공 설비의 온도 상승

CNC 설비에서 정상적으로 작동할 때 온도가 약 65℃라고 가정합니다. 생산 중 온도가 85℃까지 증가한 경우를 비교해 보겠습니다.

기존 관리 방식

1 정기 점검

작업자가 정기 점검 과정에서
온도 상승을 발견합니다.

2 설비 점검

설비를 점검하고
원인을 조사합니다.

3 조치 및 복구

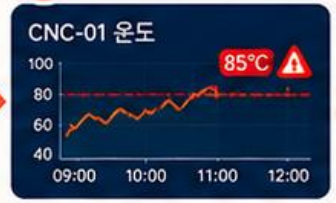
필요한 조치를 수행하여
설비를 부구합니다.

스마트팩토리 방식

1 센서 데이터 수집

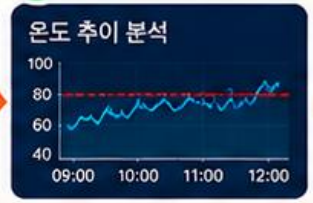
온도 센서
(실시간 측정)

2 실시간 모니터링



온도가 설정된 관리 기준을
초과하면 경고가 표시됩니다.

3 데이터 기반 분석



과거 온도 변화와 현제
운전 상태를 비교합니다.

4 AI 예측 (선택)

- 과거 데이터를 학습하여
  이상 징후를 예측할 수 있습니다.
- 설비 점검 시정을 미리 계획할 수
  있습니다.

참고 사항

- 일반적인 제조공장에서도 자동화 설비와 통계적 품질관리는 존재합니다.
- 스마트팩토리라고 해서 모든 공정에 AI가 사용되는 것은 아닙니다. 단순한 센서 모니터링이나 자동제어도 스마트팩토리의 중요한 구성 요소입니다.

※ 65℃와 85℃는 수업용 가상 설정이며
설제 CNC 설비의 정상·위험 온도를 의미하지 않습니다.

3 스마트팩토리의 주요 기술

스마트팩토리는 다양한 기술이 서로 연결되어 데이터가 수집되고, 분석되어 생산 공정을 효율적으로 운영하고 개선하는 지능형 제조 시스템입니다.

1 IoT
Internet of Things

센서와 설비를 연결하여 데이터를 수집합니다.

- 온도, 압력, 진동, 전류, 생산량 등 측정
- 유선/무선 네트워크로 데이터 전송
- 실시간 데이터 수집

2 PLC
Programmable Logic Controller

제조 설비를 자동으로 제어하는 장치입니다.

- 센서값을 기반으로 설비 동작 제어
- 모터, 밸브, 컨베이어 동 제어
- 안정적이고 신뢰성 높은 산업용 제어

3 HMI
Human-Machine Interface

작업자가 설비 상태를 확인하고 조작하는 화면입니다.

- 설비 상태 확인
- 작업자 조작 (시작/정지, 설정값 변경 동)
- 경고 및 알람 표시

4 SCADA
Supervisory Control and Data Acquisition

여러 설비의 데이터를 수집하고 감시하는 시스템입니다.

- 실시간 설비 상태 모니터링
- 알람 관리
- 데이터 기록 및 추세 분석
- 필요 시 상위 제어 기능

5 MES
Manufacturing Execution System

제조 현장의 생산 실행과 이력을 관리하는 시스템입니다.

- 생산 지시 및 실적 관리
- 공정 이력 관리
- 품질 이력 관리
- 설비 및 작업 이력 관리

6 ERP
Enterprise Resource Planning

자재, 재고, 구매, 생산 등 기업 전체 자원을 관리하는 시스템입니다.

- 구매, 자재, 재고 관리
- 생산계획 관리
- 판매, 회계, 인사 등 기업 자원 통합 관리

7 AI
Artificial Intelligence

수집된 데이터를 분석하여 이상 탐지, 품질 예측, 생산 최적화에 활용합니다.

- 제품 불량 예측
- 설비 고장 징후 탐지
- 생산량 예측
- 공정 최적화

8 Digital Twin
디지털 트윈

실제 설비와 공정을 디지털 환경에 모델링하여 분석하고 시뮬레이션합니다.

- 가상 공장 모델 구축
- 공정 시뮬레이션
- 설비 배치 및 성능 분석
- 공정 개선 방안 도출

4 IoT — 사물인터넷 (Internet of Things)

다양한 장치와 사물을 네트워크로 연결하여 데이터를 주고받을 수 있도록 하는 기술입니다.
제조업에서는 산업용 IoT(IIoT)를 많이 사용합니다.

1 IoT란?

IoT(Internet of Things)는 다양한 장치와 사물을 네트워크로 연결하여 데이터를 주고받을 수 있도록 하는 기술입니다.
제조업에서는 **산업용 IoT(Industrial IoT, IIoT)**라는 용어도 많이 사용합니다.

제조 설비의 주요 센서 예시

센서	측정 데이터	활용 목적
온도 센서	Temperature	설비 과열 상태 확인
압력 센서	Pressure	공정 압력 관리
진동 센서	Vibration	설비 진동 변화 분석
전류 센서	Current	모터 부하 상태 파악
전력 센서	Power	에너지 소비량 분석

IoT 데이터 수집 구조

자동차 부품 가공 설비 (CNC)

2 IoT를 도입하면 무엇이 달라질까?

기존에는 작업자가 매시간 온도를 기록했지만, IoT를 활용하면 설정된 수집 주기에 따라 데이터를 자동으로 저장할 수 있습니다.
예를 들어 1초마다 수집하는 경우 다음과 같은 데이터가 생성됩니다.

기존 방식 (수기 기록)

시간	온도(℃)	작업자
09:00	65	김○○
10:00	68	이○○
11:00	72	박○○
12:00	70	최○○

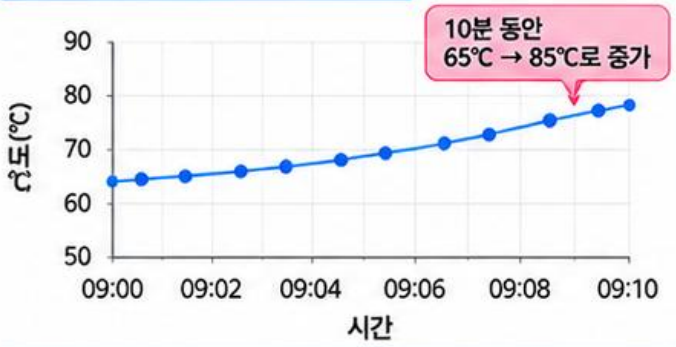
작업자가 매시간 기록

IoT 도입 후 (자동 수집)

측정 시각	온도(℃)
09:00:00	65.1
09:00:01	65.2
09:00:02	65.3
09:00:03	65.5
09:00:04	65.7

설정된 주기로 자동 저장
(예: 1초마다 수집)

시계열 데이터 예시 (온도 변화)



이러한 데이터를 **시계열 데이터(Time Series Data)**라고 합니다.
시간에 따른 변화가 중요하며, 단순한 현재 값뿐 아니라 변화 추세를 분석하는 것이 중요합니다.

3 IoT와 센서는 같은 것일까?

IoT와 센서는 같은 개념이 아닙니다.
센서는 물리량을 측정하는 장치이고,
IoT는 센서와 장치들을 연결하여 데이터를 교환하는 기술입니다.
모든 센서가 IoT 장치인 것은 아닙니다.

센서 (Sensor)

온도, 압력 등의
물리량을 측정하는 장치

IoT (사물인터넷)

센서와 장치들을 연결하여
데이터를 주고받는 기술

기존 PLC에 연결된 온도 센서도 게이트웨이나 네트워크 시스템을 통해 데이터를 수집할 수 있습니다.

5 PLC — 프로그래머블 로직 컨트롤러

PLC는 제조 현장에서 기계와 설비를 제어하기 위해 사용하는 산업용 제어장치입니다.

1 PLC란?

PLC(Programmable Logic Controller)는 제조 현장에서 기계와 설비를 제어하기 위해 사용하는 **산업용 제어장치**입니다.

주요 기능

- 모터 가동 및 정지
- 컨베이어벨트 제어
- 온도 조건에 따른 장치 제어
- 센서 입력에 따른 설비 동작
- 비상정지 및 인터록 조건 처리

안전 기능에는 해당 안전 요구사항을 충족하는 전용 안전 PLC나 안전회로 등이 사용될 수 있습니다.

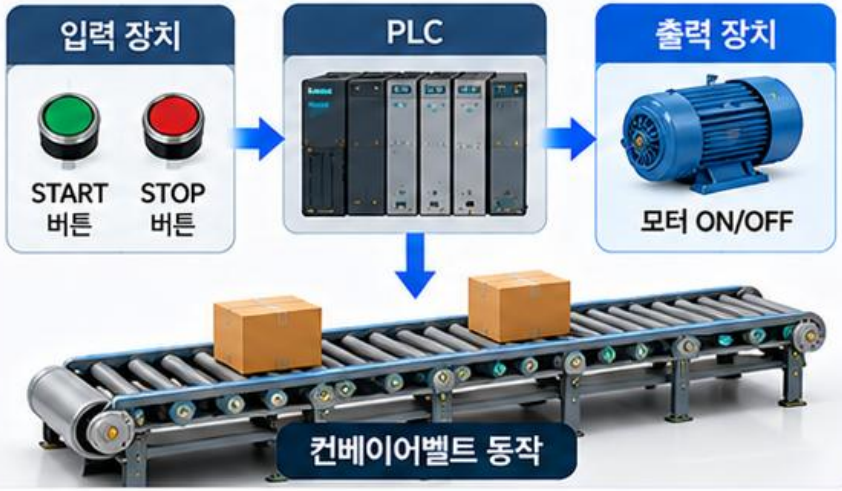
2 PLC의 기본 동작 구조

PLC는 **입력 장치**로부터 신호를 받아 제어 **로직**을 실행하고, 그 결과에 따라 **출력 장치**를 제어합니다.

입력 (Input)	제어 로직 (Control)	출력 (Output)
• 센서 신호 • 버튼 입력 • 기타 입력 신호	• 프로그램 실행 • 논리 연산 • 조건 판단 • 타이머 / 카운터 동	• 모터 제어 • 밸브 제어 • 경고등, 부저 등

3 실제 예시: 컨베이어벨트 제어

작업자가 START 버튼을 누르면 모터가 작동하고, STOP 버튼을 누르면 모터가 정지하는 예시입니다.



4 온도 제어 예시

설비 온도가 설정 범위를 초과하면 냉각 장치를 가동하도록 제어할 수 있습니다.

온도 센서 (현재 온도 측정) → **PLC** (제어 조건 실행) → **냉각 장치** (설정 온도 초과 시 가동)

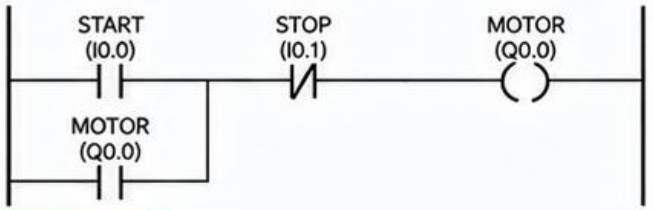
교육용 의사코드 (Python 예시)

```
temperature = 85

if temperature > 80:
    print("냉각 장치 가동 요청")
else:
    print("정상 운전")
```

- 이 코드는 실제 PLC 프로그램이 아니라 제어 조건을 이해하기 위한 Python 예시입니다.
- 실제 PLC 프로그램에서는 센서 입력, 제어 출력, 안전 조건 등을 함께 고려해야 합니다.

5 PLC 프로그램의 예 (래더 다이어그램)

PLC는 주로 **래더 다이어그램**(Ladder Diagram)과 같은 전용 프로그래밍 언어를 사용하여 제어 로직을 작성합니다.



동작 설명

1. START 버튼(I0.0)을 누르면 모터(Q0.0)가 작동합니다.
2. STOP 버튼(I0.1)을 누르면 모터가 정지합니다.
3. 보조 접점을 사용하여 START 버튼을 떼어도 모터가 유지됩니다.

6 PLC의 핵심 역할과 특징

- **정해진 제어 로직**에 따라 설비를 안정적으로 동작시킴
- 실시간 제어 (빠른 응답 속도)
- 산업 환경에 적합한 높은 **신뢰성**과 **안정성**
- 다양한 센서와 출력 장치를 연동
- AI가 아닌, 설비 제어를 위한 **산업용 제어 장치**

중요한 점!

PLC는 대어터를 분석하는 AI 시스템이 아닙니다.

PLC는 정해진 제어 로직에 따라 제조 설비를 안정적으로 동작시키는 장치입니다.

6 HMI — 인간과 기계의 인터페이스 (Human-Machine Interface)

작업자가 설비 상태를 확인하고
조작할 수 있도록 제공하는 화면 또는 인터페이스입니다.

1 HMI란?

HMI(Human-Machine Interface)는
작업자가 설비 상태를 확인하거나
장치를 **조작**할 수 있도록 제공하는
화면 또는 **인터페이스**입니다.

주요 기능

- 설비 상태 확인 (운전 상태, 온도, 속도 등)
- 설비 조작 (시작, 정지, 설정값 변경 등)
- 생산 현황 및 그래프 표시
- 경고 및 알람 표시
- 설비 설정 및 레시피 관리

2 HMI 화면 예시 (자동차 부품 생산설비)

3 PLC와 HMI의 차이

구분	PLC (Programmable Logic Controller)	HMI (Human-Machine Interface)
목적	설비 제어	설비 상태 확인 및 조작
역할	제어 프로그램 실행	작업자에게 정보 제공
입력	센서, 버튼 등	작업자의 화면 조작 (터치, 버튼 등)
출력	모터, 밸브 등에 제어 신호	상태 정보, 경고, 그래프
예시	온도 초과 시 냉각장치 제어	현재 온도와 경고 표시

4 HMI와 PLC의 연결 구조

5 핵심 정리

PLC는
기계를 제어하는 장치

- 정해진 제어 로직에 따라 설비를 자동으로 동작시킴
- 센서 입력을 받아 출력 제어
- 안정적이고 신뢰성 높은 산업용 제어 장치

HMI는
사람과 기계가 소통하는 화면

- 설비 상태를 확인하고 조작
- 현재 값, 그래프, 경고 등을 표시
- 작업자에게 필요한 정보를 제공
- 설비 설정 및 레시피 관리

PLC는 기계를 제어하고, HMI는 사람이 기계와 소통하는 화면입니다.

7 SCADA — 설비 감시 및 데이터 수집 시스템

SCADA는 여러 설비와 공정의 데이터를 한 곳에서 수집하고 감시하는 시스템으로, 공장의 전체 상황을 실시간으로 확인할 수 있습니다.

1 SCADA란?

SCADA(Supervisory Control and Data Acquisition)는 여러 설비와 공정의 데이터를 수집하고 **감시**하며, 필요에 따라 **상위 수준의 제어 기능을 제공**하는 시스템입니다. HMI가 특정 설비의 화면이라면, SCADA는 여러 설비나 생산라인을 통합하여 감시하는 형태로 활용할 수 있습니다.

2 자동차 부품 공장 예시

자동차 부품 공장에는 CNC 가공 설비, 조립 로봇, 검사 설비 등 다양한 설비가 있습니다. SCADA를 활용하면 이들 설비의 상태를 통합 화면에서 실시간으로 확인할 수 있습니다.

SCADA 시스템
(통합 감시 및 데이터 수집)

- 설비 상태 (실시간 모니터링)
- 경고 정보 (알람 관리)
- 데이터 기록 (운전 이력 저장)

3 SCADA 화면 예시

4 SCADA의 주요 기능

기능	설명
데이터 수집	PLC 및 설비로부터 데이터를 수집합니다.
실시간 감시	여러 설비의 상태를 화면에 실시간으로 표시합니다.
알람 관리	설정된 조건을 벗어나면 경고를 발생시킵니다.
데이터 기록	설비 상태 및 운전 이력을 저장합니다.
추세 분석	온도, 압력 등의 변화 추이를 시각화합니다.
상위 제어	권한과 설계에 따라 상위 수준의 제어 기능을 제공합니다.

5 HMI와 SCADA의 차이

구분	HMI	SCADA
사용 범위	개별 장비 중심	여러 장비 통합
주요 목적	설비 상태 확인 및 조작	설비 통합 감시 및 관리
화면 구성	특정 설비 화면	공장 전체 화면
기능 범위	설비 조작, 상태 표시	데이터 수집, 감시, 알람, 기록, 추세 분석, 상위 제어
대표 화면	개별 설비 터치스크린	공장 통합 모니터링 화면

HMI와 SCADA의 기능이 겹치는 경우도 있으며, 실제 제품에서는 통합되어 제공되기도 합니다.

8 MES – 제조 실행 시스템 (Manufacturing Execution System)

MES는 제조 현장에서 생산이 계획대로 진행되고 있는지 관리하는 정보시스템으로, 생산 지시부터 실적, 품질, 공정, 설비, 작업 이력까지 제조 실행에 필요한 정보를 관리합니다.

1 MES란?

MES(Manufacturing Execution System)는 제조 현장에서 생산이 계획대로 진행되고 있는지 관리하는 정보시스템입니다.
PLC와 SCADA가 설비의 제어와 상태 감시에 집중한다면, MES는 **어떤 제품을 언제, 어디에서, 얼마나 생산했는지** 관리하는 데 집중합니다.

MES의 특징

- 생산 계획을 실제 실행으로 연결
- 설비 데이터와 생산 정보를 통합 관리
- 품질, 공정, 설비, 작업 이력을 기록하고 추적

2 MES의 위치와 역할

ERP (기업 자원 관리) — 주문, 자재, 재고, 구매, 생산계획 등 기업 전체 자원 관리

↓ 생산계획 ↑ 실적정보

MES (제조 실행 관리) — 생산 지시, 실적, 품질, 공정, 설비, 작업 이력 관리

↓ 작업 지시 / 생산 정보 ↑ 설비 데이터 / 실행 결과

PLC / SCADA (설비 제어 및 감시) — 설비 제어, 데이터 수집, 실시간 감시

3 MES에서 관리하는 주요 정보

관리 항목	설명
생산 지시	어떤 제품을 얼마나 생산할 것인지
생산 실적	실제 생산된 제품 수량
품질 이력	정상 및 불량 제품 정보
공정 이력	어떤 공정을 거쳐 생산되었는지
설비 이력	어떤 설비를 사용했는지
작업 이력	언제, 어떤 작업이 수행되었는지

4 MES 화면 예시 (생산 관리 대시보드)

5 생산관리 예시 (수치 계산)

생산 결과 (예시)

항목	수량
목표 생산량	1,000 개
실제 생산량	850 개
정상 제품	825 개
불량 제품	25 개

생산 목표 달성률

$$\text{목표 달성률} = \frac{\text{실제 생산량}}{\text{목표 생산량}} \times 100$$

$$= \frac{850}{1000} \times 100$$

$$= 85\%$$

불량률

$$\text{불량률} = \frac{\text{불량 제품 수}}{\text{실제 생산량}} \times 100$$

$$= \frac{25}{850} \times 100$$

$$= 2.94\%$$

6 MES 활용 효과

1. **실시간 생산 현황 파악** — 계획 대비 실제 생산 진행 상황 확인
2. **품질 관리 강화** — 불량 발생 원인 추적 및 품질 개선
3. **공정 이력 추적** — 어떤 공정을 거쳐 생산되었는지 확인
4. **설비 및 작업 관리** — 설비 사용 이력과 작업 이력 관리
5. **데이터 기반 의사결정** — 생산 효율화 및 공정 개선에 활용

MES는 설비 데이터와 생산 이력이 연결되어 있어 특정 제품이 어떤 설비에서 생산되었는지도 추적할 수 있습니다.

9 ERP — 전사적 자원관리 시스템 (Enterprise Resource Planning)

ERP는 기업 전체의 자원을 통합적으로 관리하여 효율적인 경영을 지원하는 정보시스템입니다.

1 ERP란?

ERP(Enterprise Resource Planning)는 기업 전체의 자원을 통합적으로 관리하는 정보시스템입니다. 구매, 재고, 자재, 생산계획, 판매, 회계, 인사 등 기업의 다양한 업무를 하나의 시스템에서 통합 관리합니다.

ERP에서 관리하는 주요 업무

- **구매** (발주, 공급사 관리)
- **재고** (재고 현황 관리)
- **자재** (자재 소요 계획)
- **생산계획** (생산 일정 수립)
- **판매** (주문, 출하 관리)
- **회계** (비용, 매출 관리)
- **인사** (인력 및 조직 관리)

2 ERP와 MES의 연결 구조 (자동차 부품 예시)

고객 주문
자동차 부품 10,000개 주문

주문 정보
- 제품 종류
- 수량
- 납기일
- 고객 정보

ERP (기업 자원 관리)

생산계획 및 자재 정보

MES (제조 실행 관리)

생산 지시 및 실행

재고 및 실적 반영

생산 실적 품질 정보

제조 설비
(실제 제품 생산)

ERP의 역할
- 주문 확인
- 생산계획 수립
- 필요한 자재 및 구매 계획
- 재고 확인
- 납기 관리 등

MES의 역할
- 생산 지시
- 현장 작업 관리
- 생산 실적 기록
- 품질 및 공정 이력 관리

3 업무 흐름 예시

1. **고객 주문 접수**
   자동차 부품 10,000개 주문
2. **ERP에서 생산계획 수립**
   자재 소요량 계산 및 구매 계획 수립
3. **MES에서 생산 지시**
   생산 일정에 따라 작업 지시
4. **제조 설비에서 생산**
   CNC, 로봇, 조립 라인 등에서 실제 생산
5. **MES에서 생산 실적 기록**
   생산 수량, 불량률, 공정 이력 등 저장
6. **ERP에 재고 및 실적 반영**
   생산 완료 후 재고 업데이트 및 경영 정보에 활용

4 MES와 ERP의 차이

구분	MES (Manufacturing Execution System)	ERP (Enterprise Resource Planning)
관리 대상	제조 현장	기업 전체
주요 업무	생산 실행 및 이력 관리	자원 및 업무 계획·관리
주요 정보	생산량, 불량률, 공정 이력	주문, 재고, 구매, 회계, 인사
활용 목적	제조 운영 효율화	기업 자원 운영 효율화

5 예시: 생산 데이터 결과

항목	수량
목표 생산량	10,000 개
실제 생산량	9,500 개
정상 제품	9,200 개
불량 제품	300 개

생산 목표 달성률

$$= \frac{\text{실제 생산량}}{\text{목표 생산량}} \times 100$$

$$= \frac{9{,}500}{10{,}000} \times 100$$

$$= 95\%$$

불량률

$$= \frac{\text{불량 제품 수}}{\text{실제 생산량}} \times 100$$

$$= \frac{300}{9{,}500} \times 100$$

$$= 3.16\%$$

6 핵심 정리

- ERP는 기업의 전체 자원을 통합 관리
- MES는 제조 현장의 생산 실행 관리
- ERP와 MES는 서로 다른 역할을 하며 연결됨
- 생산 데이터는 MES에서 관리되고, 기업 자원 정보는 ERP에서 통합 관리됨

10 AI — 인공지능 (Artificial Intelligence)

AI는 제조 현장에서 수집된 데이터를 분석하여 예측과 판단을 지원하는 기술입니다.

1 스마트팩토리에서 AI의 역할

센서, PLC, SCADA, MES 동이 데이터를 수집하거나 관리하는 역할을 한다면, AI는 수집된 데이터를 분석하여 예측과 판단을 지원합니다.

데이터 수집	데이터 처리	AI 분석 및 예측	의사결정 지원
• 센서 • PLC • SCADA • MES	• 데이터 저장 • 데이터 통합 • 데이터 정제	• 불량 예측 • 고장 징후 탐지 • 생산량 예측 • 에너지 사용량 예측	• 품질 개선 • 설비 점검 • 생산 계획 조정 • 운영 효율화

2 AI 활용 사례

제조 현장의 문제	AI 활용 방식
제품 불량 발생	제품 불량 가능성 예측
설비 고장	고장 징후 탐지
생산량 변동	생산량 예측
에너지 소비 증가	전력 사용량 예측
제품 표면 결함	이미지 기반 결함 탐지

3 AI를 이용한 제품 불량 예측

입력 데이터 (제조 센서 데이터)

온도 (Temperature) 압력 (Pressure) 진동 (Vibration)

$X = (\text{Temperature}, \text{Pressure}, \text{Vibration})$

머신러닝 모델

$\hat{y} = f_\theta(X)$

예측 결과

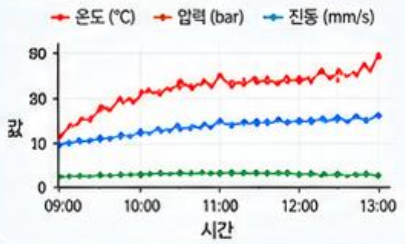
$$\hat{y} = \begin{cases} 0 & \text{정상 제품} \\ 1 & \text{불량 제품} \end{cases}$$

정상 (0) 불량 (1)

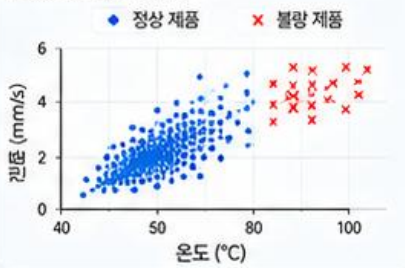
4 AI 모델 개발 과정

1	제조데이터 수집	설비, 센서, 품질 데이터 수집
2	데이터 정제	결측치 처리, 이상치 제거
3	데이터 분석	데이터 특징 파악 및 시각화
4	학습 데이터 준비	학습용 데이터셋 구성
5	AI 모델 학습	머신러닝 모델 학습
6	성능 평가	모델 성능 검증 (정확도, 재현율 등)
7	새로운 데이터 예측	실제 데이터에 모델 적용
8	의사결정 지원	품질 개선, 설비 점검 등 활용

센서 데이터 시각화 예시



불량 예측 결과 예시



5 생성형 AI의 역할 (ChatGPT / Codex)

생성형 AI는 제조데이터 분석을 더 쉽고 빠르게 수행할 수 있도록 코드 작성, 그래프 생성, 결과 해석을 지원합니다.

제조 센서 데이터 (CSV 파일)

ChatGPT / Codex
Python 분석 코드 생성

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
...
```

데이터 분석 실행	
머신러닝 모델 학습	• 불량 예측 모델 학습 • 모델 성능 평가
제품 불량 여부 예측	새로운 데이터에 예측 모델 적용
분석 결과 및 보고서	그래프, 요약, 인사이트 제공

6 머신러닝과 생성형 AI의 차이

머신러닝 모델 (예측 AI)

- 제조데이터를 학습하여 제품 불량, 고장, 생산량 등을 예측
- 수치 데이터, 이미지 등을 기반으로 예측 모델을 개발

예시: 센서 데이터를 학습하여 제품 불량 여부를 분류

생성형 AI (ChatGPT / Codex)

- 데이터 분석을 위한 Python 코드 작성
- 데이터 전처리, 분석, 그래프 생성
- 분석 결과를 설명하고 보고서 작성 지원

예시:
"이 데이터를 분석하는 파이썬 코드를 작성해줘"
"이 그래프를 해석해줘"

생성형 AI를 활용하면 프로그래밍 경험이 부족한 수강생도 제조데이터 분석을 시작할 수 있습니다.
하지만 AI가 생성한 코드와 분석 결과는 **직접 검증**해야 합니다.

11 Digital Twin — 디지털 트윈

디저털 트윈은 실제 설비나 생산 공정을 디지털 환경에 표현하고, 실제 시스템의 데이터와 연결하여 상태를 분석하거나 시뮬레이션하는 기술입니다.

1 디지털 트윈이란?

디지털 트윈(Digital Twin)은 실제 설비나 생산 공정을 디지털 환경에 표현하고, 실제 시스템의 데이터와 연결하여 **상태를 분석하거나 시뮬레이션**하는 기술입니다.

실제 공장의 운영 상태를 가상 공간에 그대로 반영하여 미리 예측하고, 다양한 운영 방안을 검토할 수 있습니다.

2 실제 공장과 디지털 트윈의 연결 구조

실제 제조공장 (Physical Factory)

센서 데이터 수집
(운도, 압력, 진동, 생산량 등)

산업용 네트워크

실시간 데이터 연동
(PLC, SCADA, MES 등)

디지털 트윈 모델 (Virtual Factory)

- **상태 분석**
  설비 상태 모니터링
  이상 징후 분석
- **시뮬레이션**
  공정 변경 시 영향 분석
  생산량 예측
- **성능 평가**
  생산 효율, 병목 구간 분석
  다양한 운영 방안 비교

개선 방안 도출
최적의 운영 방안 선택 및 적용

3 디지털 트윈을 사용하는 이유

1. **공정 변경 전 사전 검토**
   실제 공정을 변경하기 전에 가상 환경에서 검토 가능
2. **다양한 운영 시나리오 비교**
   생산량, 공정 시간, 설비 배치 등을 여러 조건으로 시뮬레이션
3. **비용과 시간 절감**
   실제 적용 전에 문제점을 확인하여 불필요한 비용과 시간 감소
4. **안전성 향상**
   설비 간 충돌 가능성, 위험 구간 등을 사전에 확인
5. **지속적인 운영 최적화**
   실시간 데이터를 기반으로 공정 개선 방안 도출

4 적용 예시 : 새로운 로봇 추가 시뮬레이션

현재 생산라인 (실제 공장)

현재 상황

- 로봇 4대 운용
- 1시간 생산량 : 100개
- 공정 시간 : 60초
- 병목 구간 : 조립 공정

디지털 트윈 시뮬레이션

새로운 로봇 추가 (가상 환경)

시뮬레이션 결과

- 로봇 5대 운용
- 1시간 생산량 : 120개 (20% 증가)
- 공정 시간 : 48초 (20% 단축)
- 설비 간 간섭 없음

5 디지털 트윈의 주요 활용 분야

- **생산 라인 설계 및 최적화**
  설비 배치, 공정 흐름, 생산량 예측
- **설비 성능 분석**
  설비 상태 모니터링, 예지보전
- **제품 및 공정 시뮬레이션**
  생산 조건 변경 시 영향 분석
- **운영 효율 개선**
  데이터 기반 의사결정 지원

6 주의할 점

- 디지털 트윈의 예측이 실제 공정과 얼마나 일치하는지 검증해야 합니다.
- 단순한 3D 공장 모델이나 일회성 시뮬레이션을 모두 디지털 트윈이라고 부르는 것은 적절하지 않습니다.

12 스마트팩토리의 전체 시스템 구조

스마트팩토리는 여러 계층의 시스템이 서로 다른 역할을 수행하면서 데이터를 교환하여, 생산계획부터 실제 설비 제어까지 통합적으로 운영하는 지능형 제조 시스템입니다.

기업 수준 (비즈니스)

Level 4 기업 계획 및 물류 관리

ERP
(Enterprise Resource Planning)

- 주문, 자재, 재고, 구매
- 생산계획, 판매, 회계, 인사

생산계획, 자재정보 ↕ ↕ 생산실적, 재고정보

제조 운영 수준 (운영관리)

Level 3 제조 운영관리

MES
(Manufacturing Execution System)

- 생산 지시 및 실행
- 생산 실적, 품질 이력, 공정 이력

작업 지시, 공정 정보 ↕ ↕ 생산 데이터, 설비 상태

현장 제어 수준 (설비 제어)

Level 2 공정 감시 및 제어

SCADA
(Supervisory Control and Data Acquisition)

- 설비 데이터 수집 및 감시
- 실시간 모니터링, 알람 관리

설비 제어 명령 ↕ ↕ 설비 상태 데이터

물리적 수준 (실제 설비)

Level 1 측정 및 제어

PLC
(Programmable Logic Controller)

- 설비 제어 프로그램 실행
- 모터, 밸브 동 제어

HMI
(Human-Machine Interface)

- 설비 상태 확인 및 조작
- 운전 화면, 경고 표시

센서 입력 신호 ↕ ↕ 제어 출력 신호

Level 0 실제 생산 공정

센서

- 온도, 압력, 진동, 전류, 전력 동
- 설비 상태 및 공정 데이터 측정

제조 설비

- CNC, 로봇, 컨베이어, 검사기
- 실제 제품 생산 및 물리적 공정

1 시스템 구조 요약

ERP	기업 자원 및 생산계획 관리 (주문, 재고, 구매, 회계 동)
MES	생산 실행 및 품질 이력 관리 (생산 지시, 생산 실적, 공정 이력)
SCADA	설비 데이터 수집 및 감시 (실시간 모니터링, 알람 관리)
PLC / HMI	설비 제어 및 작업자 인터페이스 (설비 제어, 상태 확인, 운전 화면)
센서	온도 / 압력 / 진동 / 전류 동 데이터 수집
제조 설비	CNC / 로봇 / 컨베이어 / 검사기 (실제 제품 생산)

2 ISA-95(IEC 62264)와의 관계

ISA-95는 제조 제어 시스템과 기업 업무 시스템 사이의 정보 통합을 위한 표준 모델로, 스마트팩토리의 시스템 계층을 이해하는 데 도움을 줍니다.

계층	주요 역할	대표 기술
Level 4	기업 계획 및 물류 관리	ERP
Level 3	제조 운영관리	MES, 생산 이력 시스템
Level 2	공정 감시 및 제어	PLC, SCADA, HMI
Level 1	측정 및 물리적 조작	센서, 액추에이터
Level 0	실제 생산 공정	CNC, 생산설비, 물리적 공정

- 실제 현장에서는 HMI와 SCADA가 별도로 배치되거나 기능이 중복될 수 있고, AI 분석 시스템, 데이터 저장 서버 동이 여러 계층과 직접 연결될 수 있습니다.
- 이 그림은 각 기술의 역할을 이해하기 위한 교육용 단순화 모델입니다.

13 스마트팩토리에서 데이터는 어떻게 이동할까?

하나의 제조데이터가 센서에서 시작하여
설비 제어, 데이터 저장, 모니터링, 분석, AI 활용까지
어떤 과정을 거치는지 살펴보겠습니다.

예시: 자동차 부품 가공 설비의 온도 데이터

CNC 설비의 온도 센서에서 측정된 $T = 75.2°C$ 데이터가 각 시스템을 거쳐 활용되는 과정을 살펴봅니다.

1 센서에서 데이터 측정

온도 센서가 CNC 설비의 온도를 측정합니다.

CNC 설비의 온도, 압력, 진동 등 다양한 센서가 실시간으로 데이터를 측정합니다.

2 PLC 또는 수집 장치로 전달

측정된 온도 데이터가 PLC나 데이터 수집 장치로 전달됩니다. PLC는 제어 로직에 따라 온도를 확인하고 필요한 동작을 수행할 수 있습니다.

$T = 75.2°C$

- 정상 (계속 운전)
- 주의 (경고 알람)
- 초과 (냉각장치 동작)

3 데이터 서버에 저장

온도 데이터가 측정 시각과 함께 데이터베이스에 저장됩니다.

Timestamp	Machine	Temperature
09:00:00	CNC_01	75.2
09:00:01	CNC_01	75.3
09:00:02	CNC_01	75.4
09:00:03	CNC_01	75.6
...	...	...

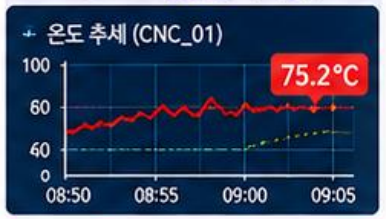
Timestamp는 데이터를 측정한 시각입니다.

4 실시간 모니터링

HMI 또는 SCADA 화면에서 온도 변화를 확인합니다. 관리 기준을 초과하면 경고를 발생시킬 수 있습니다.

HMI (개별 설비 화면)

SCADA (통합 감시 화면)



⚠ 경고 CNC_01 온도가 설정값에 근접합니다.

5 데이터 분석

과거에 수집한 온도 데이터를 이용하여 평균과 표준편차를 계산합니다.

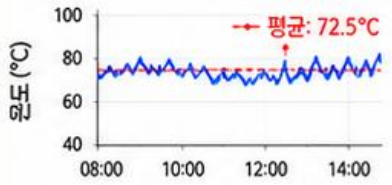
평균 온도

$$\bar{T} = \frac{1}{n}\sum_{i=1}^{n} T_i$$

표본 표준편차

$$s_T = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(T_i - \bar{T})^2}$$

온도 데이터 분석 예시



- 평균 온도로 전반적인 운전 수준 파악
- 표준편차로 온도 변동의 크기 파악

6 AI 모델 활용

온도뿐 아니라 압력, 진동 등의 데이터를 함께 사용하여 불량 발생 가능성을 예측할 수 있습니다.

- Temperature = 75.2°C
- Pressure = 125 kPa
- Vibration = 0.18 mm/s

→ 머신러닝 모델 (불량 예측)

→ 불량 발생 가능성 = 12%

→ 관리자 검토 (의사결정 지원)

AI 예측 결과는 의사결정을 지원하는 자료로 활용하며, 자동제어 적용 시에는 추가적인 검증이 필요합니다.

데이터 흐름 요약

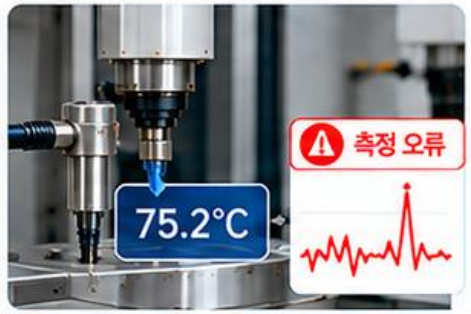
센서 (온도, 압력, 진동 등) → 데이터 전송 → PLC (데이터 수집 및 제어) → 데이터 저장 → 데이터 서버 (데이터베이스) → 데이터 모니터링 → HMI / SCADA (실시간 감시) → 데이터 분석 → 데이터 분석 (통계 분석) → 예측 모델 → AI 모델 (불량 예측 등) → 의사결정 지원 → 관리자 (생산, 품질, 설비 관리)

14 스마트팩토리 도입 시 고려해야 할 문제

스마트팩토리에는 다양한 장점이 있지만, 기술을 도입한다고 해서 모든 문제가 자동으로 해결되는 것은 아닙니다.

1 데이터 품질

센서값이 잘못 측정되거나 데이터에 결측치가 많이 발생하면 분석 결과의 신뢰성이 낮아질 수 있습니다.



결측치 발생

주요 고려사항

- 센서의 정확도 및 교정
- 결측치 처리 방법
- 이상치 탐지 및 제거
- 데이터 품질 모니터링

2 시스템 연결

서로 다른 제조사에서 만든 설비들이 사용하는 통신 방식이나 데이터 형식이 다를 수 있습니다.
따라서 시스템 간 데이터 교환 방식과 공통적인 데이터 구조를 고려해야 합니다.

주요 고려사항

- 통신 프로토콜 호환성
- 데이터 형식 및 표준화
- 시스템 간 연동 방식
- 기존 설비와의 호환성

3 보안

스마트팩토리는 다양한 장지를 네트워크로 연결하므로 산업용 제어 시스템의 보안이 중요합니다.
실제 설비를 제어하는 시스템은 일반적인 업무용 정보시스템보다 안전성과 가용성에 대한 요구사항이 높을 수 있습니다.

- 접근 통제
- 네트워크 분리
- 지속적 모니터링

NIST SP 800-82 Rev. 3
(Guide to Operational Technology (OT) Security)

제조업을 포함한 운영기술 환경의 보안과 안전성, 신뢰성을 다루는 대표적인 지침입니다.

4 비용과 운영 인력

센서, 네트워크, 데이터 서버, 분석 시스템 등을 구축하기 위해 비용이 발생합니다.
또한 시스템을 운영하고 데이터를 분석할 수 있는 전문 인력이 필요합니다.

주요 고려사항

- 초기 구축 비용
- 유지보수 및 운영 비용
- 전문 인력 확보 및 교육
- 단계적 도입 전략

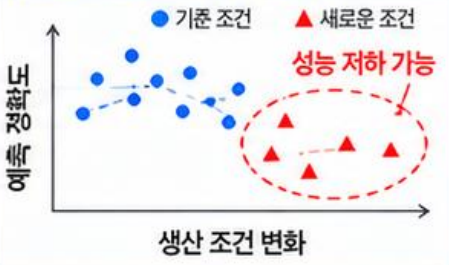
5 AI 모델의 신뢰성

AI 모델의 정확도가 높더라도 새로운 생산 조건이나 설비에서 동일한 성능을 보장하지는 않습니다.
따라서 현장 적용 전 검증과 적용 후 지속적인 성능 관리가 필요합니다.

현장 적용 시 고려사항

- 다양한 생산 조건에서 검증
- 채로운 설비에 대한 성능 평가
- 지속적인 모델 업데이트
- 예측 결과의 해석 및 신뢰성 확인

생산 조건에 따른 모델 성능 예시

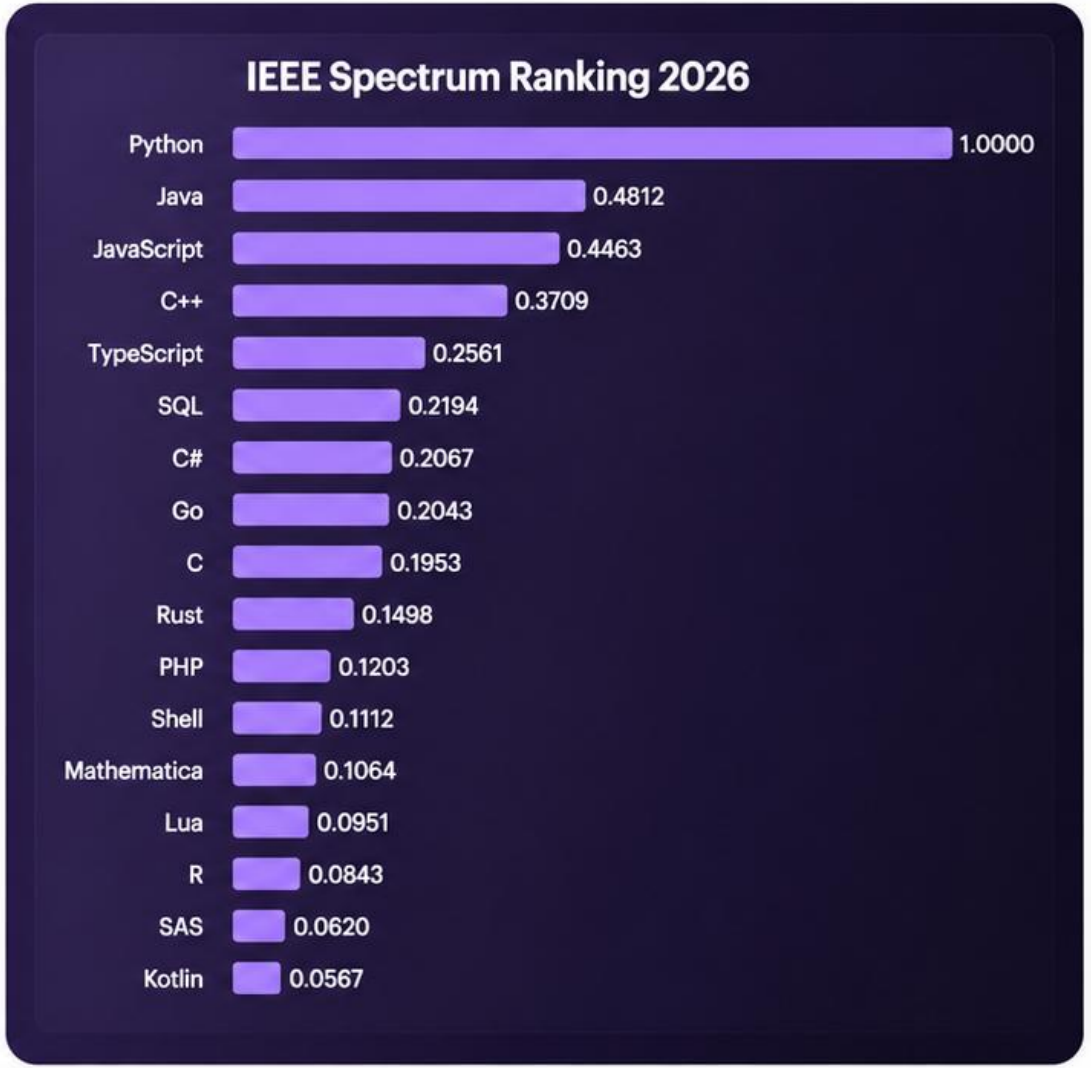


스마트팩토리는 기술 도입만으로 완성되지 않습니다.
데이터 품질, 시스템 연결, 보안, 비용, AI 모델 신뢰성 등 다양한 요소를 함께 고려하여 단계적으로 도입하고 지속적으로 관리하는 것이 중요합니다.

Why Python

The TIOBE Ranking 2026

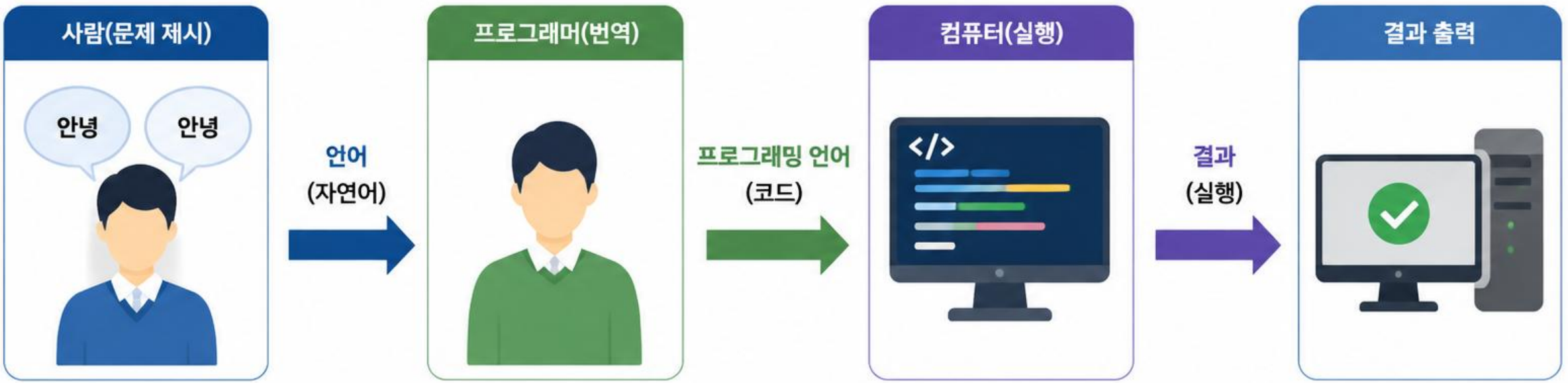
FEB 2026	FEB 2025	LANGUAGE	RATINGS	CHANGE
1	1	Python	24.31%	+8.91%
^ 2	3	C++	11.96%	+0.59%
^ 3	4	Java	10.79%	+0.13%
v 4	2	C	9.33%	-0.51%
5	5	C#	4.24%	+0.12%



What is programming?

프로그래밍(programming)이란?

사람이 이해하는 언어(자연어)를 컴퓨터가 이해하고 실행할 수 있는 언어(프로그래밍 언어)로 바꾸어,
컴퓨터가 **문제를 해결**하도록 지시하는 과정

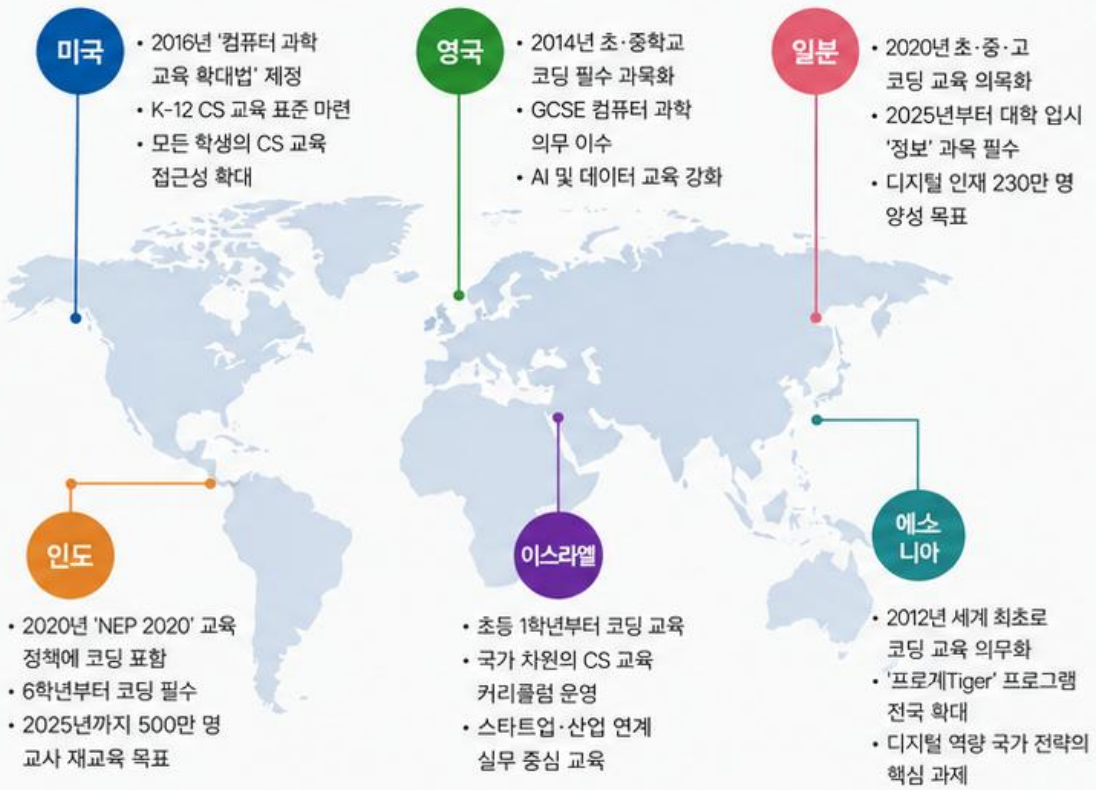


생각 / 요구 ▶ 설계 / 번역 ▶ 작성 / 코드 ▶ 실행 / 결과

★ 프로그래밍은 사람과 컴퓨터 사이의 다리(번역자) 역할을 하여, 우리의 아이디어를 컴퓨터가 수행할 수 있는 명령으로 바꿔줍니다.

코딩 교육

주요 국가 코딩 교육 현황 (2024~2025)



공통 흐름 코딩 교육을 기초 소양으로 규정하고, 초등 저학년부터 필수화하여 디지털 인재 양성과 국가 경쟁력 강화에 집중하고 있습니다.

디지털 인재 양성 주요 내용 (최신)

자료: 교육부, 각국 교육부(2024~2025)

구분	주요 내용	적용 시기
초·중·고 디지털 교육 강화	✓ 코딩 교육 의무화 및 교육과정 개선 ✓ AI·데이터·컴퓨팅 사고력 중심 교육 확대 ✓ 실습 중심 프로젝트 학습 강화	2024~2025년 지속 확대
디지털 영재 육성	✓ 소프트웨어(SW)·AI 영재 교육 확대 ✓ 영재교육원 및 특화 프로그램 확충 ✓ 국제 경진대회 및 산학 연계 강화	2024~2025년 지속 확대
대학·고등교육 혁신	✓ 학·석·박사 통합과정 확대(5년 6개월~11학기) ✓ AI·SW 융합전공 및 마이크로디그리 도입 ✓ 산학협력 및 취업 연계 강화	2023~2025년
산업·사회 연계 및 인프라	✓ 기업·학교 연계 교육(인턴십, 실무 프로젝트) ✓ 디지털 교육 플랫폼 및 학습 인프라 확충 ✓ 지역·계층 간 디지털 교육 격차 해소	2024~2025년 지속 확대
정책·제도 지원	✓ 디지털 인재 양성 종합 계획 추진 ✓ 교원 연수 및 전문성 강화 ✓ 교육 예산 및 지원 체계 강화	2024년부터 지속 추진

코딩 교육은 미래 사회의 필수 역량!
모든 학생이 디지털 세상을 이해하고 창의적으로 문제를 해결할 수 있도록 지원합니다.

파이썬 이란?

역사

- **1989년 크리스마스:** 개발 시작
  (Guido van Rossum)
- **1991년:** 첫 공식 배포

특징

쉽고 직관적인 문법
누구나 쉽게 읽고 이해할 수 있는 문법

객체지향 + 고수준 언어
효율적이고 유연한 프로그래밍 지원

다른 언어(C, C++)보다 배우기 쉬움
프로그래밍 입문에 최적화

강력한 라이브러리와 생태계

데이터 과학, 인공지능, 웹, 게임, IoT 등
다양한 분야를 지원하는 풍부한 라이브러리

Data Science | AI / ML | Web | Game | IoT | ...

객체지향 언어

- 프로그램을 객체(Object) 단위로 구성
- 재사용, 유지보수, 확장성에 유리

고수준 언어

- 하드웨어 세부사항을 몰라도 개발 가능
- 생산성 높고 코드 작성 속도 빠름

왜 많은 개발자가 선택할까?

- 빠른 개발과 유지보수
- 다양한 분야에서 활용 가능
- 커뮤니티와 자료가 풍부

귀도 반 로섬
(Guido van Rossum)

파이썬의 창시자
네덜란드 출신 프로그래머

> *"Life is too short. You need python."*
> (인생은 너무 짧으니 파이썬이 필요해.)

파이썬 이란?

- Python 이름은 영국 코미디 프로그램 *Monty Python's Flying Circus* 에서 유래
  → 재미있고 가벼운 느낌을 주는 이름을 원해 선택

- 단어 자체는 그리스 신화 속 거대한 뱀을 뜻하기도 함
  → 하지만 이는 공식 유래와는 무관

그리스 신화 속 거대한 뱀 '파이톤(Python)'

Python 로고

Python의 특징

1. 간결한 문법

볼필요한 코드가 적고,
간결하고 직관적입니다.

C++

```cpp
#include <iostream>
using namespace std;
int main() {
    cout << "Hello, gabia!";
    return 0;
}
```

Java

```java
public class HelloGabia {
    public static void main(String args[]) {
        System.out.println("Hello, gabia!");
    }
}
```

Python

```python
print("Hello, gabia!")
```

2. 간단한 반복 처리

반복문을 더 쉽고
간결하게 작성할 수 있습니다.

C++

```cpp
#include <iostream>
using namespace std;
int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    for(int i=0; i<5; i++) {
        cout << arr[i] << endl;
    }
    return 0;
}
```

Java

```java
public class HelloGabia {
    public static void main(String args[]) {
        int arr[] = {1, 2, 3, 4, 5};
        for(int i=0; i<5, i++) {
            System.out.println(arr[i]);
        }
    }
}
```

Python

```python
arr = [1, 2, 3, 4, 5]
for el in arr:
    print(el)
```

3. 가독성 높은 코드

들여쓰기를 기반으로 하여
코드 구조가 명확하고
가독성이 뛰어납니다.

C++

```cpp
#include <iostream>
using namespace std;
int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    for(int i=0; i<5; i++) {
        cout << arr[i] << endl;
    }
    return 0;
}
```

Java

```java
public class HelloGabia {
    public static void main(String args[]) {
        int arr[] = {1, 2, 3, 4, 5};
        for(int i=0; i<5, i++) {
            System.out.println(arr[i]);
        }
    }
}
```

Python

```python
arr = [1, 2, 3, 4, 5]
for el in arr:
    print(el)
```

Python의 특징

- **플랫폼** : 프로그램이 실행되는 환경 (운영체제: Windows, MacOS, Linux 등)

- **플랫폼 독립성** : 한 번 작성한 코드가 여러 운영체제에서 실행 가능

- **인터프리터** : 코드를 한 줄씩 번역하며 실행

  → 결과를 바로 확인 가능

- **플랫폼 독립적인 인터프리터 언어**

Python의 특징

사람

사람이 알아본다

의도와 생각을 표현

프로그래밍 언어

```
# Python 예제
name = input("이름이 뭐예요? ")
print(f"안녕하세요, {name}님!")

# 사람이 이해할 수 있는
# 프로그래밍 언어입니다.

안녕하세요, Python님!
```

사람이 이해할 수 있는 언어

Python은 쉬운 문법으로
문제를 표현한다

C, C++

미리 해놓는다
= 컴파일러

번역

그때 그때 번역한다
= 인터프리터

자바스크립트, 파이썬, 루비

기계어

```
01001011 01100101 01110010 01101110
01100101 01101100 00100000 01101110
01100101 01110100 01100011 11101100
00100000 01110101 01101000 01101111
01101001 00100000 01100101 01100010
01110110 01100101 01101010 01100110
01101110 00100000 01101010 01110010
01011111 00100000 01111010 01110010
```

0,1로 이루어진 언어

기계가 이해할 수 있는 언어

기계

기계가 알아본다

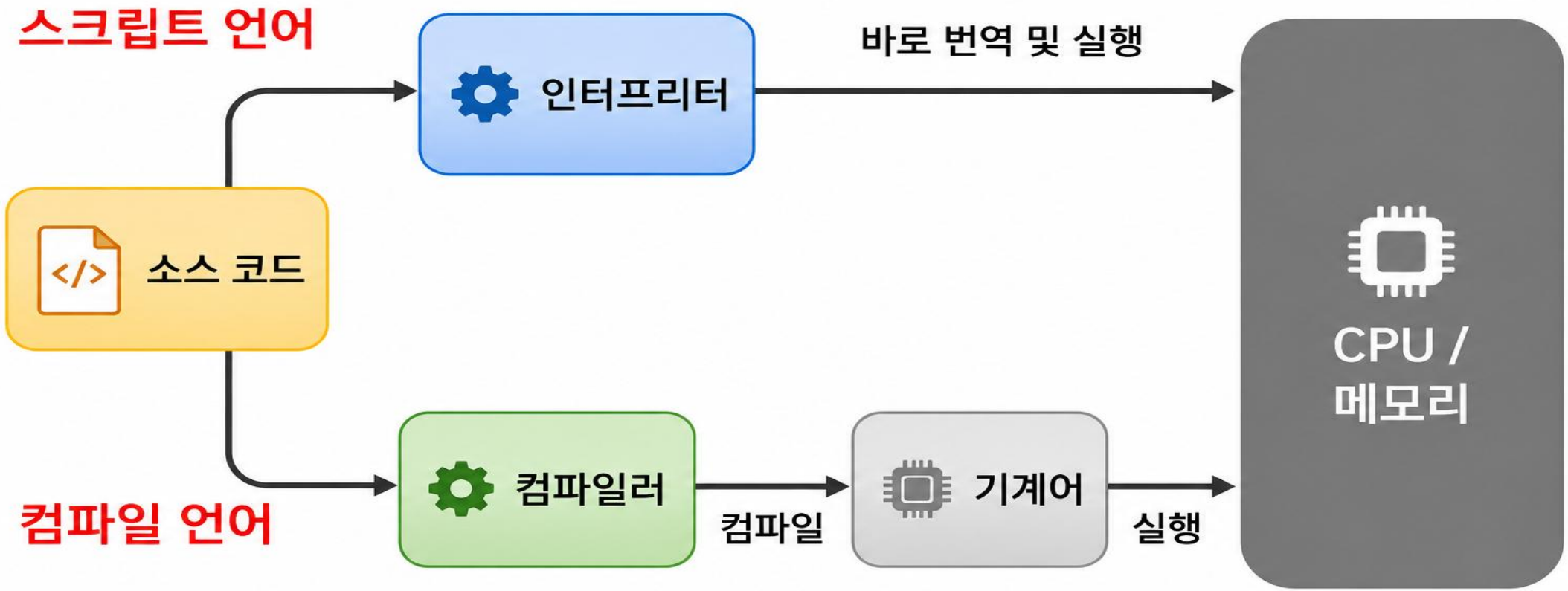
명령을 실행한다

Python은 **인터프리터 방식**으로 동작하여, 쉽게 작성하고 바로 실행할 수 있는 언어입니다.

컴파일러와 인터프리터 비교

특징	컴파일러	인터프리터
번역 방법	프로그램 전체 번역	실행되는 줄(라인) 단위 번역
장점	한 번 컴파일한 후에는 매번 빠른 시간 내에 전체 실행 가능	번역 과정이 비교적 간단하고 대화형 언어에 편리함
단점	프로그램의 일부를 수정하는 경우에도 전체 프로그램을 다시 컴파일해야 함	실행할 때마다 매번 기계어로 바꾸는 과정을 다시 수행해야 하므로 항상 인터프리터가 필요함
출력물	목적 코드	즉시 실행
언어 종류	FORTRAN, COBOL, C 등	BASIC 등

[참고] 프로그램의 동작 과정



Python의 특징

객체지향 언어

객체 = 속성 + 행동

속성	행동
색상	달리다

동적 타이핑

실행 시점에 타입 결정

10 → 'Hello'

컴퓨터와 인간 중 어디에 가까운가?

Machine code

Assembly language

C

Java

C++

Ruby

Python

저수준 언어
Low-level

고수준 언어
High-level

저수준 언어 Low-level		고수준 언어 High-level
	하드웨어에 가까움	
	실행 속도 빠름	
	사용이 어렵고 복잡함	
		사람의 언어에 가까움
		개발 생산성 높음
		실행 속도는 상대적으로 느림

자바 VS 파이썬

Dynamic Typing

: Java는 미리 변수 타입을 선언해야 하지만, Python은 실행 시점에 자동으로 결정

예제 1 (숫자 연산)

Java

```java
public class Main {
  public static void main(String[] args) {
    int a = 10, b = 20;
    int result = a + b;
    System.out.println("The result a+b = " + result);
  }
}
```

Python

```python
a = 10
b = 20

print(f"The result a+b = {a+b}")
```

Output:

```
The result a+b = 30
```

```
The result a+b = 30
```

예제 2 (문자열 출력)

Java

```java
public class Main {

  public static void main(String[] args) {

     System.out.println("Hello World");

  }

}
```

Python

```python
print("Hello World")
```

Output:

```
Hello World
```

```
Hello World
```

자바 (JAVA)

: 정적 타이핑 언어, 변수 타입을 미리 선언해야 함,
C는 절차지향 + 정적 타이핑.

파이썬 (Python)

: 동적 타이핑 언어, 변수 타입을 선언하지 않고
실행 시 자동 결정

파이썬 활용도

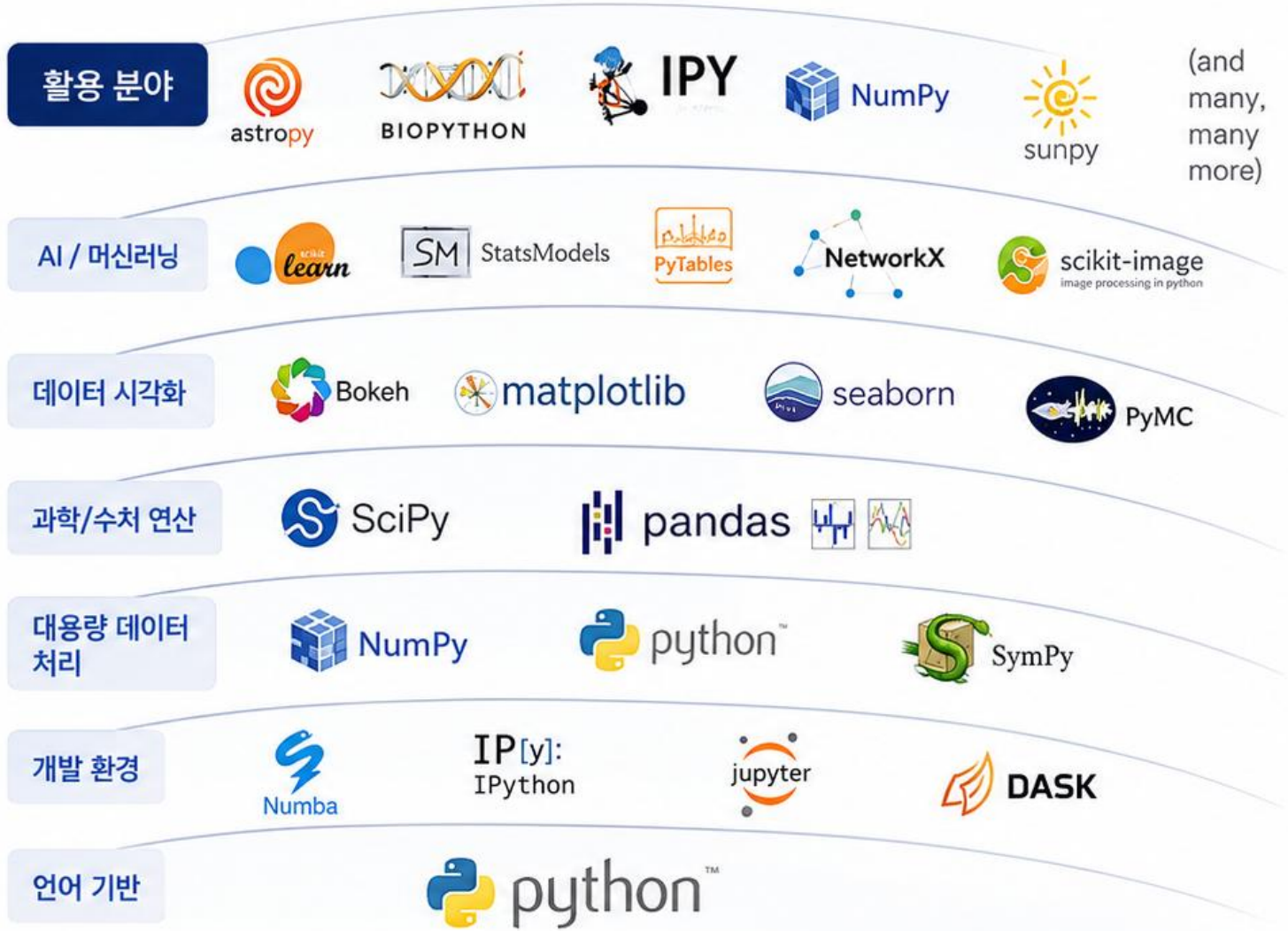
데이터분석
: pandas, numpy ···

데이터시각화
: matplotlib, seaborn ···

AI
: sklearn, tensorflow, pytorch ···

대용량 데이터처리
: pyspark, pandas ···

백엔드 개발
: django, flask, fastapi ···

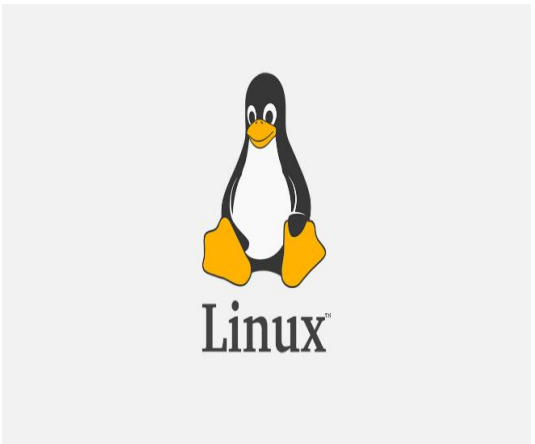


개발 환경 개요

- **프로그램**을 **작성**하고, **실행** 시키는 환경
- 일반적으로 **<코딩 환경>** 이라고 부름
- 개발환경을 **결정**

1. **운영 체재** (Operating System, OS)
   컴퓨터 하드웨어를 관리하고 프로그램 실행을 지원
2. **Python interpreter**
   파이썬 코드를 해석하고 실행하는 프로그램
3. **코드 편집기** (Editor)
   코드를 작성하고 저장하는 도구

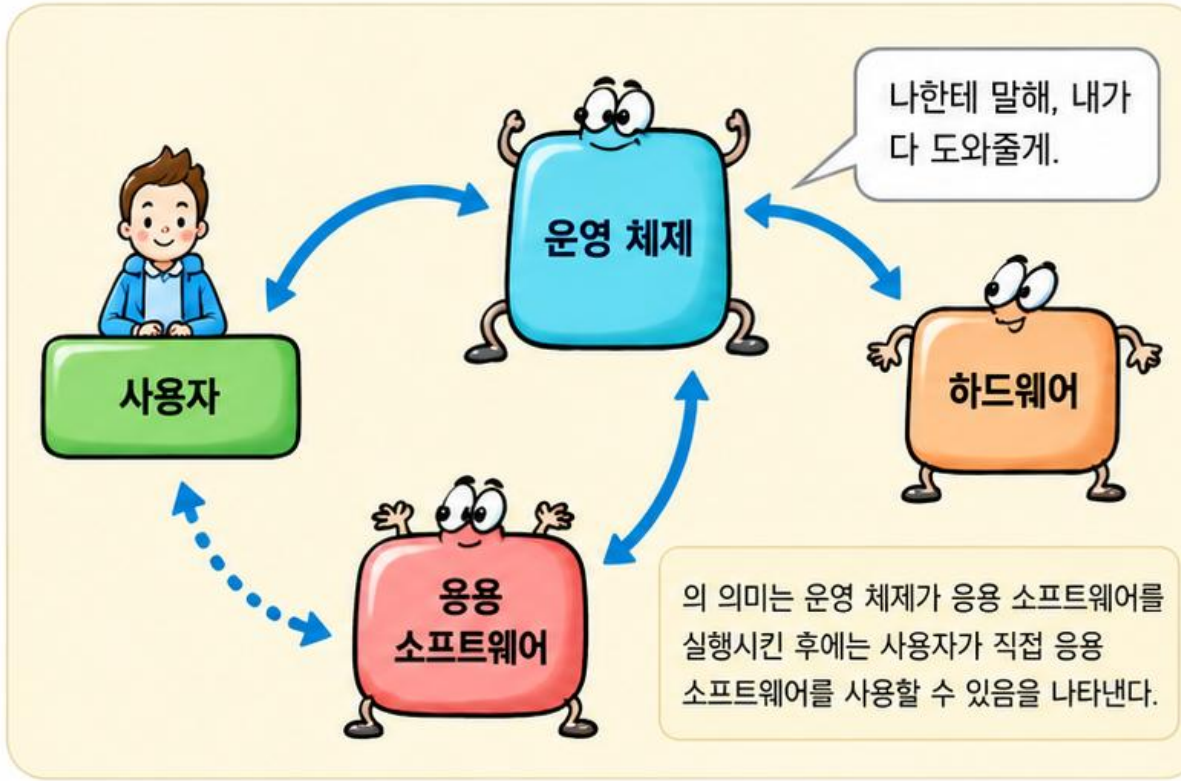
운영 체제 - Operation System



장점	친숙함 초기에 쉬움	모듈 설치 쉬움 무료, 참고문서 많음	모듈 설치 쉬움 참고문서 많음
단점	모듈 설치 어려움 / 유료	OS 자체 사용이 어려움	비쌈

운영 체제 - Operation System

하드웨어 vs 소프트웨어



하드웨어

- 컴퓨터 본체, 모니터, 마우스, 키보드, 하드디스크, CPU, RAM 등

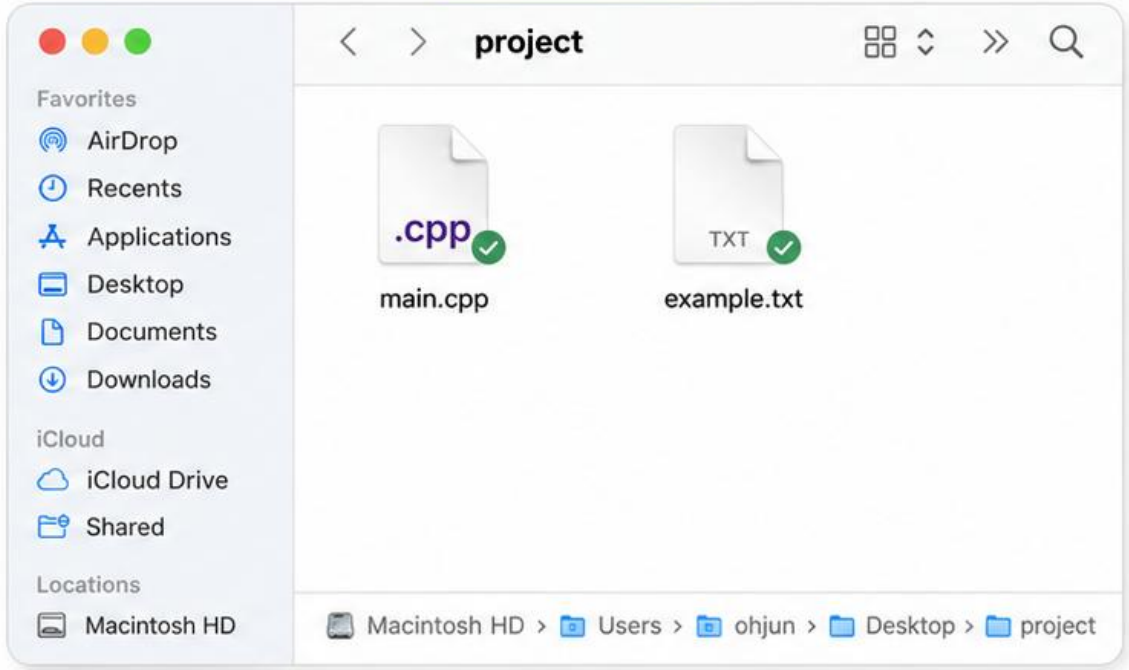
소프트웨어

- 한글, 엑셀, 파워포인트, 크롬 등

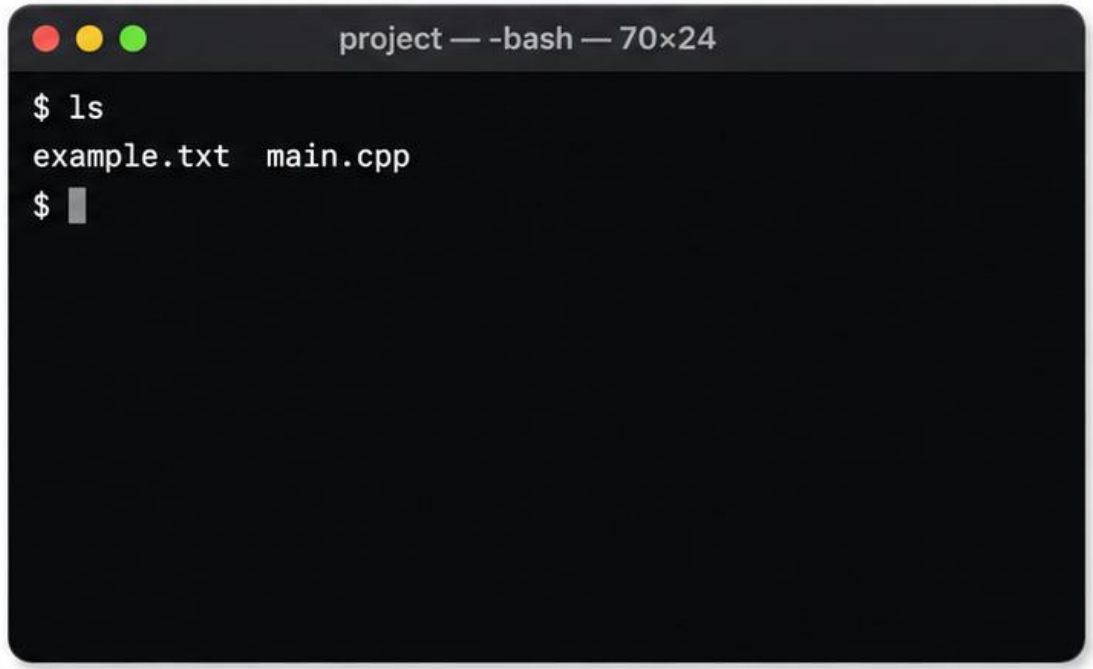
Terminal

- 터미널 - Mouse가 아닌 키보드로 명령을 입력 프로그램 실행

GUI



CLI



Graphical User Interface

마우스와 아이콘 등을 사용하여
직관적으로 작업을 수행하는 환경

Command Line Interface

명령어를 직접 입력하여
컴퓨터와 상호작용하는 환경

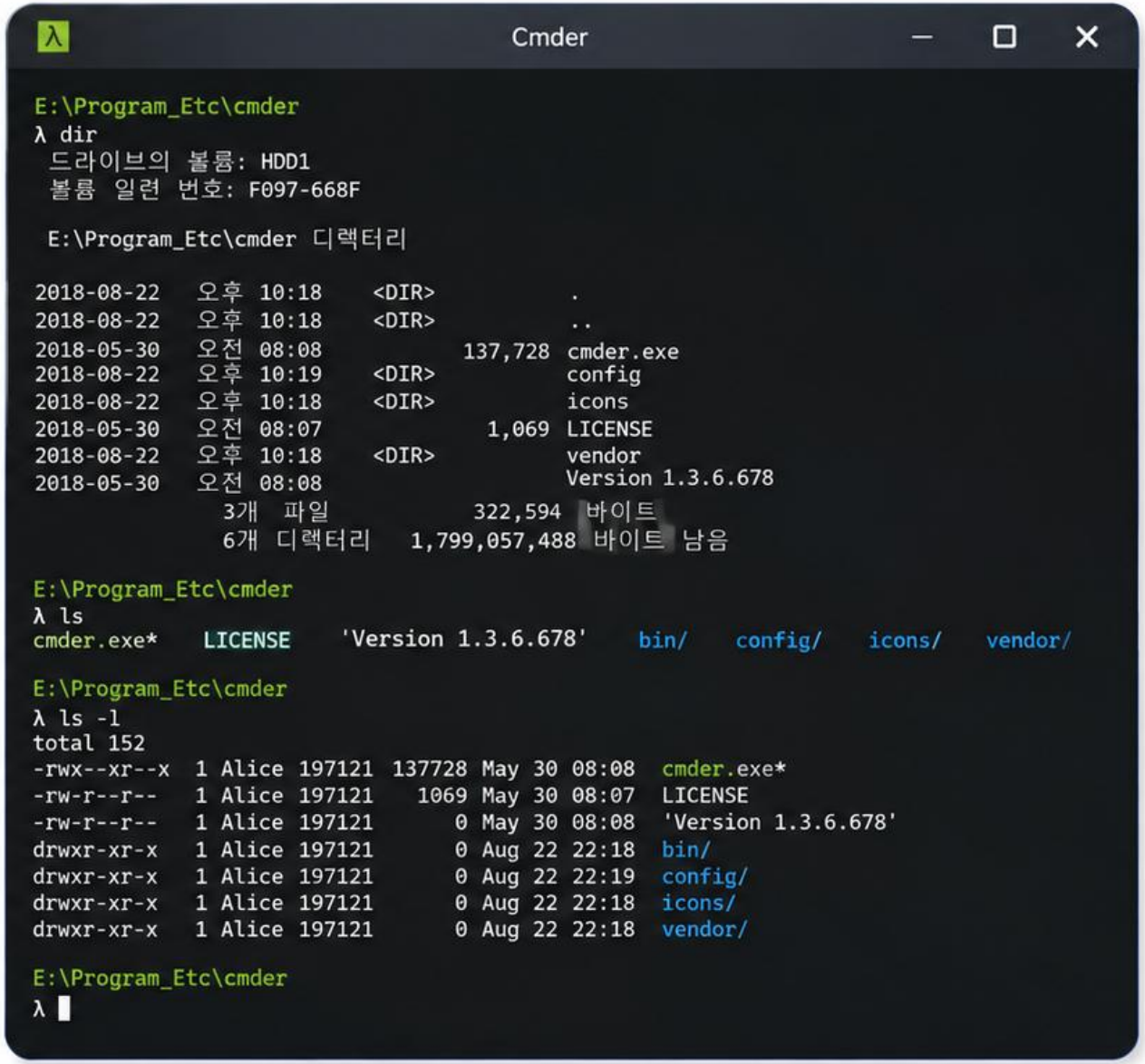
Command Line Interface

Graphic User Interface(GUI)와 달리 Text를 사용하여
컴퓨터에 명령을 입력하는 인터페이스 체계

Windows
- CMD Window, Windows Terminal

Mac, Linux
- Terminal

원도우 cmder도 권장
https://cmder.net/



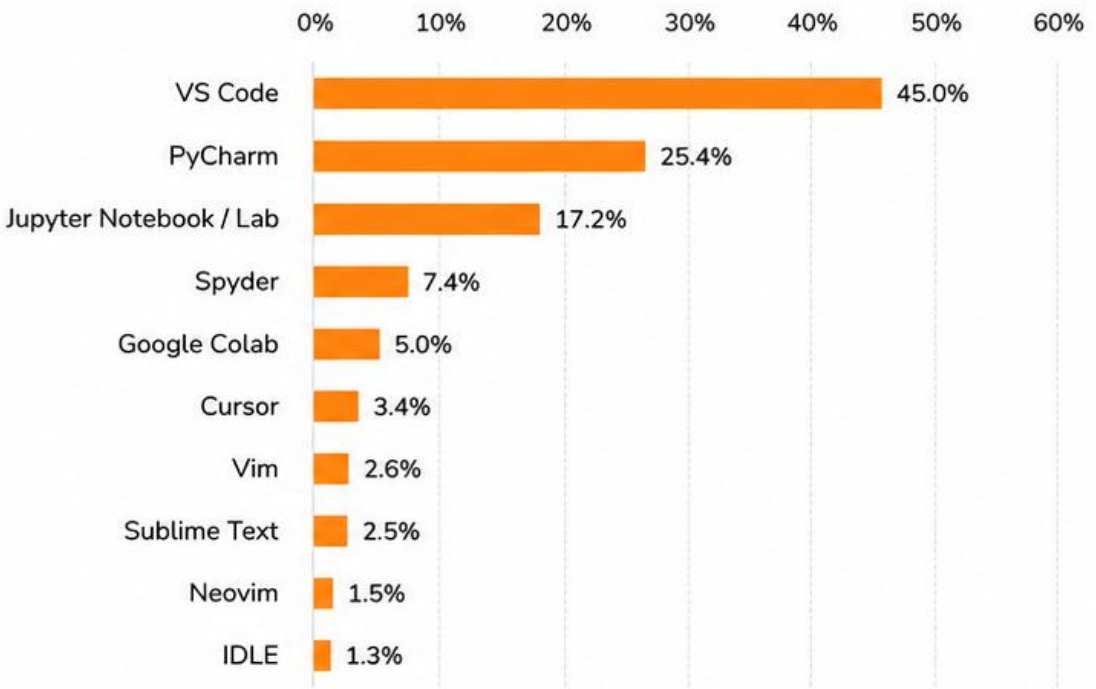
코드 편집기(Editor)

- **파이썬 코드도 일종의 문서**
  - 한글, 워드처럼 코드를 입력할 문서 편집기가 필요함.
  - text 타입의 문서를 저장하는 모든 편집기를 사용가능.

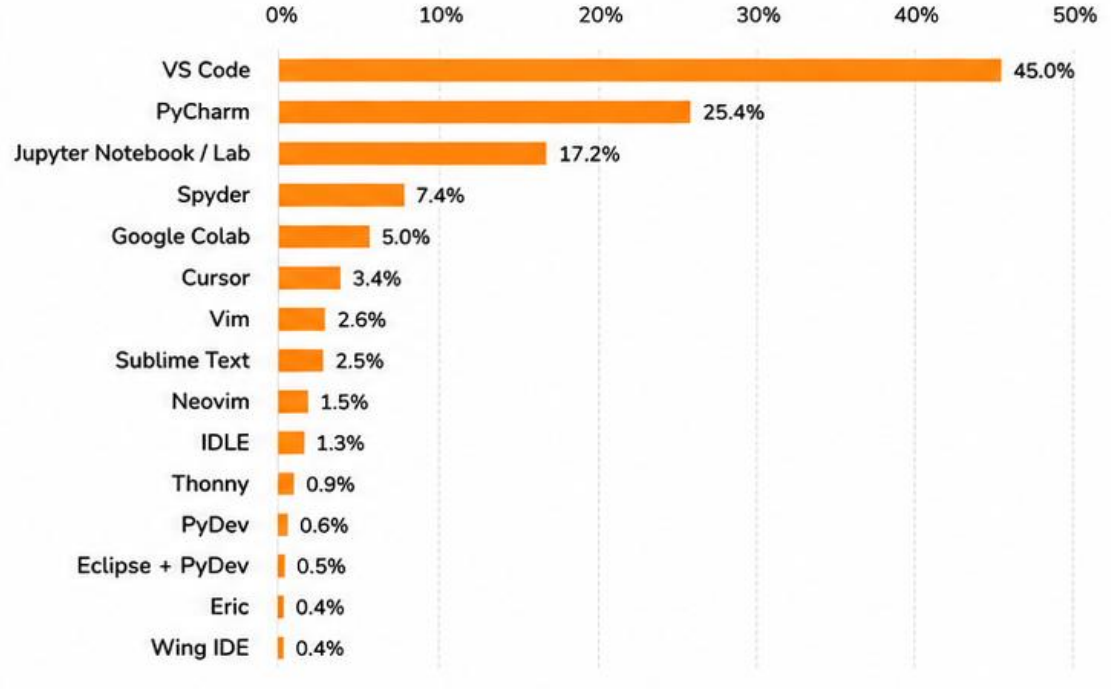
종류	설명	다운로드
메모장	원도우의 기본 문서 편집도구	시작 -> 메모장(Windows)
VI editor	리눅스의 기본 문서 편집도구	https://www.vim.org/download.php
Sublime Text	프로그래밍에 특화된 문서 편집도구	https://www.sublimetext.com/
Atom		https://atom.io/
VS code		https://code.visualstudio.com/
Pycharm	다양한 기능을 갖춘 파이썬 전용 개발 도구	https://www.jetbrains.com/pycharm/

파이썬 개발환경

2025년 기준, 파이썬 개발자들이 가장 많이 사용하는 IDE 및 에디터



※ Source: JetBrains Developer Ecosystem Survey 2025



※ Source: JetBrains Developer Ecosystem Survey 2025

VS Code가 가장 높은 점유율을 차지하고 있으며, PyCharm과 Jupyter Notebook이 그 뒤를 잇고 있습니다.

파이썬 개발환경

- 손쉬운 패키지 설치
- 가상 환경을 통한 패키지 버전관리가 용이

파이썬 개발환경

- web을 통한 접근 가능
- Coding 결과를 실시간으로 확인 가능
- 자동완성 기능
- 다양한 언어 지원
- Markdown을 통해 문서화 가능

ANACONDA 설치

https://www.anaconda.com/products/individual

Anaconda Installers

Windows

Python 3.8

64-Bit Graphical Installer (466 MB)

32-Bit Graphical Installer (397 MB)

MacOS

Python 3.8

64-Bit Graphical Installer (462 MB)

64-Bit Command Line Installer (454 MB)

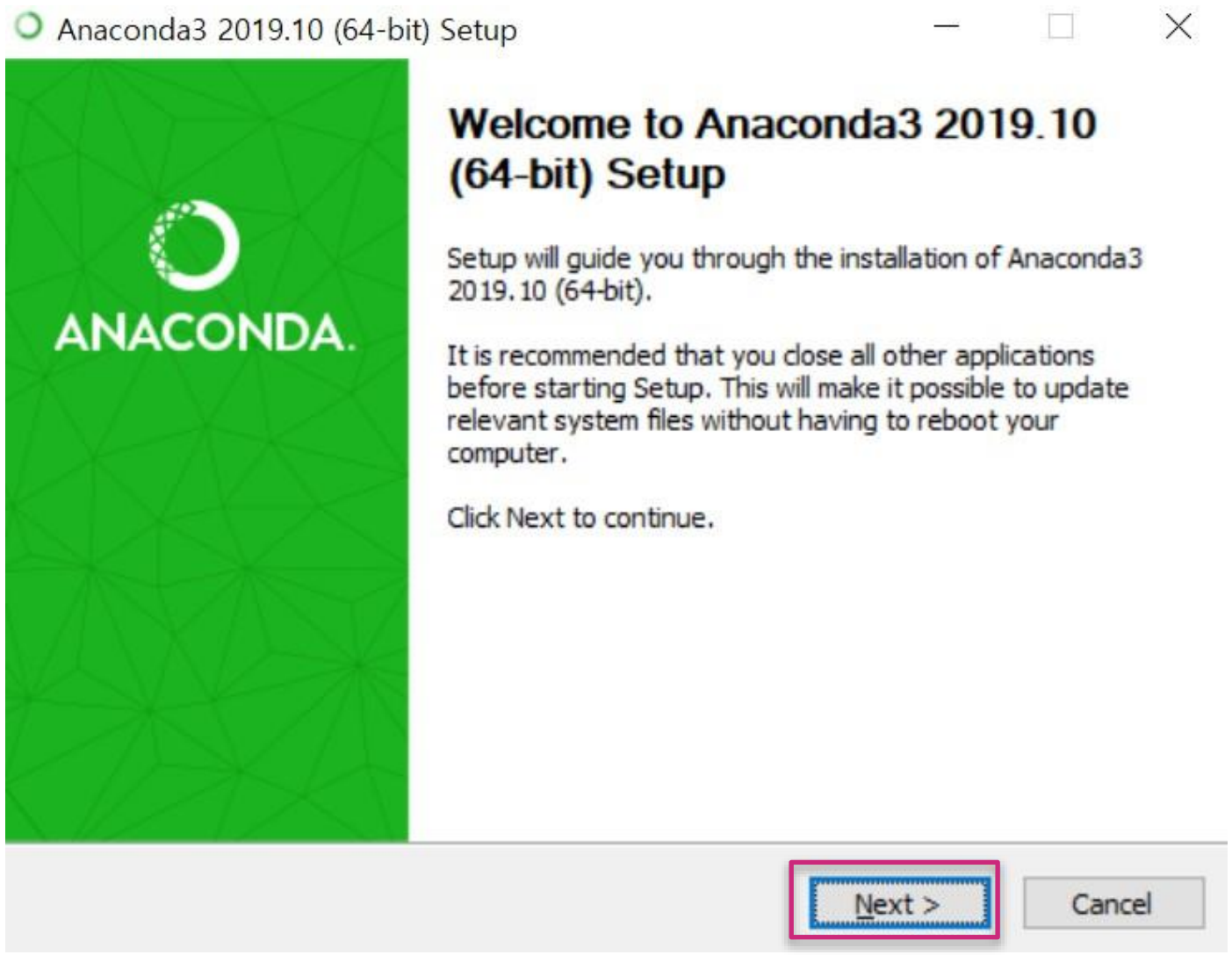
Linux

Python 3.8

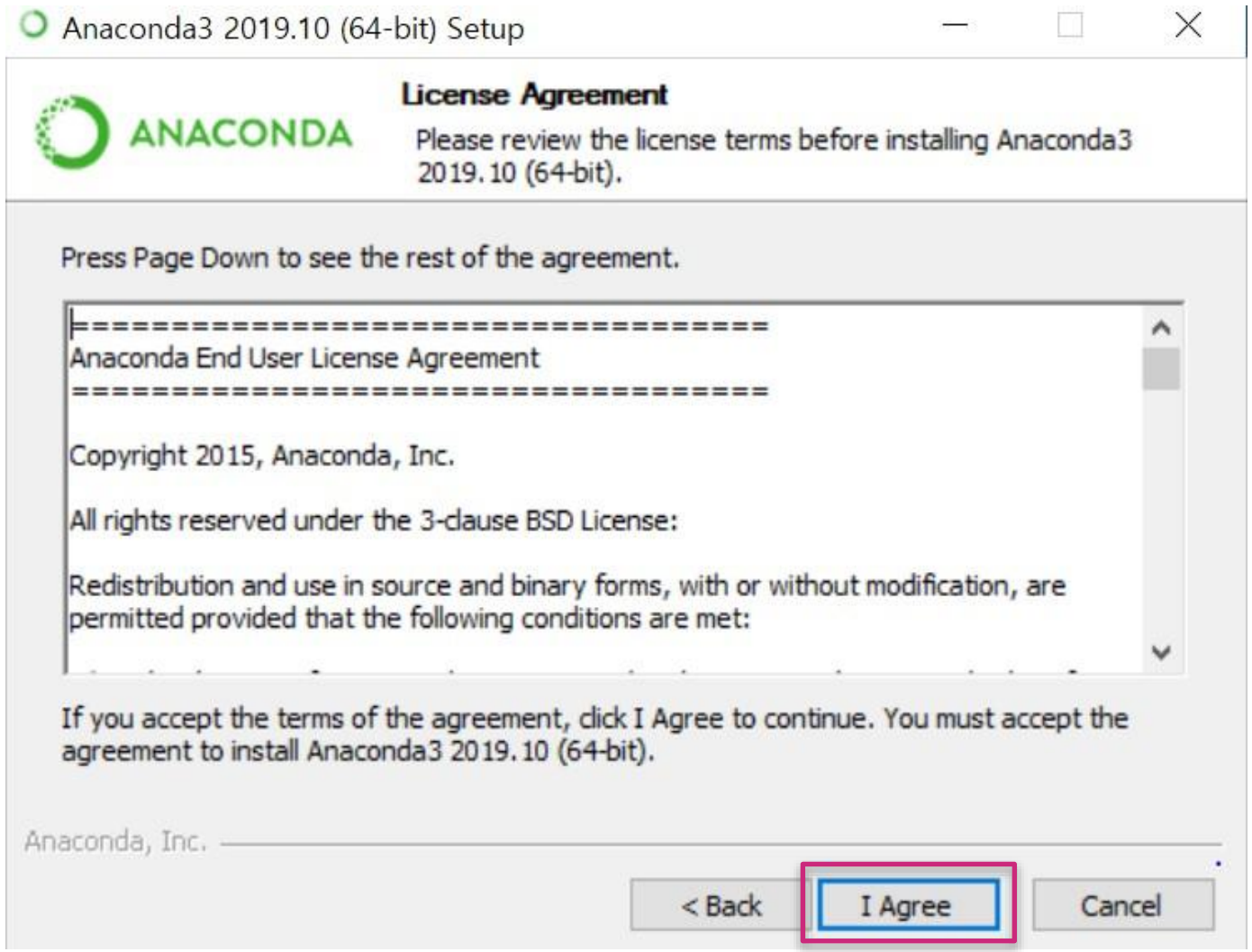
64-Bit (x86) Installer (550 MB)

64-Bit (Power8 and Power9) Installer (290 MB)

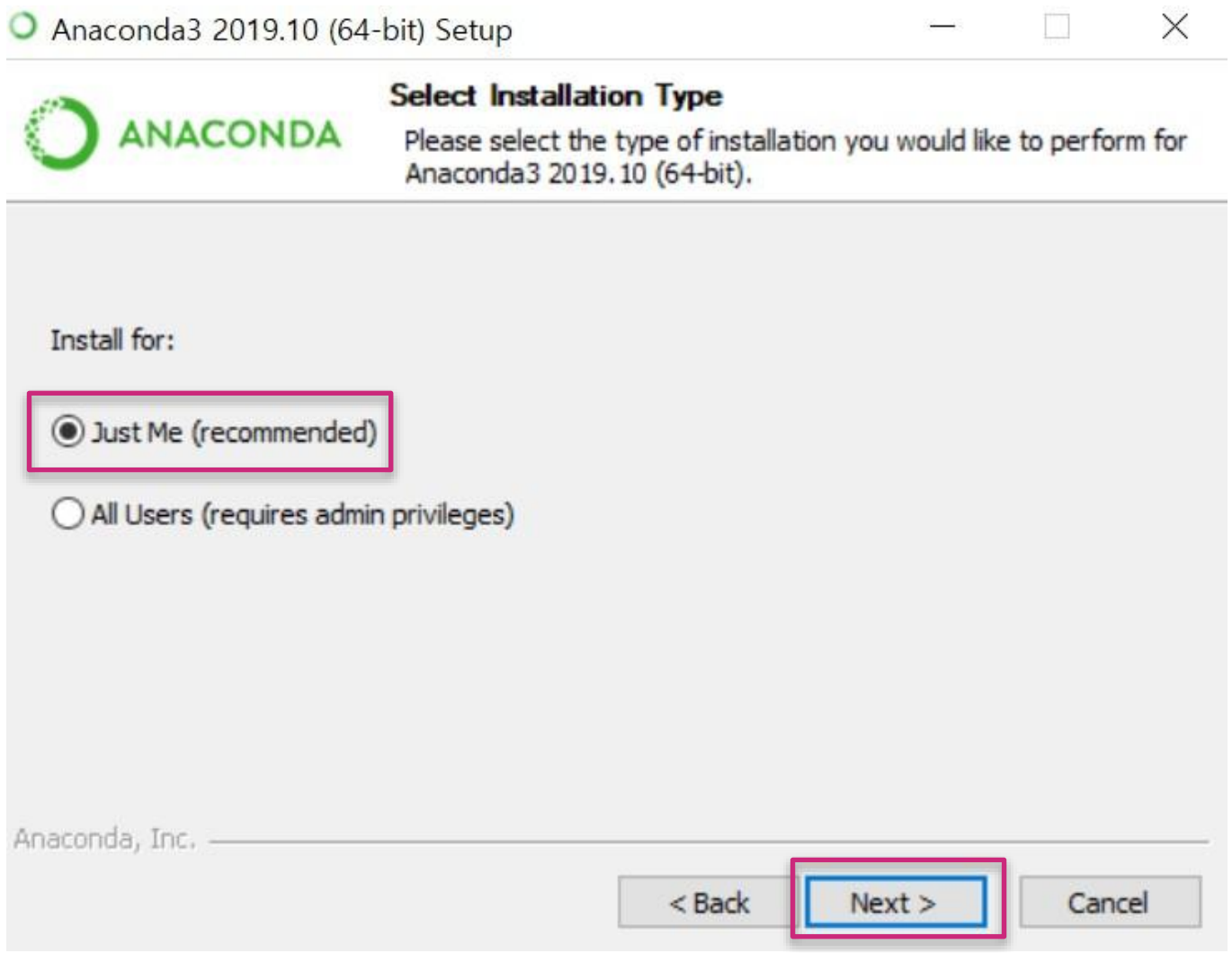
ANACONDA 설치


Anaconda3 2019.10 (64-bit) Setup
ANACONDA.
Welcome to Anaconda3 2019.10 (64-bit) Setup
Setup will guide you through the installation of Anaconda3 2019.10 (64-bit).
It is recommended that you close all other applications before starting Setup. This will make it possible to update relevant system files without having to reboot your computer.
Click Next to continue.
Next >
Cancel

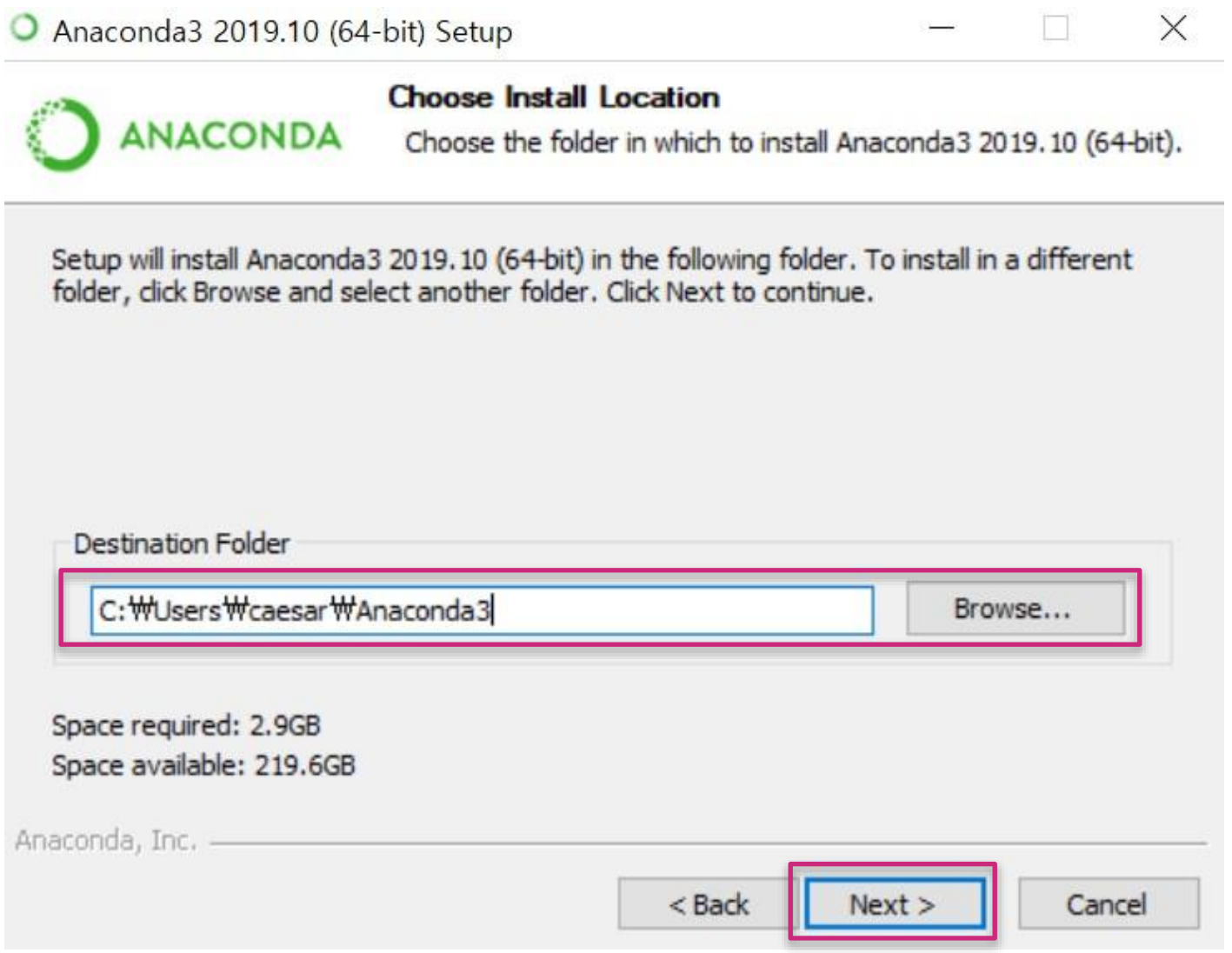
ANACONDA 설치


Anaconda3 2019.10 (64-bit) Setup
ANACONDA
License Agreement
Please review the license terms before installing Anaconda3 2019.10 (64-bit).
Press Page Down to see the rest of the agreement.
Anaconda End User License Agreement
Copyright 2015, Anaconda, Inc.
All rights reserved under the 3-clause BSD License:
Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:
If you accept the terms of the agreement, click I Agree to continue. You must accept the agreement to install Anaconda3 2019.10 (64-bit).
Anaconda, Inc.
< Back
I Agree
Cancel

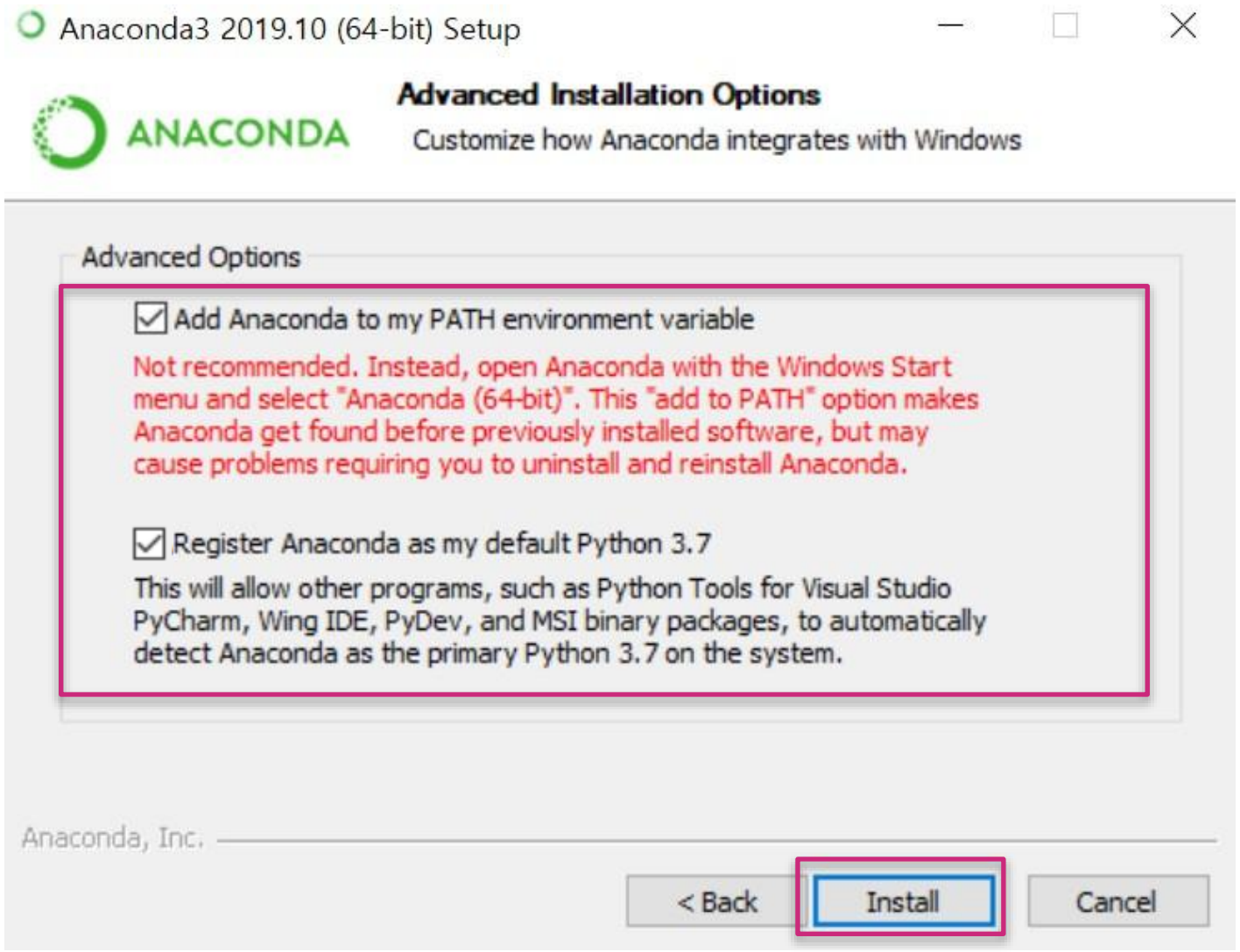
ANACONDA 설치


Anaconda3 2019.10 (64-bit) Setup
ANACONDA
Select Installation Type
Please select the type of installation you would like to perform for Anaconda3 2019.10 (64-bit).
Install for:
Just Me (recommended)
All Users (requires admin privileges)
Anaconda, Inc.
< Back
Next >
Cancel

ANACONDA 설치

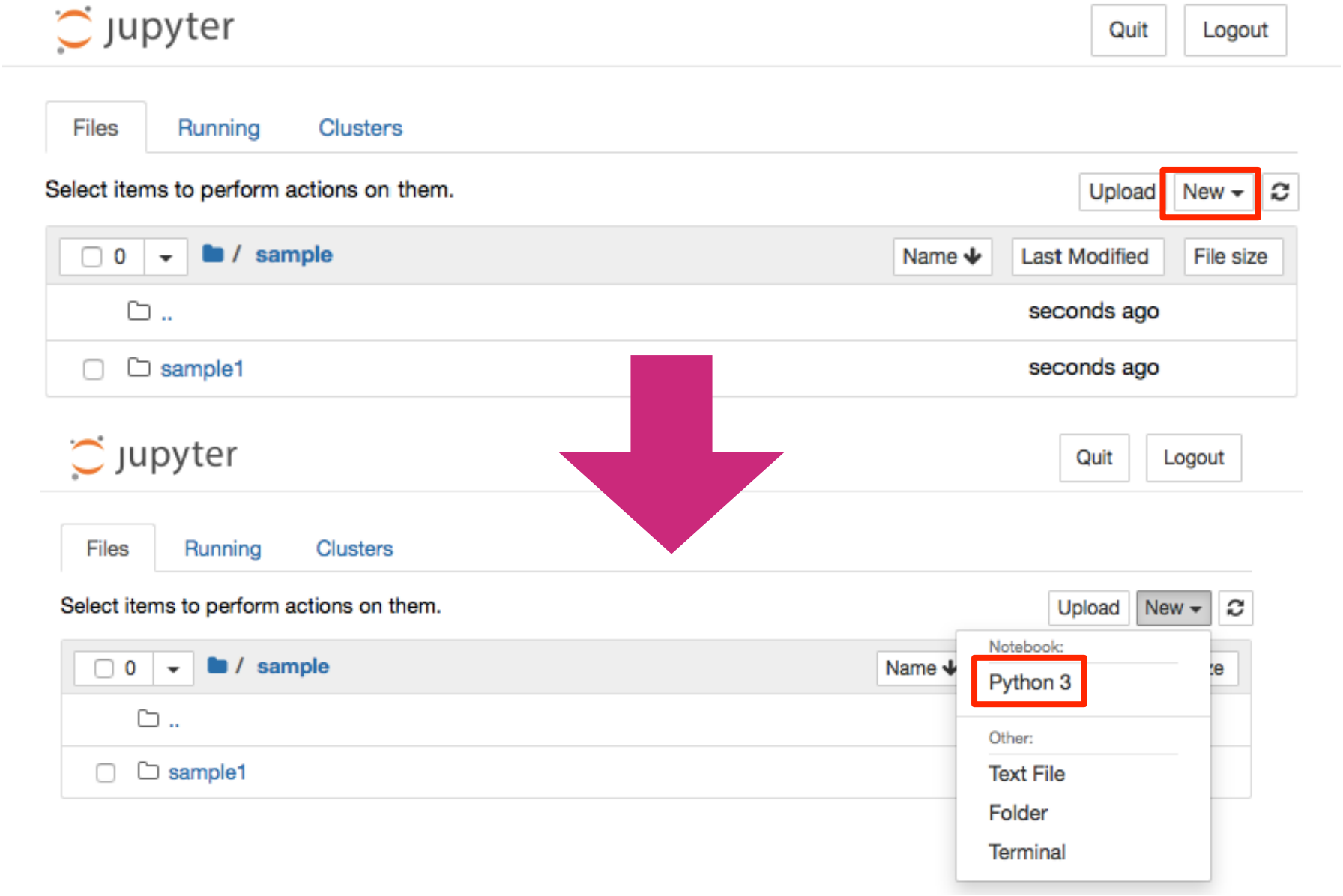

Anaconda3 2019.10 (64-bit) Setup
ANACONDA
Choose Install Location
Choose the folder in which to install Anaconda3 2019.10 (64-bit).
Setup will install Anaconda3 2019.10 (64-bit) in the following folder. To install in a different folder, click Browse and select another folder. Click Next to continue.
Destination Folder
C:₩Users₩caesar₩Anaconda3
Browse...
Space required: 2.9GB
Space available: 219.6GB
Anaconda, Inc.
< Back
Next >
Cancel

ANACONDA 설치


Anaconda3 2019.10 (64-bit) Setup
ANACONDA
Advanced Installation Options
Customize how Anaconda integrates with Windows
Advanced Options
Add Anaconda to my PATH environment variable
Not recommended. Instead, open Anaconda with the Windows Start menu and select "Anaconda (64-bit)". This "add to PATH" option makes Anaconda get found before previously installed software, but may cause problems requiring you to uninstall and reinstall Anaconda.
Register Anaconda as my default Python 3.7
This will allow other programs, such as Python Tools for Visual Studio PyCharm, Wing IDE, PyDev, and MSI binary packages, to automatically detect Anaconda as the primary Python 3.7 on the system.
Anaconda, Inc.
< Back
Install
Cancel

Jupyter Notebook 사용하기

Jupyter Notebook 사용하기



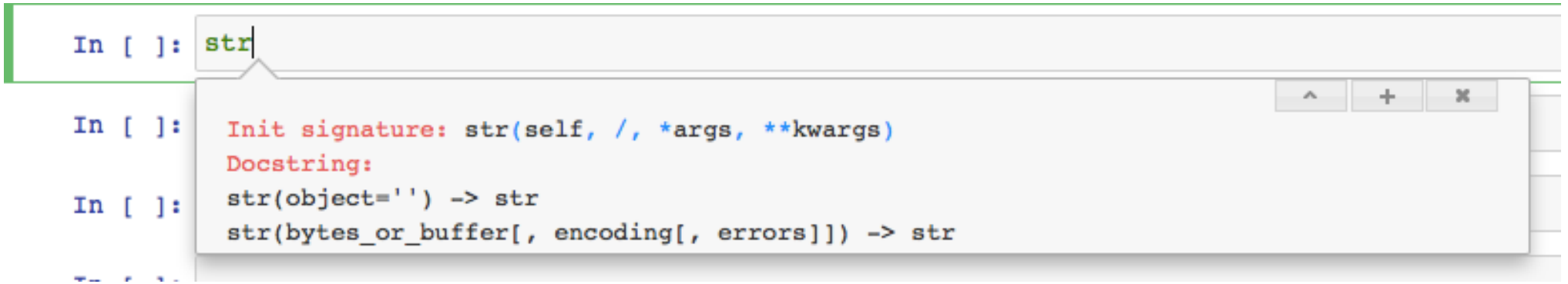
Jupyter Notebook 사용하기

유용한 단축키

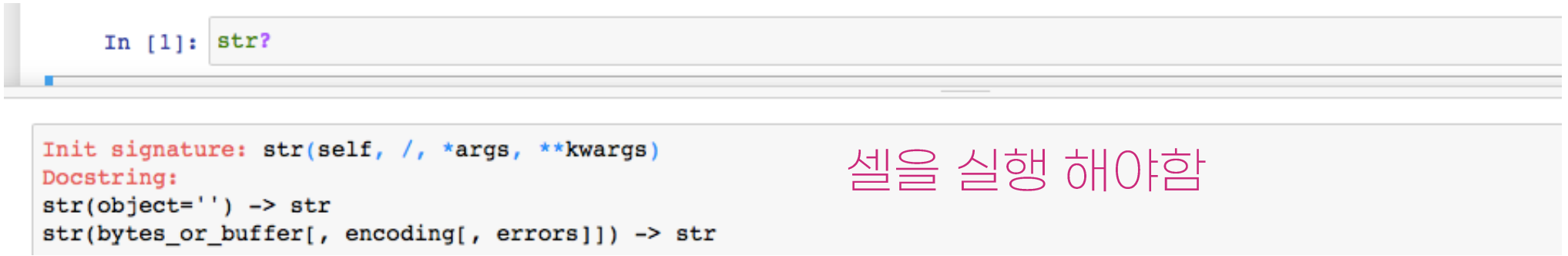
```
In [ ]: # Edit mode : Enter
        # Command mode : ESC

In [ ]: # 셀 추가 (Command mode에서)
        # 현재 셀 위로 : a
        # 현재 셀 아래로 : b

In [ ]: # 셀 삭제 (Command mode에서) : dd

In [ ]: # 선택한 셀을
        # 복사하기 : c
        # 잘라내기 : x

In [ ]: # 셀 붙여넣기
        # 선택한 셀 위로 : Shift + v
        # 선택한 셀 아래로 : v

In [ ]: # 셀에 라인 추가하기 (Command mode에서) : l
```

Jupyter Notebook 사용하기

- 함수나 변수에 대한 설명 1) : Shift + Tab



- 함수나 변수에 대한 설명 2) : ?



셀을 실행 해야함

Jupyter Notebook 사용하기

Jupyter 맛보기

markdown

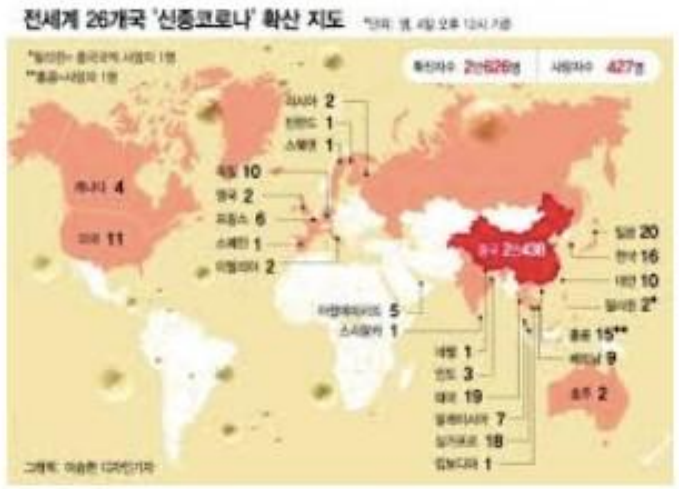
문서처럼 작성할 수도 있습니다.

In [2]:
```
print("당연히 코딩도 할 수 있습니다.")
```

당연히 코딩도 할 수 있습니다.

In [7]:
```
from IPython.display import Image
Image(url="
```

Out[7]:



Google Colab 사용법

1. Google Colab이란?

풀 네임은 Google Colaboratory(Google Colab)

2. Google Drive + Jupyter Notebook

- Google Drive처럼 협업 가능(동시에 수정 가능)
- https://colab.research.google.com/ 로 접속 시 사용 가능

3. 컴퓨터 사양 (2025년 기준)

OS	Ubuntu 20.04 LTS
CPU	2 vCPU (Xeon 계열), 2.2 ~ 2.3GHz 수준
메모리	약 13GB
GPU	T4, V100, L4 등 무작위 제공
GPU/TPU	무료로 사용 가능 (보장되지 않고 자원 상황에 따라 제한)

4. 세션 시간

세션 최대 약 12시간

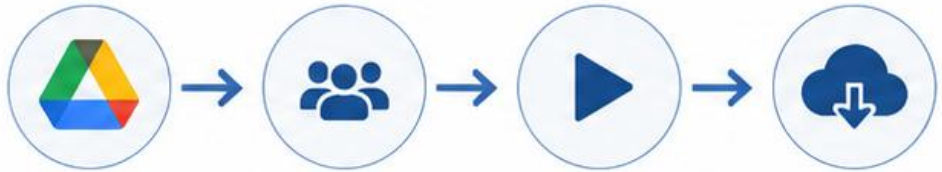
(유휴 시 더 빨리 종료될 수 있음)

5. 요금제

pro/pro+ 요금제 사용 시

더 긴 세션, 더 강력한 GPU 이용 가능

Google Colab 사용 흐름



1. Google Drive에서 새 노트북 생성
2. 실시간으로 협업 및 수정
3. 코드 실행
4. 결과 저장 및 공유

TIP

무료로 강력한 GPU/TPU를 사용할 수 있어 머신러닝, 딥러닝 학습에 매우 유용합니다.

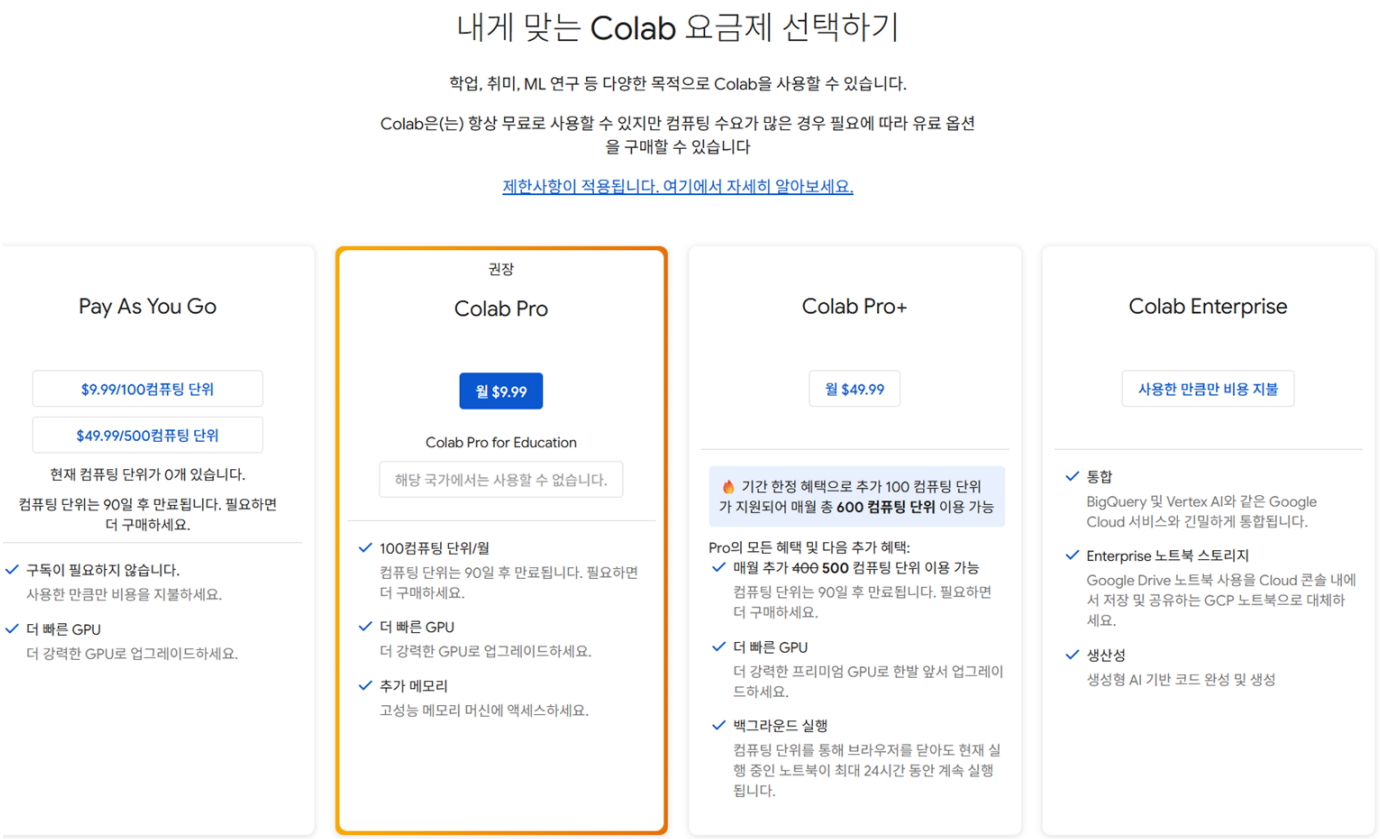
Google Colab

OS 확인

- !cat /etc/issue.net

하드웨어 사양

1. CPU
   - !cat /proc/cpuinfo
2. Memory
   - !cat /proc/meminfo
3. Disk
   - !df -h
4. GPU
   - !nvidia-smi



Google Colab 런타임 방지

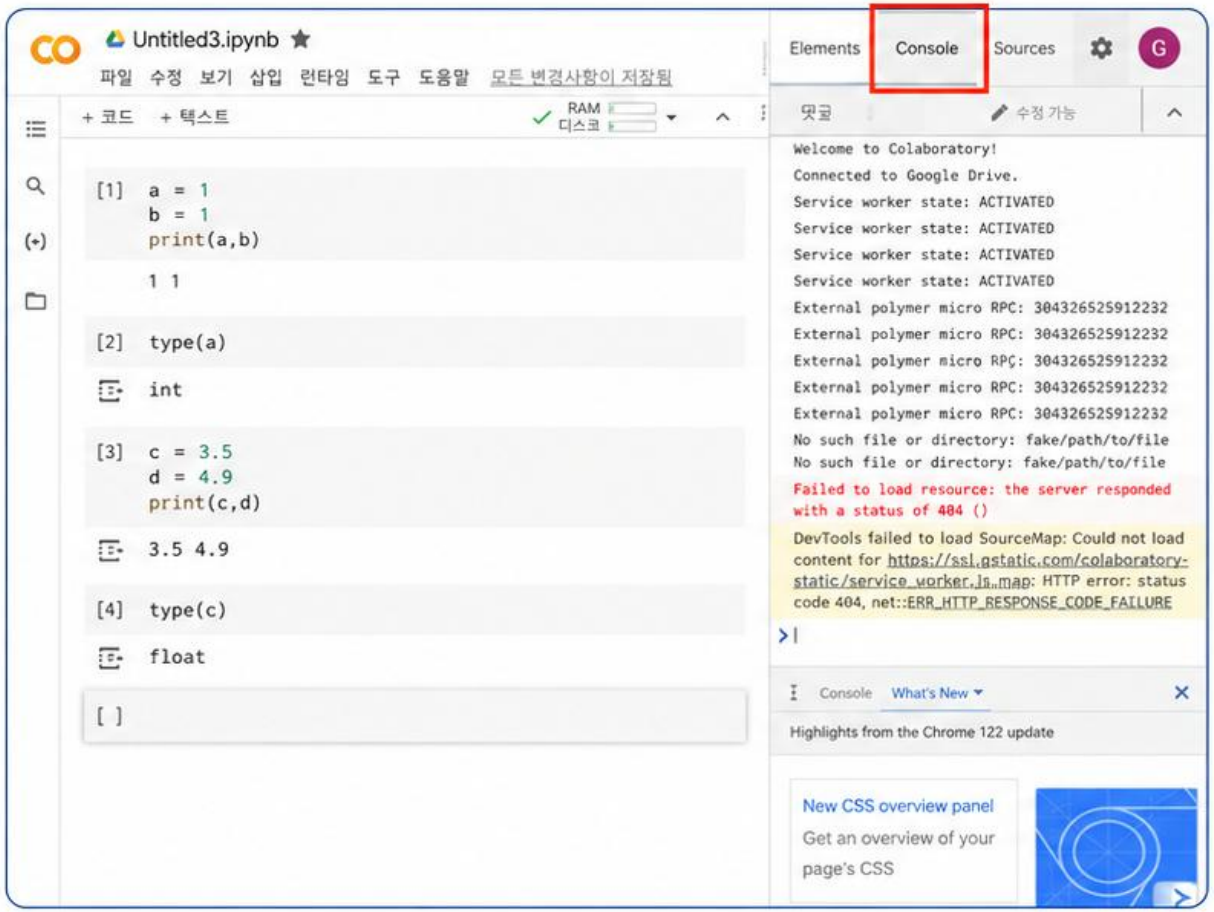
Google Colab의 전체 **세션 유지 시간은 12시간**이고, 90분 이상 비 활성화 되어 있으면 자동으로 끊기고,
몇 분 동안 사용을 안하면 자동으로 세션이 끊긴다.

방법

크롬 브라우저에서 F12(혹은 Ctrl+shift+I)를 눌러 개발자 도구 창을 열고
console 창에서 아래 코드를 입력해 주면 된다.

```
function ClickConnect(){
    var buttons
    =  document.querySelectorAll("colab-
    dialog.yes-no-dialog paper-button#cancel");
    buttons.forEach(function(btn){ btn.click(); });
    console.log("1분마다 자동 재연결");
    document.querySelector("colab-toolbar-button#connect").click(); }
setInterval(ClickConnect,1000*60);
```

현재는 되지 않음(2024.02)



Python IDE VS code 이란?

VS Code의 장점

1. MS의 친절한 관리

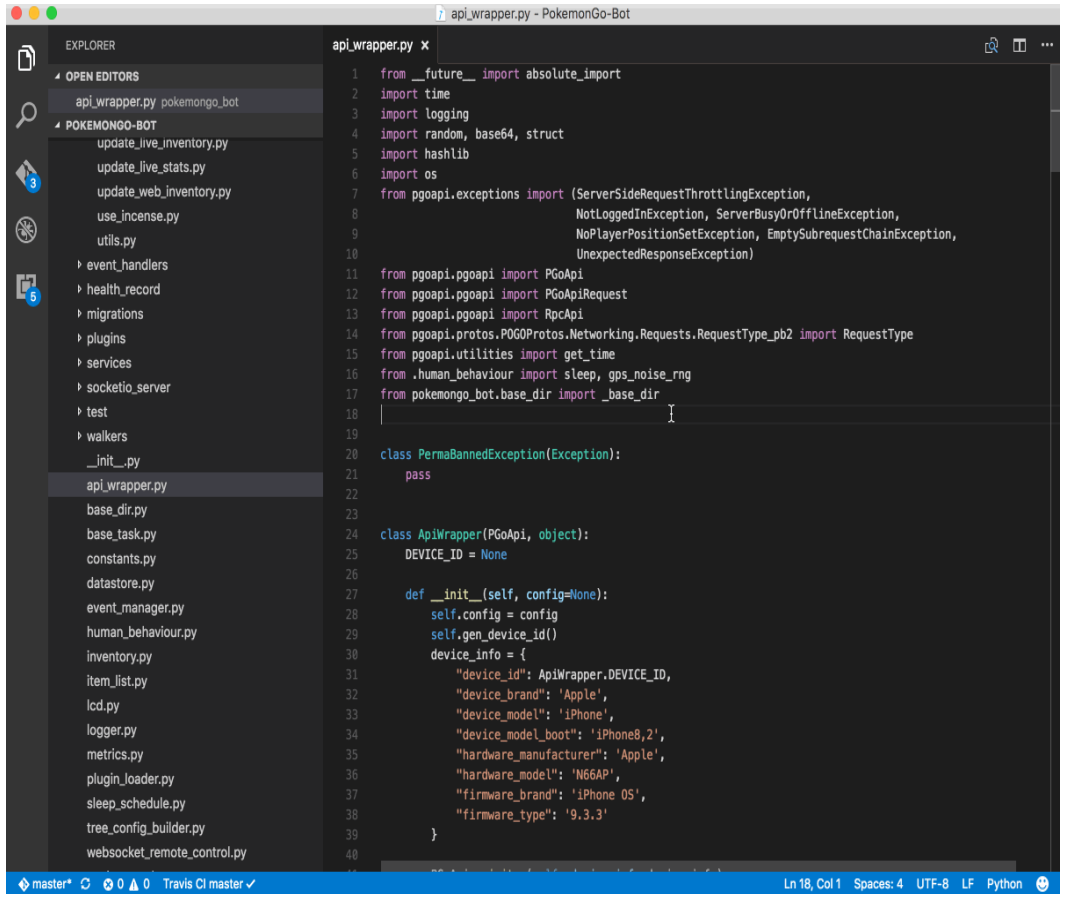
   - MS의 프로그램은 비싸지만 그만큼 친절함.

2. 무료 정책

3. 점유율 1위

4. 마이크로 소프트가 직접 배포하는 확장성

5. 언어 확장성이 좋음(C,C++,JAVA 등 지원함)

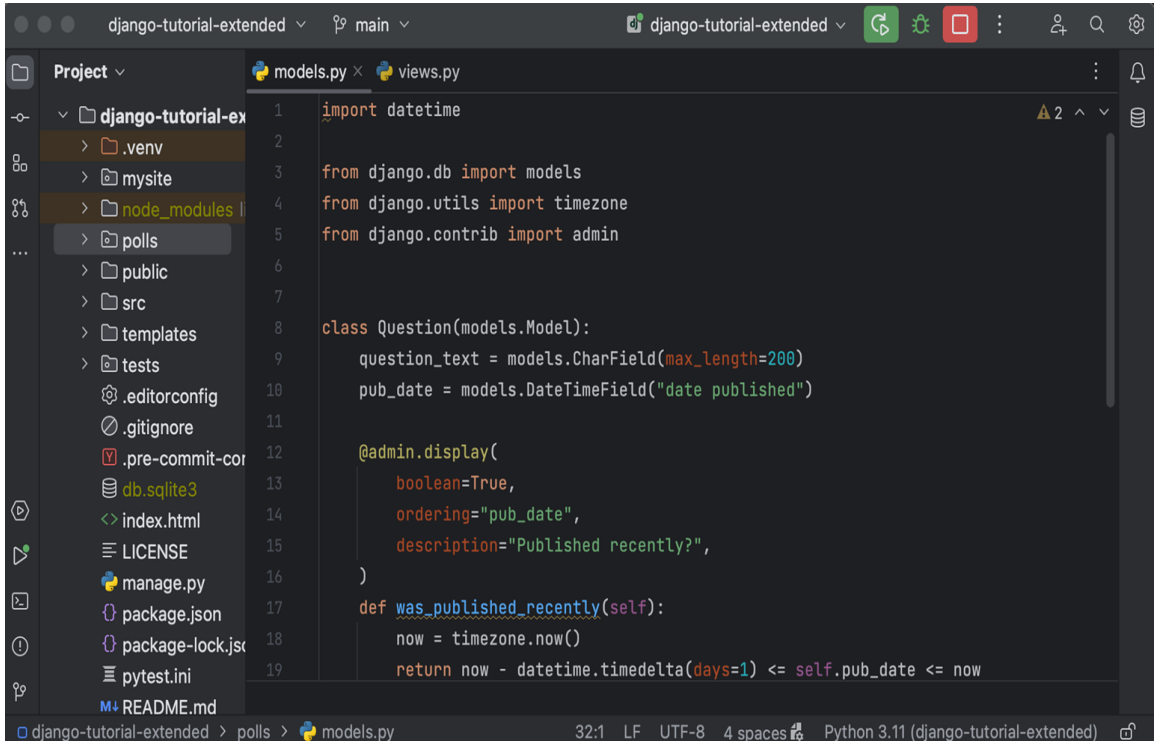


Python IDE Pycharm 이란?

- Pycharm은 Python 언어에 사용되는 JetBrains사에 의해 개발된 IDE입니다.
- IDE는 컴파일, 코드 편집기,디버그, 배포 등의 모든 작업을 하나의 프로그램에서 할 수 있도록 해주는 통합 개발 환경

Pycharm 의 특징

1. 코드 분석 및 코딩 지원
2. 프로젝트 및 코드 탐색
3. 파이썬
4. 웹 프레임 워크 지원(Professional 전용, 유료버전)
5. 통합 파이썬 디버그
6. 라인 단위 테스트
7. Google App Engine python(Professional 전용, 유료버전)
8. 과학적 도구 지원(Professional 전용, 유료버전)



Python 속도

“2~5배 더 빠르게”… 귀도 반 로섬, 파이썬 속도 개선한다

Paul Krill | InfoWorld

파이썬(Python) 창시자 귀도 반 로섬이 파이콘 2021(Pycon 2021)에서 진행된 파이썬 랭귀지 서밋(Python Language Summit)에서 파이썬 속도를 2배에서 5배까지 더 빠르게 만들기 위한 단기 및 장기 계획을 발표했다.

ⓒGetty Images

파이썬에는 이미 파이파이(PyPy)와 같은 대체 런타임부터 C/C++로 작성된 랩핑 모듈까지 더 빠르게 실행할 수 있는 여러 방법이 있다. 하지만 C로 작성된 파이썬 참조 구현체이자 가장 널리 사용되는 언어 버전인 C파이썬(CPython) 자체의 속도를 높이는 방법은 거의 없다는 게 반 로섬의 설명이다.

출처 : https://www.ciokorea.com/news/193909

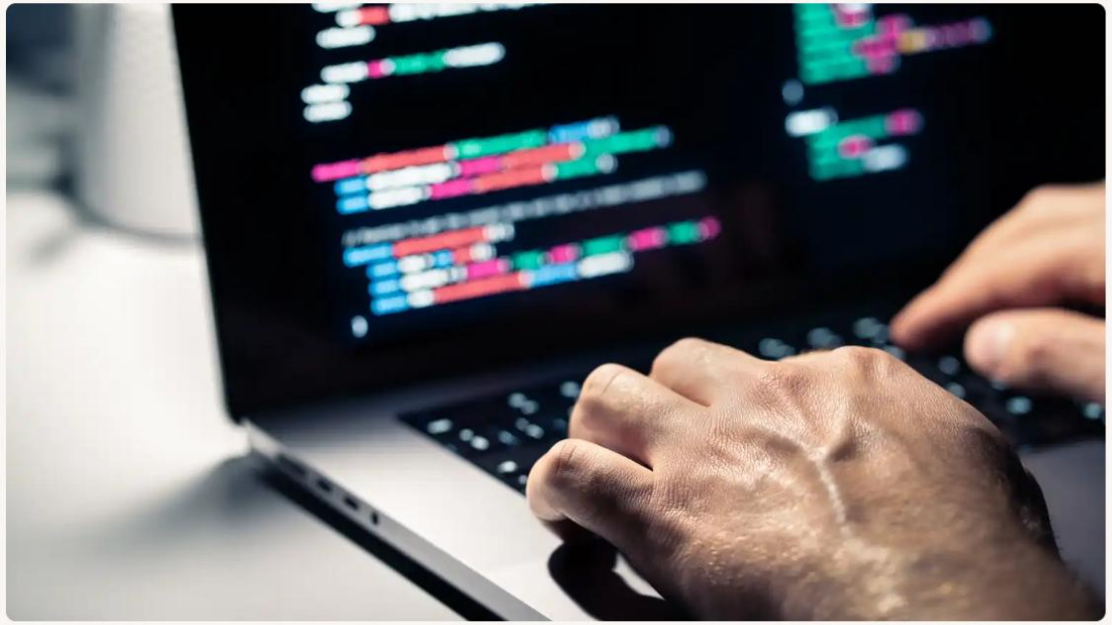
Python 속도

C언어와 속도 격차 좁힌다… 파이썬 새 버전, 인터프리터 개선해 30% 성능↑

뉴스
2025.02.14 • 2분

개발자 프로그래밍 언어

올해 말 출시 예정인 파이썬 3.14에 새로운 인터프리터가 추가되면서, 기존 코드 수정 없이도 최대 30% 성능 향상이 가능해진다. 첫 번째 베타 버전은 2025년 5월 공개될 예정이다.



CREDIT: TERO VESALAINEN / SHUTTERSTOCK

파이썬 재단의 공식 문서에 따르면, 파이썬 3.14 버전에는 '꼬리 호출(tail calls) 기반의 새로운 인터프리터'가 탑재된다. 사실 이 문구는 파이썬 개발 과정을 잘 모르는 사람들에게는 다소 오해를 불러일으킬 수 있는 표현이다. 여기서 말하는 '꼬리 호출'은 CPython 또는 파이썬 언어 자체가 꼬리 호출 최적화를 지원한다는 의미가 아니다. 그보다는 C 컴파일러가 CPython 코드에서 수행하는 최적화 기법으로, 인터프리터가 바이트코드를 실행하는 과정을 최적화해 전체 실행 속도를 높인다.

관련 콘텐츠

뉴스

출처 : https://www.cio.com/article/3824874/c%EC%96%B8%EC%96%B4%EC%99%80-%EC%86%8D%EB%8F%84-%EA%B2%A9%EC%B0%A8-%EC%A2%81%ED%9E%8C%EB%8B%A4%C2%B7%C2%B7%C2%B7-%ED%8C%8C%EC%9D%B4%EC%8D%AC-%EC%83%88-%EB%B2%84%EC%A0%84-%EC%9D%B8%ED%84%B0%ED%94%84.html

Python in HTML

출처 : https://www.youtube.com/watch?v=3DuyJf_XPtM

프론트 엔드 VS 백엔드

	프론트엔드	백엔드
정의	사용자들에게 보여지는 부분 Ex) 레이아웃, 텍스트, 그림 등	사용자들에게 보이지 않는 시스템 안쪽 부분 Ex) 내부적 DB
기술	**HTML&CSS** *HTML : 콘텐츠에 구조와 의미를 주기 위한 것 *CSS : 콘텐츠의 모양과 스타일을 주기 위한 것	**스프링프레임워크** *동적인 웹사이트 개발을 위한 여러가지 서비스 제공
	JAVA script *웹페이지에 기능을 더해 HTML 웹페이지를 동적이고 살아있게 만드는 것	**JAVA** *컴퓨터가 이해할 수 있는 0과 1로 이루어진 기계어
직무	**UI/UX 디자인** *UX디자인 : 사용자가 경험하게 될 전반적 디자인 *UI디자인 : 겉으로 시각화 되는 디자인	**서버코딩** *시스템이 운용되기 위해 필요한 시스템의 관련 기능 구현 등을 담당
	Front-end 개발 *웹상의 표현을 도와주는 여러 기술을 활용하여 UX와 Web 디자인 요소를 구현	**DB 활용** *데이터베이스를 구축하고 운영

출처 : https://www.jobindexworld.com/contents/view/4270 / https://www.facebook.com/coding.comu/posts/372409044728656

파이썬의 기본

- 산술 연산자

1. 더하기 : +

```
[ ] 1 + 2
```

3

```
[ ] 250.7 + 300.2
```

550.9

2. 빼기 : -

```
[ ] 5 - 3
```

2

```
[ ] 250.5 - 100.3
```

150.2

3. 곱하기 : *

```
[ ] 5 * 3
```

15

```
[ ] 20.5 * 2
```

41.0

4. 나누기 : /

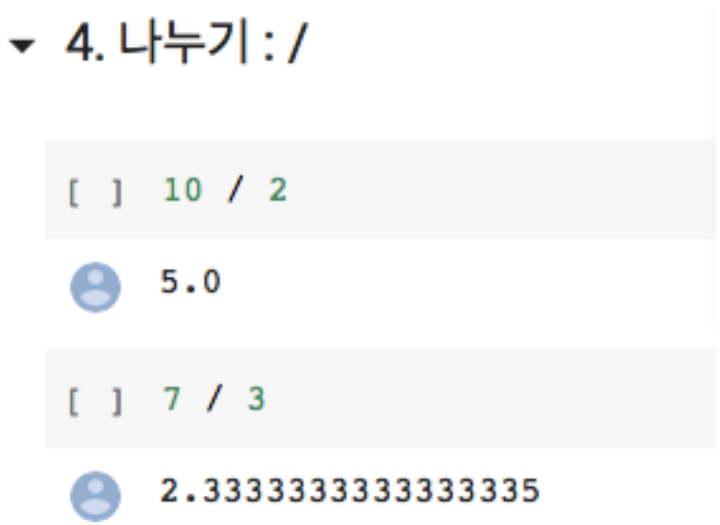
```
[ ] 10 / 2
```

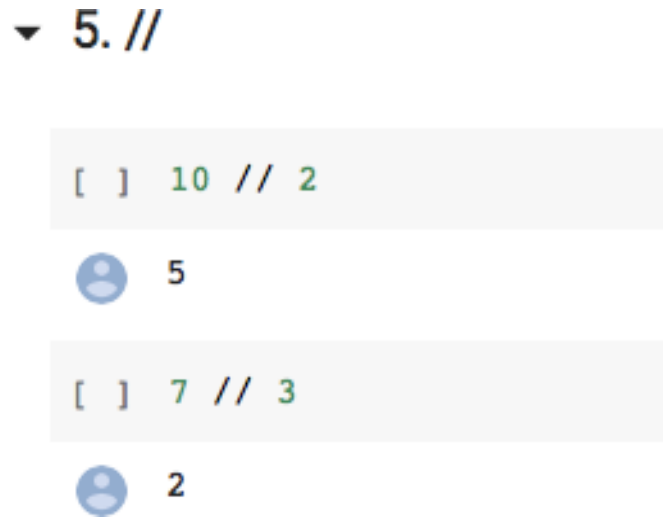
5.0

```
[ ] 7 / 3
```

2.3333333333333335

- 산술 연산자





/의 연산 결과 : 소수점 표시 → 실수형

//의 연산 결과 : 소수점 없음(나누기의 몫) → 정수형

- 산술 연산자

6. 나머지 : %

```
[ ]  10 % 2
```

0

```
[ ]  7 % 3
```

1

7. 그 밖에 연산자

제곱 : **

```
[ ]  2 ** 3
```

8

```
[ ]  5 ** 3
```

125

파이썬의 기본

- 연산자의 우선순위

사칙연산의 연산자 우선순위와 같음

괄호 () 가 있다면 가자 우선 처리

```
[ ]  5 + 3 * 4
```

```
17
```

```
[ ]  (8 - 3) * 2
```

```
10
```

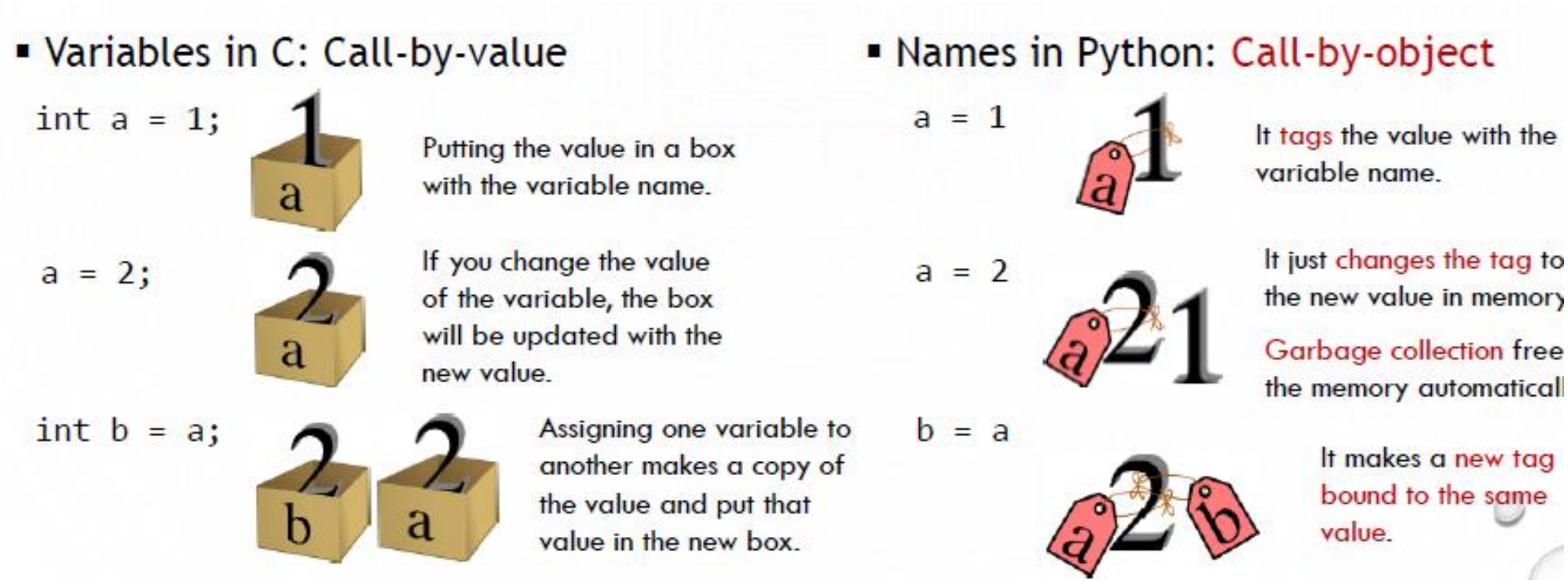
파이썬의 기본

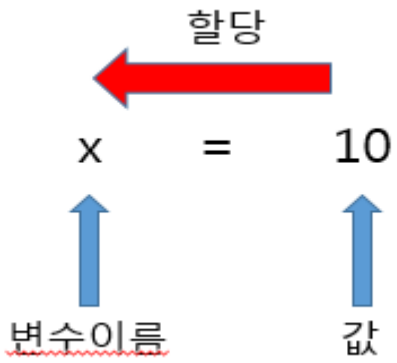
- 가장 기초적인 프로그래밍 문법 개념
- 데이터(값)을 저장하기 위한 메모리 공간의 프로그래밍 상 이름
- 변수는 메모리 주소를 가지고 있고, 변수에 들어가는 값을 메모리 주소에 할당됨.
- 할당 연산자(Assignment) : =

변수 = 연산 / 조건/ 함수

- 할당 연산자 우측을 연산을 한 후, 그 결과를 좌측에 있는 변수에 할당(저장)

- 할당 연산자(Assignment) : =





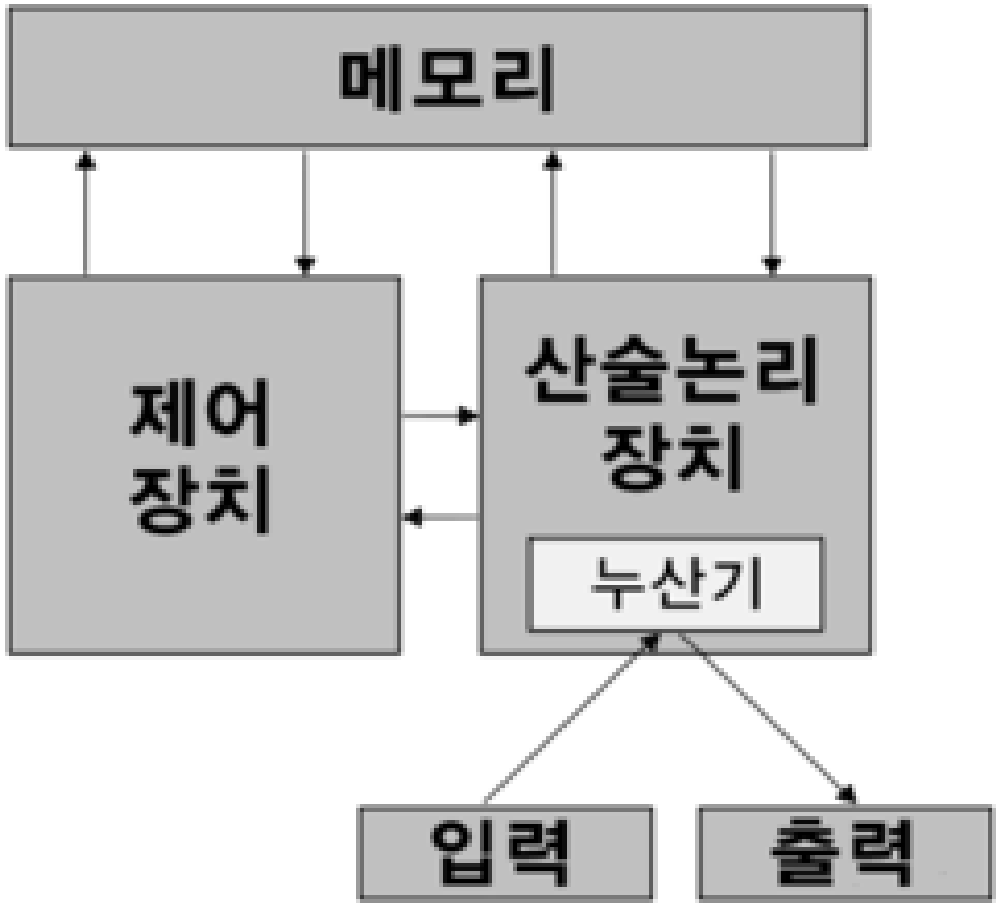
영문, 숫자 사용가능

대소문자 구분

숫자부터 시작할 수 없음.

특수문자 사용 불가(+, -, *, /, $ 등)

컴퓨터의 구조 - 폰노이만 아키텍처



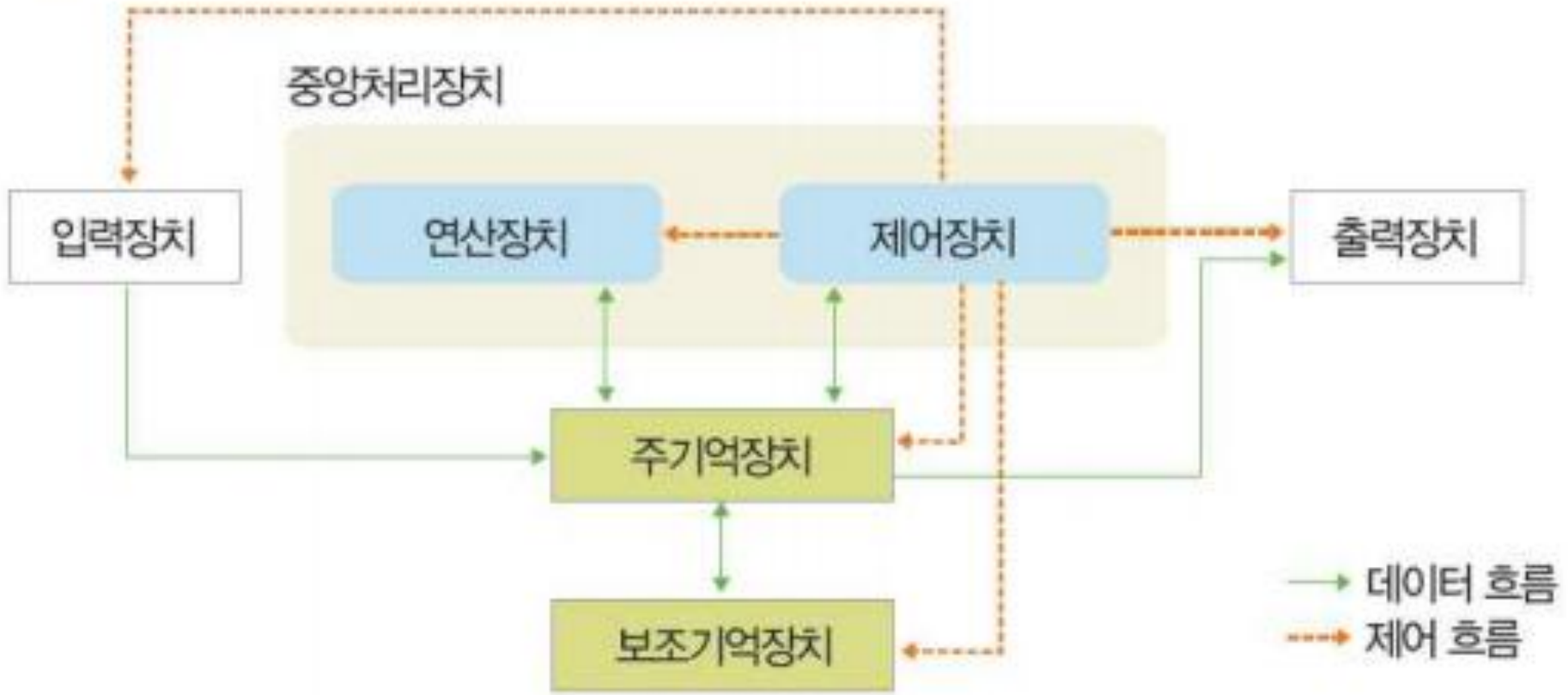
폰 노이만 아키텍처에서는 사용자가 컴퓨터에 값을 입력 하거나 프로그램을 실행하는 경우, 그 정보를 먼저 메모리에 저장시키고 CPU가 순차적으로 그 정보를 해석하고 계산하여 사용자에게 결과값 전달

- 변수는 프로그램에서 사용하기 위한 특정한 값을 저장하는 공간
- 선언 되는 순간 메모리 특정영역에 물리적인 공간이 할당됨

출처 : https://codingisgame.tistory.com/7

중앙처리장치

그림 3-2 하드웨어의 구성



출처 : http://pds21.egloos.com/pds/201504/09/88/computer03.pdf

- 할당 연산자(Assignment) : =

```
In [ ]: a = 3

In [ ]: b = 5

In [ ]: print(a+b)

In [ ]: print("a+b")

In [ ]: a=8

In [ ]: print(a+b)
```

할당 연산자 우측을 연산을 한 후, 그 결과를 좌측에 있는 변수에 할당(저장)

문제) Teacher = 'Youngpyo Ryu'의 의미는?

1. Teacher의 이름은 Youngpyo Ryu이다.
2. Teacher의 Youngpyo Ryu이다.
3. Teacher와 Youngpyo Ryu는 같다.
4. Teacher에 Youngpyo Ryu를 넣어라.

4. Teacher에 Youngpyo Ryu 를 넣어라

-> 정확히는 Teacher라는 변수에 'Youngpyo Ryu'라는 값을 넣으라는 의미.

- 할당 연산자(Assignment) : =

```
In [3]: a += 1
        a
Out[3]: 4

In [5]: a = a+1
        a
Out[5]: 4
```

  - 산술 연산자와 할당 연산자를 함께 쓸 수 있음
  - 코드를 더욱 간결하게 쓸 수 있음

파이썬의 기본

- 할당 연산자(Assignment) : =

```
In [6]: a +=1
        a
Out[6]: 5

In [7]: a -=5
        a
Out[7]: 0

In [10]: a *= -1
         a
Out[10]: 0

In [11]: a /= 5
         a
Out[11]: 0.0

In [12]: a * = -1
         a
  File "<ipython-input-12-
    a * = -1
        ^
SyntaxError: invalid syntax

In [13]: a / = 5
         a
  File "<ipython-input-13-
    a / = 5
        ^
SyntaxError: invalid syntax
```

파이썬의 기본

- 알파벳, 숫자, 언더스코어(_)로 선언이 가능

  ex) data = 0, a12_ = 2, tt_ = 'afdf'

- 변수명은 의미 있는 단어로 표기하는 것이 좋다.

  Ex) teacher = 'Youngpyo Ryu'

- 변수명은 대소문자가 구분 된다.

  Ex) ABC와 Abc는 같지 않다.

- 특별한 의미가 있는 예약어는 쓰지 않는다.

  Ex) for, if, elif, else 등

파이썬의 기본

파이썬 예약어는 파이썬에서 이미 키워드로 지정되어 있어 변수나 함수의 이름으로 사용할 수가 없다.

```
import keyword
print(keyword.kwlist)
```

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break',
'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for',
'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not',
'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
```

출처 : https://isss.tistory.com/842

파이썬의 기본

- 할당 연산자(Assignment) : =

  변수간의 연산도 가능

```
[ ]  a = 5
     b = 7
     c = a + b
     print(c)
```

```
12
```

주의 파이썬은 대소문자를 구분함

```
[ ]  c = A + B
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-26-b698ae2737d5> in <module>
----> 1 c = A + B

NameError: name 'A' is not defined
```

파이썬의 기본

- 할당 연산자(Assignment) : =

파이썬은 들여쓰기(Indent)에 굉장히 민감(jupyter에서는 보정)

```
(base) C:₩Users₩USER>python
Python 3.7.3 (default, Apr 24 2019, 15:29:51) [MSC v.1915 64 bit (AMD64)] :: Anaconda, Inc. on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>  1+1
  File "<stdin>", line 1
    1+1
    ^
IndentationError: unexpected indent
>>>
```

1. 숫자의 자료형

데이터 형식	설명	크기(Byte)	값의 범위
bool	논리 형식	1(8bit)	true, false
sbyte	signed byte 정수	1(8bit)	-128 ~ 127
byte	부호 없는 정수	1(8bit)	0 ~ 255
short	정수	2(16bit)	-32,768 ~ 32,767
ushort	unsigned short 부호 없는 정수	2(16bit)	0 ~ 65,535
char	유니코드 문자	2(16bit)	U+0000 ~ U+ffff
int	정수	4(32bit)	-2,147,483,648 ~ 2,147,483,647
uint	unsigned int 부호 없는 정수	4(32bit)	0 ~ 4,294,967,295
float	단일 정밀도 부동 소수점 형식	4(32bit)	-3.402823e38 ~ 3.402823e38
long	정수	8(64bit)	-9,223,372,036,854,775,808 ~ 9,223,372,036,854,775,807
ulong	unsigned long 부호 없는 정수	8(64bit)	0 ~ 18,446,744,073,709,551,615
double	복수 정밀도 부동 소수점 형식	8(64bit)	-1.79769313486232e308 ~ 1.79769313486232e308
decimal	29자리 데이터를 표현할 수 있는 소수 형식	16(128bit)	±1.0×10e-28 ~ ±7.9×10e28

출처 : 고등학교 상) 수의 체계 / https://mhchoi8423.tistory.com/5

1. 숫자(int, float)

- int와 float

```
[1]  a = 1
     b = 1
     print(a,b)
```

```
1 1
```

```
[3]  type(a)
```

```
int
```

```
[3]  c = 3.5
     d = 4.9

     print(c,d)
```

```
3.5 4.9
```

```
[4]  type(c)
```

```
float
```

int는 정수, float는 실수

type(변수) : 변수의 자료형을 알려주는 함수

Print(변수) : 모니터 화면에 결과물을 출력해주는 함수

1. 숫자(int, float)

- int와 float

```
[4]   d = int(c)
      print(d)
      type(d)
```

```
3
int
```

```
[8]   a = 10
      type(a)
```

```
int
```

```
[9]   float(a)
```

```
10.0
```

Quiz) 현재 a의 자료형은?

Ans) 현재 a는 int이다. 그 이유는 형 변환 해주지 않음

숫자는 형 변환을 할 수 있음

```
[11]  a = float(a)

      type(a)
```

```
float
```

Quiz)

- 5.3과 5.7 정수형으로 형 변환 후 덧셈하면 결과값은?

```
a = 5.3
b = 5.7

a = int(a)
b = int(b)

print(a+b)
```

```
10
```

※ round() : 반올림

2. 문자(String)

- string

‘ ’(작은 따옴표) 혹은 “ ”(큰 따옴표)를 이용하여 문자형(string)임을 표시

```
[11] f = '문자형 예시입니다.'
     print(f)
     type(f)
```

```
문자형 예시입니다.
str
```

2. 문자(String)

```
[12] 따옴표 = '문장안에 따옴표를 쓰고 싶다면? "" 번갈아 쓰면 됩니다.'
     print(따옴표)
```

```
문장안에 따옴표를 쓰고 싶다면? "" 번갈아 쓰면 됩니다.
```

```
[13] escape = "escape 문자인 \" \\ \"(역슬래시 혹은 원화표시)도 사용가능"
     print(escape)
```

```
escape 문자인 " \ "(역슬래시 혹은 원화표시)도 사용가능
```

2. 문자(String)

```
print('''실패란 넘어지는 것이 아니라,
넘어진 자리에 머무는 것이다.
- 도서 '프린세스, 라 브라바!' 中''')
```

```
실패란 넘어지는 것이 아니라,
넘어진 자리에 머무는 것이다.
- 도서 '프린세스, 라 브라바!' 中
```

```
print('₩t탭₩n다음줄')
```

```
        탭
다음줄
```

2. 문자(String)

```
[21] g = 3
     h = '5'
```

Quiz) g,h의 자료형은?

```
[22] type(g)
```

```
int
```

```
[23] type(h)
```

```
str
```

2. 문자(String)

```
[21] g = 3
     h = '5'
```

```
[24] i = str(g)
     print(i)
     type(i)
```

```
3
str
```

```
[ ] j = int(h)
    print(j)
    type(j)
```

```
5
int
```

str와 int도 형 변환이 가능

2. 문자(String)

- string의 연산

```
[26] str_1 = '문자'
     str_2 = '의 연산'
     result = str_1 + str_2
     result
```

```
'문자의 연산'
```

```
[27] str_3 = '더하기'
     result += str_3
     result
```

```
'문자의 연산더하기'
```

더하기를 하면 두 문자열을 붙여줌

2. 문자(String)

- string의 연산

```
[28] str_4 = '곱하기'
     print(str_4 * 5)
```

```
곱하기곱하기곱하기곱하기곱하기
```

```
[29] str_5 = '반복'
     print((str_4 + str_5)*2)
```

```
곱하기반복곱하기반복
```

곱하기는 그 수만큼 문자열을 반복함

2. 문자(String)

- string의 연산

단, 자료형이 다르다면 연산이 불가

```
int_1 = 8
str_6 = '문자'

error = str_6+int_1
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most
<ipython-input-1-9ac98546a9bd> in <module>
      2 str_6 = '문자'
      3
----> 4 error = str_6+int_1

TypeError: can only concatenate str (not "int") to str
```

2. 문자(String)

- string의 indexing(인덱싱)

  인덱싱 ? 순서가 있는 변수의 특정 위치에 접근 할 수 있게 해줌

변수[위치]

(*주의*) 파이썬의 인덱싱은 0부터 시작

```
[47] str_5 = '인덱싱을 하기 위한 string입니다.'
     len(str_5)
```

len(변수)는 변수의 길이를 재는 함수

```
21
```

```
[48] str_5[2]
```

```
'싱'
```

2. 문자(String)

- string의 slicing(슬라이싱)

슬라이싱 ? 변수의 일정 부분에 접근하여 잘라옴

변수[시작 : 끝 : 간격]

```
[49] str_5 = '인덱싱을 하기 위한 string입니다.'
     str_5[2:7]
```

```
'싱을 하기'
```

(*주의*)

- 슬라이싱은 시작부터 끝 -1까지 가져옴
- 간격이 1이라면 생략해도 됨

2. 문자(String)

- string의 slicing(슬라이싱)

슬라이싱 ? 변수의 일정 부분에 접근하여 잘라옴

변수[시작 : 끝 : 간격]

```
[50] str_6 = '안녕하세요. 홍길동 고객님!'
```

```
[52] str_6[7:10]
```

```
'홍길동'
```

2. 문자(String)

- string의 slicing(슬라이싱)

```
[54] str_6 = '안녕하세요. 홍길동 고객님!'
     print(str_6[:2])
```

```
안녕
```

시작점이 없다면 처음부터

```
[57] print(str_6[11:])
```

```
고객님!
```

끝이 없다면 끝까지

```
[58] print(str_6[:])
```

```
안녕하세요. 홍길동 고객님!
```

시작과 끝이 없다면 전체

2. 문자(String)

- string의 slicing(슬라이싱)

  파이썬은 마이너스 인덱싱을 지원함

```
str_6 = '안녕하세요. 홍길동 고객님!'
```

```
print(str_6[-1])
```

```
!
```

-1은 끝에서부터 거꾸로

```
print(str_6[-4:])
```

```
고객님!
```

```
print(str_6[:-2])
```

```
안녕하세요. 홍길동 고객
```

2. 문자(String)

- string의 slicing(슬라이싱)

  파이썬은 마이너스 인덱싱을 지원함

```
[65] str_6 = '안녕하세요. 홍길동 고객님!'
     print(str_6[::-1])
```

```
!님객고 동길홍 .요세하녕안
```

[::-1]은 str 전체를 뒤집음

2. 문자(String)

- string의 method

  method? 파이썬의 객체가 가지고 있는 고유의 함수 . 을 이용하여 사용

  1. 대소문자 변환 : lower, upper

```
[66] str_7 = 'Alphabet'
     print(str_7.lower())
```

```
alphabet
```

```
[67] print(str_7.upper())
```

```
ALPHABET
```

2. 문자(String)

- string의 method

2. 해당 문자의 개수 세기 : count

```
[68] str_7 = 'Alphabet'
     str_7.count('a')
```

```
1
```

count는 대소문자를 구분함

Quiz) str_7에서 모든 것을 소문자 바꾼 다음에 a 문자의 개수를 세어보려면?

```
[71] str_7.lower().count('a')
```

```
2
```

2. 문자(String)

- string의 method

3. 문자열의 위치 찾기

1) find 해당하는 문자열의 위치(인덱스)를 반환

```
[72] str_7 = 'Alphabet'
     str_7.find('t')
```

```
7
```

2) index

```
[73] str_7.index('t')
```

```
7
```

2. 문자(String)

- string의 method

  더 많은 메소드는 Python 공식문서나 아래와 같은 방법으로 확인 가능

```
[21] print(dir(str_8))
```

```
['__add__', '__class__', '__contains__', '__delattr__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__ge
tattribute__', '__getitem__', '__getnewargs__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__iter__', '_
_le__', '__len__', '__lt__', '__mod__', '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '_
_rmod__', '__rmul__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', 'capitalize', 'casefold', 'center',
'count', 'encode', 'endswith', 'expandtabs', 'find', 'format', 'format_map', 'index', 'isalnum', 'isalpha', 'isascii'
, 'isdecimal', 'isdigit', 'isidentifier', 'islower', 'isnumeric', 'isprintable', 'isspace', 'istitle', 'isupper', 'jo
in', 'ljust', 'lower', 'lstrip', 'maketrans', 'partition', 'replace', 'rfind', 'rindex', 'rjust', 'rpartition', 'rspl
it', 'rstrip', 'split', 'splitlines', 'startswith', 'strip', 'swapcase', 'title', 'translate', 'upper', 'zfill']
```

3. 리스트(list)

- 시퀀스 자료형, 여러 데이터들의 집합
- int, float 같은 다양한 데이터 타입 포함

[ ]을 이용하여 리스트(list)임을 선언

```
[1]  list_1 = [3,2,5]
     list_2 = list()
```

```
[2]  type(list_1)
```

```
list
```

3. 리스트(list)_특징

- 리스트는 모든 자료형을 담을 수 있음
- 리스트는 삽입, 수정, 삭제가 자유로움
- 리스트는 순서가 있는 자료형 → 인덱싱과 슬라이싱이 가능

```
[4] list_3 = [1, 2, '문자', ['이중 리스트', '가능'], ('리스트 속', '튜플')]
```

```
[5] list_3[3]
```

```
['이중 리스트', '가능']
```

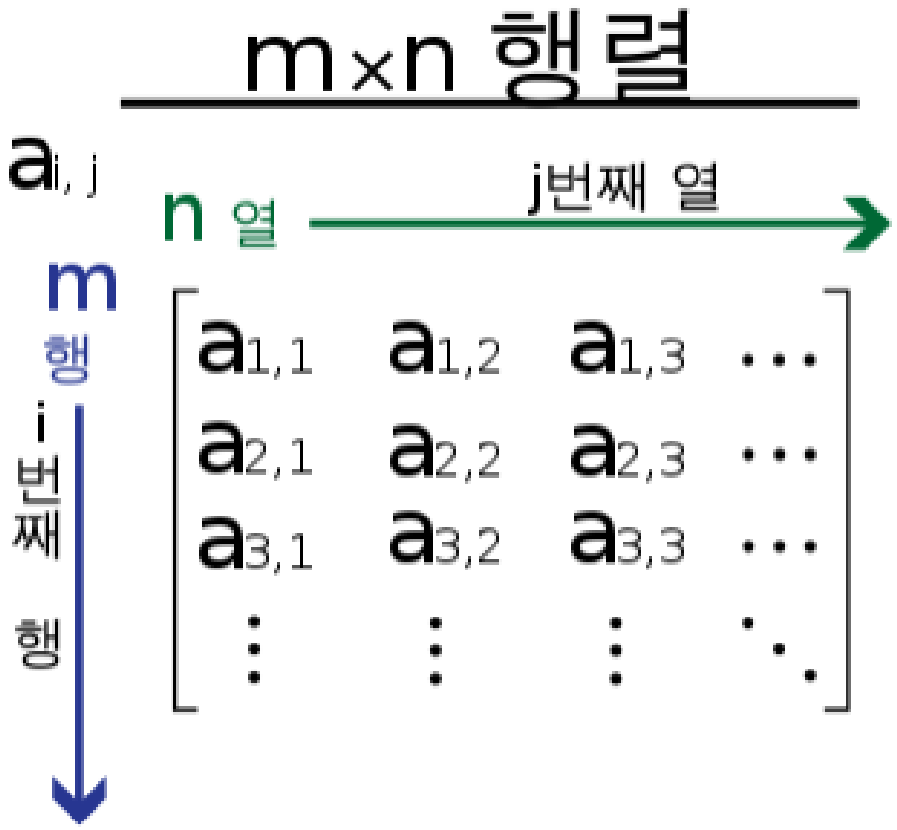
```
[6] list_3[4][1]
```

```
'튜플'
```

주의 python의 인덱스는 0부터 시작

3. 리스트(list)

- 2차원 리스트는 가로X세로 형태로 이루어져 있으며,
  행(row)과 열(column) 모두 0부터 시작



출처 : https://blog.naver.com/cindyvelyn/222126373034

3. 리스트(list)

- 2차원 리스트는 리스트 안에 리스트를 넣어서 만듦.
- , (콤마)로 구분
- 리스트 = [[값, 값],[값, 값],[값, 값]]
- 리스트 [세로인덱스] [가로인덱스]

```
a = [[10,20],[30,40],[50,60]]
a
```

```
[[10, 20], [30, 40], [50, 60]]
```

```
a[0][0]
```

```
10
```

```
a[1][0]
```

```
30
```

3. 리스트(list)

- list의 인덱싱

```
[7]  list_4 = [[1,2,3,4],
               [2,4,6,8],
               [3,6,9,12],
               [4,8,12,16],
               [5,10,15,20]]
```

가장 바깥쪽부터 괄호부터

```
[9]  list_4[1][3]

     8
```

앞에서부터 순서대로

3. 리스트(list)

- list의 인덱싱

```
[ ]  list_5 = [[[1,2,3],[4],[5,[6,7,8,9]]],
               [10,11,[12,13,[14]]]]
```

```
[ ]  list_5[1][2][2]
```

```
[14]
```

3. 리스트(list)

- list의 인덱싱

```
[7]  list_4 = [[1,2,3,4],
               [2,4,6,8],
               [3,6,9,12],
               [4,8,12,16],
               [5,10,15,20]]
```

```
[ ]  list_4[1][:3]
```

```
[2, 4, 6]
```

```
[ ]  list_4[:2][1]
```

```
[2, 4, 6, 8]
```

3. 리스트(list)

- list의 인덱싱

```
[9]  list_5 = [[[1,2,3],[4],[5,[6,7,8,9]]],
               [10,11,[12,13,[14]]]]
```

```
[13] list_5[:2][0]

     [[1, 2, 3], [4], [5, [6, 7, 8, 9]]]
```

```
[14] list_5[0][:2]

     [[1, 2, 3], [4]]
```

3. 리스트(list)

- list의 연산

```
[15] list_6 = ['a', 'b', 'c']
```

```
[16] list_7 = ['가', '나', '다']
```

```
[17] list_6 + list_7
```

```
['a', 'b', 'c', '가', '나', '다']
```

```
[18] list_6 * 3
```

```
['a', 'b', 'c', 'a', 'b', 'c', 'a', 'b', 'c']
```

```
[15] list_6 = ['a','b','c']
```

```
[16] a in list_6
```

```
False
```

3. 리스트(list)

- 메소드

  리스트는 삽입, 수정, 삭제가 자유로움

1. 삽입 : append, insert

```
[19] list_8 = ['파이썬', 'C', 'java']
```

```
[20] list_8.append('R')
     list_8
```

```
['파이썬', 'C', 'java', 'R']
```

→ append는 리스트 제일 뒤에

```
[21] list_8.insert(2, 'C++')
     list_8
```

```
['파이썬', 'C', 'C++', 'java', 'R']
```

```
[22] list_8.insert(5, 'C')
     list_8
```

```
['파이썬', 'C', 'C++', 'java', 'R', 'C']
```

3. 리스트(list)

- 메소드

  2. 수정 : 덮어쓰기

```
[23] list_8[0] = 'Python'
     list_8
```

인덱스나 슬라이싱을 통해 수정

```
['Python', 'C', 'C++', 'java', 'R', 'C']
```

2-1) 확장

```
[23] list_8.extend(['MATLAB','Julia'])
     list_8

     ['Python', 'C', 'C++', 'java', 'R', 'C', 'MATLAB', 'Julia']
```

3. 리스트(list)

- 메소드

  3. 삭제

```
[24] list_8
```

```
['Python', 'C', 'C++', 'java', 'R', 'C']
```

```
[25] list_8.remove('C')
     list_8
```

```
['Python', 'C++', 'java', 'R', 'C']
```

→ remove는 지우고자 하는 요소를 명시해야함

→ 가장 앞에 있는 요소가 **삭제**됨

- remove(x) -> 리스트 안에서 값 x를 찾아서 지움
- del list[i] -> 리스트에서 i번째를 지움
- del list -> 리스트 변수 자체를 없앰

```
[34] del list_8[6]
     list_8
```

```
['Python', 'C', 'C++', 'java', 'R', 'C']
```

```
del list_8
print(list_8)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipython-input-1175381942.py in <cell line: 0>()
      1 del list_8
----> 2 print(list_8)

NameError: name 'list_8' is not defined
```

3. 리스트(list)

- 메소드

  4. 꺼내오기

```
[26] list_8.pop()

     'C'
```

```
[28] list_8

     ['Python', 'C++', 'java', 'R']
```

→ pop은 괄호( )안 인덱스에 해당하는 요소를 **반환**

→ 인덱스를 생략하면 가장 마지막 요소를 꺼내옴

3. 리스트(list)

- 메소드

  5. 정렬 : sort

```
[29] list_9 = [1,5,7,2,6]
```

```
[30] list_9.sort()
     list_9
```

리스트 자체를 정렬

```
[1, 2, 5, 6, 7]
```

list.sort(key=None, reverse=False)

- key : 정렬해주고 싶은 기준, 기본적으로 < 비교에 의해 정렬(오름차순)
- reverse : True로 설정하게 되면 > 비교에 의해 정렬(내림차순)

3. 리스트(list)

- 메소드

  5. 정렬 : sort

```
[32] list_10 = [1,47,12,6]
     list_10.sort(reverse=True)
     list_10
```

```
[47, 12, 6, 1]
```

- 메소드

  6. 뒤집기 : reverse

```
[37] list_9
```

```
[1, 2, 5, 6, 7]
```

```
[38] list_9.reverse()
     list_9
```

```
[7, 6, 5, 2, 1]
```

→ 현재 리스트 요소 순서를 뒤집음
→ 리스트 **자체**를 뒤집음

3. 리스트(list)

- 메소드

  6. 뒤집기 : reverse

```
[32] list_10 = [1,47,12,6]
     list_10.sort(reverse=True)
     list_10
```

```
[47, 12, 6, 1]
```

```
[41] list_10.reverse()
     list_10
```

```
[1, 6, 12, 47]
```

```
sorted(list_10)
```

```
[1, 6, 12, 47]
```

sorted() : 원본 리스트를 바꾸지 않고, 정렬된 "새 리스트"를 만들어 변환하는 함수

: 원본을 변경하지 않음

4. 튜플(tuple)

- ( )을 이용하여 튜플(tuple)임을 선언

```
[42] tuple_1 = ()
     tuple_2 = tuple()
```

```
[43] tuple_3 = (1, 2)
     tuple_4 = (3,)
     tuple_5 = (4, 5, (6, 7))
     tuple_6 = 8, 9, 10
```

- 한 개의 요소만 사용 할 때에는 콤마(,)를 반드시 사용해야함
- 괄호가 없어도 튜플로 선언할 수 있음

4. 튜플(tuple)

- 인덱싱과 슬라이싱이 가능
- 리스트와 달리 요소를 삽입/삭제/수정 할 수 없음

```
[44] tuple_3 = (1, 2)
     tuple_3[0] = 3
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-44-c8372825bae8> in <module>()
      1 tuple_3 = (1, 2)
----> 2 tuple_3[0] = 3

TypeError: 'tuple' object does not support item assignment
```

SEARCH STACK OVERFLOW

4. 튜플(tuple)

- tuple의 연산

```
[45] tuple_3 = (1, 2)
     tuple_4 = (3, )
     tuple_3 + tuple_4
```

```
(1, 2, 3)
```

```
[46] tuple_4 * 2
```

```
(3, 3)
```

5. 딕셔너리(dictionary)

- dictionary
  { }을 이용하여 딕셔너리(dictionary)임을 선언

```
[47] dict_1 = {'key' : 'value'}
     dict_2 = dict()
```

- 딕셔너리는 key와 value의 값으로 이뤄진 자료형
- 순서가 있는 자료형이 아니며, key를 통해 value값에 접근이 가능
- key는 고유한 값으로 중복될 수 없음
- value는 중복 가능

5. 딕셔너리(dictionary)

- dictionary 추가/삭제/수정

1. 요소 추가하기 : 새로운 key에 value를 할당

```
[48] dict_3 = {'홍길동' : 100, '홍계월' : 200}
```

주의 인덱스 대신 key 사용

```
[49] dict_3['슈퍼맨'] = 300
     dict_3
```

```
{'슈퍼맨': 300, '홍계월': 200, '홍길동': 100}
```

2. 요소 수정하기 : 덮어쓰기

```
[51] dict_3['홍길동'] = 1
     dict_3
```

```
{'홍계월': 200, '홍길동': 1}
```

5. 딕셔너리(dictionary)

- dictionary 추가/삭제/수정

  3. 요소 삭제하기

```
[50] del dict_3['슈퍼맨']
     dict_3
```

```
{'홍계월': 200, '홍길동': 100}
```

주의 인덱스 대신 key 사용

del : 파이썬 자체의 명령어로 메모리 자체에서 삭제

5. 딕셔너리(dictionary)

- key와 value에 접근하기

1. key에 접근하기 : keys()

```
[1] dict_3 = {'홍길동' : 100, '홍계월' : 200, '슈퍼맨' : 150, '배트맨' : 250}
    print(dict_3.keys())
    print(type(dict_3.keys()))
```

```
dict_keys(['홍길동', '홍계월', '슈퍼맨', '배트맨'])
<class 'dict_keys'>
```

2. value에 접근하기 : values()

```
[2] print(dict_3.values())
    print(type(dict_3.values()))
```

```
dict_values([100, 200, 150, 250])
<class 'dict_values'>
```

5. 딕셔너리(dictionary)

- key와 value에 접근하기

  3. key와 value에 함께 접근하기

```
[3]  print(dict_3.items())
     print(type(dict_3.items()))
```

```
dict_items([('홍길동', 100), ('홍계월', 200), ('슈퍼맨', 150), ('배트맨', 250)])
<class 'dict_items'>
```

주의

리스트 처럼 보이나 실제로는 dict의 개체 이므로 리스트의 메소드를 사용할 수 없음

리스트처럼 사용하고 싶다면 리스트로 형 변환을 해야함

```
[4]  list(dict_3.keys())[0]
```

```
'홍길동'
```

5. 딕셔너리(dictionary)

- key와 value에 접근하기

4. key와 value에 함께 접근하기

1) 인덱싱처럼 키값을 이용하여 바로 접근

2) 메소드 get을 사용

```
[5]  dict_3
```

```
{'배트맨': 250, '슈퍼맨': 150, '홍계월': 200, '홍길동': 100}
```

```
[6]  print(dict_3['배트맨'])
     print(dict_3.get('배트맨'))
```

```
250
250
```

5. 딕셔너리(dictionary)

- key와 value에 접근하기

4. key와 value에 함께 접근하기

```
[7]  print(dict_3['펭수'])
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
<ipython-input-7-43368d90b0e9> in <module>()
----> 1 print(dict_3['펭수'])

KeyError: '펭수'

SEARCH STACK OVERFLOW
```

→ 키 값을 이용하여 접근 할 경우,
해당 키가 없으면 오류 발생

```
[8]  print(dict_3.get('펭수'))
```

→ get을 통해 접근할 경우, None 반환

```
None
```

6. set(집합)

- set

  {}을 이용하여 set을 선언

```
[9]  set_1 = set()
     set_2 = set('Hello')
     set_3 = set([1, 2, 3])
     set_4 = {3,5, 'hi'}
```

```
[10] set_2
```

```
{'H', 'e', 'l', 'o'}
```

```
[11] set_3
```

```
{1, 2, 3}
```

```
[12] set_4
```

```
{3, 5, 'hi'}
```

- 중복을 허용하지 않음
- 순서가 없음

6. set(집합)

- set의 연산

  1. 교집합

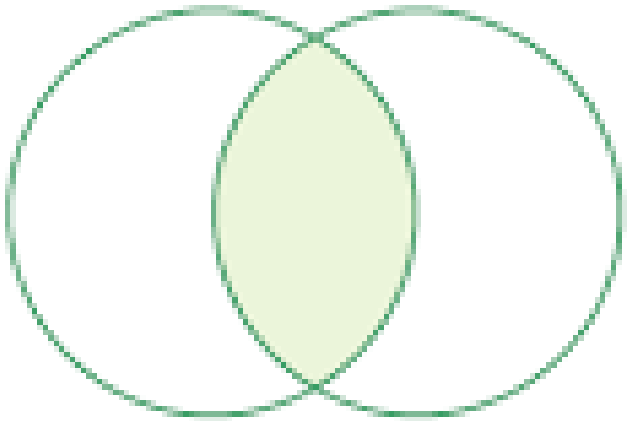
```
[13] set_5 = {1,2,3,4}
     set_6 = {4,5,6,7}
```

```
[14] set_5 & set_6
```

```
{4}
```

```
[15] set_5.intersection(set_6)
```

```
{4}
```



(a) 교집합

6. set(집합)

- set의 연산

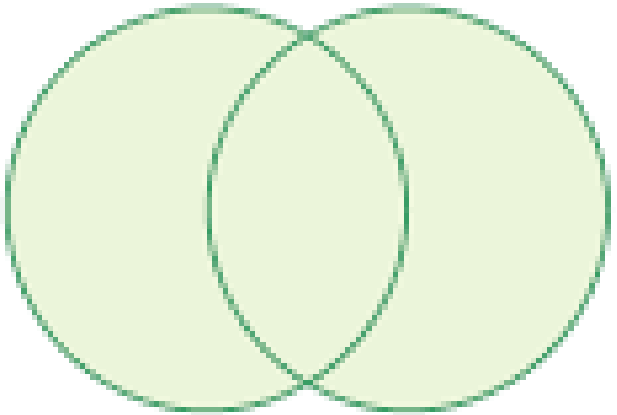
  2. 합집합

```
[16] set_5 | set_6
```

```
{1, 2, 3, 4, 5, 6, 7}
```

```
[17] set_5.union(set_6)
```

```
{1, 2, 3, 4, 5, 6, 7}
```



(b) 합집합

6. set(집합)

- set의 연산

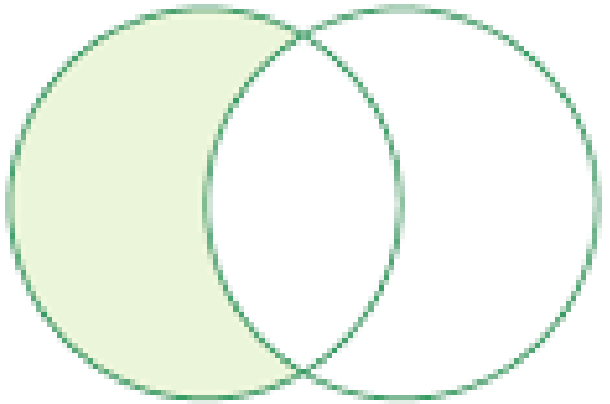
  3. 차집합

```
[18] set_5 - set_6
```

```
{1, 2, 3}
```

```
[19] set_5.difference(set_6)
```

```
{1, 2, 3}
```



(c) 차집합

6. set(집합)

- set의 메소드

1. 한 개 추가하기 : add

```
[22] set_6 = {4, 5, 6, 7}
     set_6.add(8)
     set_6
```

```
{4, 5, 6, 7, 8}
```

2. 여러 개 추가하기 : update

```
[23] set_6.update([1,2,3])
     set_6
```

```
{1, 2, 3, 4, 5, 6, 7, 8}
```

3. 한 개 지우기 : remove

```
[24] set_6.remove(3)
     set_6
```

```
{1, 2, 4, 5, 6, 7, 8}
```

7. 불(bool)

- bool : 참과 거짓을 나타내는 자료형

  참 : True

  거짓 : False

```
[26] True
```

```
True
```

```
[27] type(False)
```

```
bool
```

```
[28] 1 == 1
```

```
True
```

```
[29] 1 > 2
```

```
False
```

비교 연산자	의미
==	같음
!=	같지 않음
>, >=, <, <=	크거나 같음/ 작거나 같음

→ 비교 연산자와 조건문의 반환값으로 불자료형이 사용됨

7. 불(bool)

- bool : 참과 거짓을 나타내는 자료형

자료형에도 참/거짓이 있음

자료	참 / 거짓
'python'	참
''	거짓
[1,2,3]	참
[ ]	거짓
( )	거짓
{ }	거짓
1	참
0	거짓
None	거짓

```
[30] bool(1)
```

```
True
```

```
[31] bool(' ')
```

```
True
```

```
[32] bool('')
```

```
False
```

문제해결 프로세스

과학적 문제해결 프로세스
PART1.
문제발생의 원인 규명 과정
PART2.
해법의 수립과 적용 과정
문제 인식
가설 설정
실증
해법 규명

조건문이란 ?

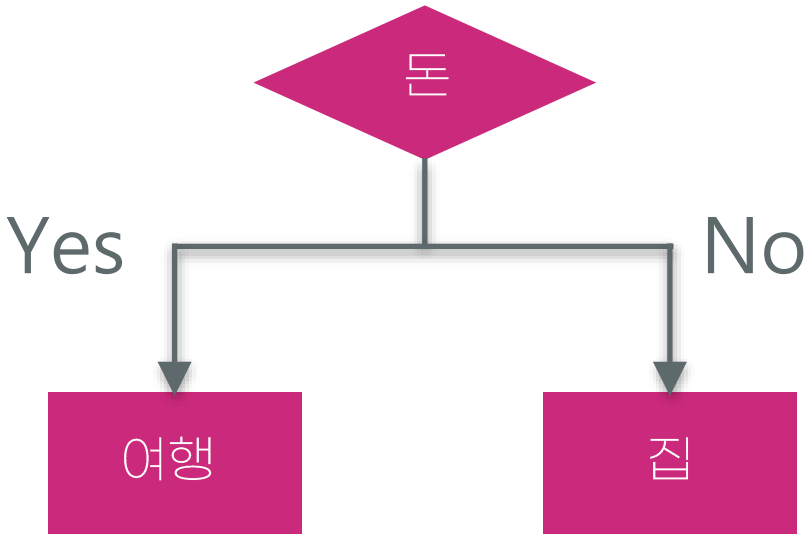
- 조건에 따라 특정한 동작을 하게하는 명령어
- 프로그램 예시 in 생활

  - 자동차 앞차 간격이 100M 이하면 속도를 50km 이하로 낮춰라
  - 사용자가 어린이인 경우에는 VOD를 플레이 하지 마라
  - 패턴이 5회 틀리면 10초 동안 대기 상태로 만들어라

- 조건문은 조건을 나타내는 기준과 실행해야 할 명령으로 구성됨
- 조건의 참, 거짓에 따라 실행해야 할 명령이 수행되거나 되지 않음
- 파이썬은 조건문으로 if, else, elif 등의 명령 키워드를 사용함.

조건문이란 ?

- if / else




```
[ ]  money = True

     if money :
         print("여행을 간다")
     else :
         print("집에서 쉰다")
```

```
여행을 간다
```

if 조건식:

코드

들여쓰기 or 탭

공식 4칸

조건문이란 ?

- if / else

```
[ ]  if 조건 :
         수행 할 문장1
         수행 할 문장2
     else :
         수행 할 문장3
         수행 할 문장4
```

주의

콜론(:)과 들여쓰기에 주의할 것

```
[ ]  money = True

     if money :
         print("여행을 간다")
     else :
         print("집에서 쉰다")
```

```
여행을 간다
```

조건문이란 ?

- if / elif / else

```
[ ]  if 조건1 :
         수행 할 문장1
     elif 조건2 :
         수행 할 문장2
     else :
         수행 할 문장3
```

```
[ ]  money = 20000

     if money < 5000 :
         print('라면을 먹는다')
     elif 5000 < money < 25000 :
         print('치킨을 먹는다')
     elif 25000 < money < 50000 :
         print('삼겹살을 먹는다')
     else :
         print('소고기를 먹는다')
```

치킨을 먹는다

조건문이란 ?

- 비교 연산자

```
[ ]  a = 3
     b = 5

     a < b
```

```
True
```

```
[ ]  c = 3
     d = 3

     c <= d
```

```
True
```

```
[ ]  c != d
```

```
False
```

조건문이란 ?

- 논리연산자 : and / or / not

```
[ ] True and True
```

```
True
```

```
[ ] True and False
```

```
False
```

1) and

비교하는 대상이 모두 참이여야만

True를 반환

```
boolean_list = [True,False,True,False,True]

all(boolean_list)
```

```
False
```

조건문이란 ?

- 논리연산자 : and / or / not

```
[ ] True or True
```

True

```
[ ] True or False
```

True

```
[ ] False or False
```

False

2) or

비교하는 대상 중 하나만 참이여도

True 반환

```
boolean_list = [True,False,True,False,True]

any(boolean_list)
```

True

조건문이란 ?

- 논리연산자 : and / or / not

```
[ ] not True
```

```
False
```

```
[ ] not False
```

```
True
```

3) not

참이면 False, 거짓이면 True을 반환

조건문이란 ?

- 논리연산자 : and

```
a=100

if a>0 and (a%2)==0:
    print('a is even')
```

```
a is even
```

```
5 and 7
```

```
7
```

```
7 and 13
```

```
13
```

- and : 조건이 True/ False 로 반환이 됨.
- & : 비트연산자로 작동 됨.

```
7 & 13 # 0111&1101
```

```
5
```

```
13 & 7# 0111&1101
```

```
5
```

조건문이란 ?

- 논리연산자 : or

```
a=100

if a>0 or (a%2)!=0:
    print('a is even')
```

```
a is even
```

```
5 or 7
```

```
5
```

```
7 or 13
```

```
7
```

- or : 조건이 True/ False 로 반환이 됨.
- | : 비트연산자로 작동 됨.

```
7 | 13 # 0111|1101
```

```
15
```

```
13 | 7# 0111|1101
```

```
15
```

조건문이란 ?

- 요소인지 파악하기 : in / not in

```
[ ] e = [1, 3, 5, 7]
```

```
[ ] 0 in e
```

```
False
```

```
[ ] 1 in e
```

```
True
```

1) x in s

x가 s의 요소인가

```
[ ] 2 not in e
```

```
True
```

```
[ ] 3 not in e
```

```
False
```

2) x not in s

x가 s의 요소가 아닌가

조건문이란 ?

- if문 안에 if문

  조건 안에 조건을 만들 수 있음

```
[ ]  money = 20000
     card = True
```

```
[ ]  if card :
         if money < 30000 :
             print("삼겹살을 먹는다")
         else :
             print("소고기를 먹는다")
     else :
         if money <= 1000 :
             pass
         else :
             print("라면을 먹는다")
```

삼겹살을 먹는다

- pass

  조건을 만족해도 아무일도 일어나지 않게 하려면?

```
[ ]  money = 1000
     card = False
```

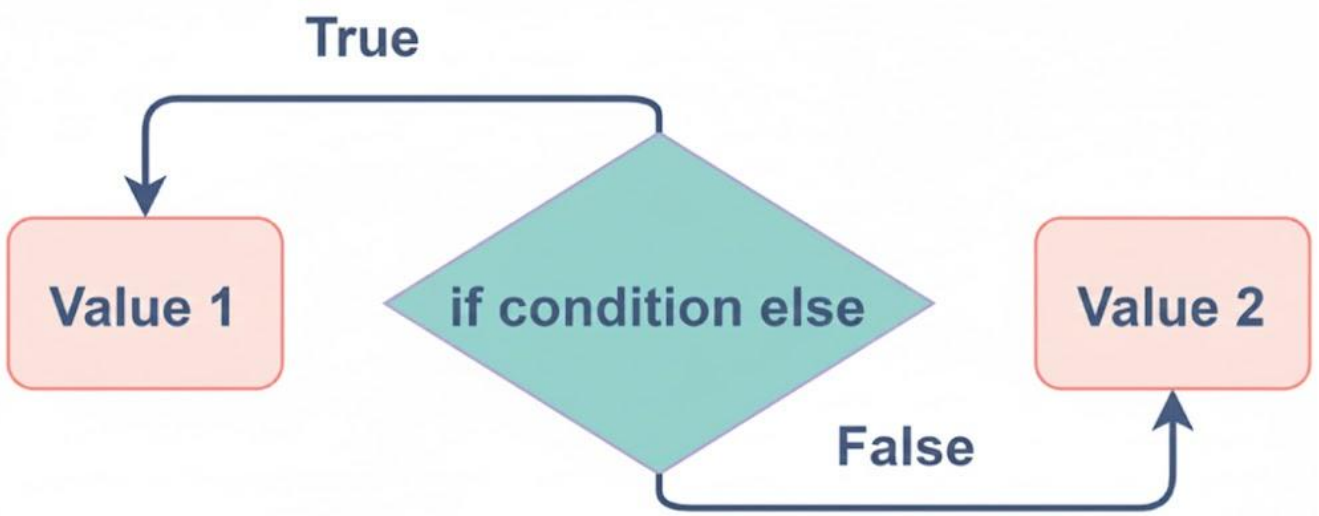
```
[ ]  if card :
         if money < 30000 :
             print("삼겹살을 먹는다")
         else :
             print("소고기를 먹는다")
     else :
         if money <= 1000 :
             pass
         else :
             print("라면을 먹는다")
```

조건문이란 ?

- 삼항 연산자(Ternary Operators)

  - Condition(조건)이 참이면 true_value 값을 취하고 거짓이면 false_value 값을 취함

- [true_value] if [condition] else [false_value]




```
value = 20

is_even = True if value %2 ==0 else False

print(is_even)
```

```
True
```

출처 : https://www.educative.io/edpresso/what-is-the-ternary-operator-in-python

- Formatting

1. %

코드	설명
%d	int
%f	float
%s	string
%c	문자 1개
%%	% 자체

```
[4]  num = 1
```

```
[5]  print('문자열 포맷팅 하기 예시 %d' %num)
```

```
문자열 포맷팅 하기 예시 1
```

```
[6]  ch = '이름'
```

```
[7]  print('내 이름은 : %s' %ch)
```

```
내 이름은 : 이름
```

```
[8]  fl = 2.566132
```

```
[9]  print('반올림 하기 : %0.1f' %fl)
```

```
반올림 하기 : 2.6
```

- Formatting

1. %

코드	설명
%d	int
%f	float
%s	string
%c	문자 1개
%%	% 자체

괄호로 묶어주면 두 개 이상도 포맷팅 할 수 있음

```
[10] a = 3
     b = 5
```

```
[11] print('%d 와 %d 를 더하면 %d' %(a,b,a+b))
```

```
3 와 5 를 더하면 8
```

- Formatting

2. format

'{ }'.format(포맷팅 할 변수)

```
[12] print('문자열 포맷팅 하기 예시 {}'.format(2))

     문자열 포맷팅 하기 예시 2
```

```
[14] print('내 이름은 : {}'.format(ch))

     내 이름은 : 이름
```

- Formatting

2. format

- 괄호로 묶어주면 두 개 이상도 포맷팅 할 수 있음

```
a = '류'
b = '영표'
```

```
print('성은 {}, 이름은 {}'.format(a,b))
```

```
성은 류, 이름은 영표
```

- 인덱스를 통해 순서를 지정할 수도 있음

```
print('성은 {0}, 이름은 {1}'.format(a,b))
```

```
성은 류, 이름은 영표
```

```
print('Frist name : {1}, Last name : {0}'.format(a,b))
```

```
Frist name : 영표, Last name : 류
```

- Formatting

2. format

  - 이름을 통해서 포맷팅 할 수도 있음

```
[20] print('나는 오늘 {음료}를 {개수} 잔이나 마셨다'.format(음료='커피', 개수=3))

     나는 오늘 커피를 3 잔이나 마셨다
```

  - 이 밖에도 정렬이나 공백 이용하는 등 다양한 포맷팅이 가능

- Formatting

3. f-string(f 문자열 포맷팅)

f '{ }'

```
[28] year = 2020
```

```
[29] f'올해는 {year}'
```

```
'올해는 2020'
```

- Formatting

3. f-string(f 문자열 포맷팅)

- 다른 포맷팅과는 달리 표현식을 지원함

```
[39] drink = '커피'
     nums = 3
```

```
[40] f'나는 오늘 {drink}를 {nums + 1} 잔이나 마셨다'
```

```
'나는 오늘 커피를 4 잔이나 마셨다'
```

```
# 문자열 맨 앞에 f를 붙이고, 출력할 변수, 값을 중괄호 안에 넣습니다.
s = 'coffee'
n = 5
result1 = f'저는 {s}를 좋아합니다. 하루 {n}잔 마셔요.'
print(result1)
```

```
저는 coffee를 좋아합니다. 하루 5잔 마셔요.
```

- Formatting

```
[37] '나는 오늘 %s 를 %d + 1 잔이나 마셨다' %('커피', 3)

     '나는 오늘 커피 를 3 + 1 잔이나 마셨다'
```

```
[38] '나는 오늘 {음료}를 {개수+1} 잔이나 마셨다'.format(음료='커피', 개수=3)

     ---------------------------------------------------------------------------
     KeyError                                  Traceback (most recent call last)
     <ipython-input-38-21356a636e64> in <module>()
     ----> 1 '나는 오늘 {음료}를 {개수+1} 잔이나 마셨다'.format(음료='커피', 개수=3)

     KeyError: '개수+1'

     SEARCH STACK OVERFLOW
```

```
[39] drink = '커피'
     nums = 3
```

```
[40] f'나는 오늘 {drink}를 {nums + 1} 잔이나 마셨다'

     '나는 오늘 커피를 4 잔이나 마셨다'
```

연습문제 5-1

b로 a를 나눈 나머지가 3 초과면 실패, 3이면 무승부, 3 미만이면 성공이 출력되도록 만들어 보자

```
a = 34
b = 4
```

```
# 정답을 적어주세요
```

성공

연습문제 5-2

[홀수 짝수 판별기]

사용자로부터 정수를 하나 입력받아

입력한 정수가 홀수인지 짝수인지 판별하여라.

(** 0은 짝수라 하자.)

[출력결과]

```
정수를 입력해주세요: 5
입력하신 5는 홀수입니다.

짝수를 입력해주세요: 10
입력하신 10은 짝수입니다.
```

연습문제 5-3

무슨 학교 다니세요?

태어난 연도를 계산하여 학교 종류를 맞추는 프로그램

```
당신이 태어난 년도를 입력하세요.
2000
학생이 아닙니다.
```

- 나이는 현재년도-태어난 년도+1로 계산
- 26세 이하 20세 이상이면 “대학생“, 20세 미만 17세 이상이면 “고등학생”, 17세 미만 14세 이상이면 “중학생“, 14세 미만 8세 이상이면 “초등학생”
- 그 외의 경우는 “학생이 아닙니다 " 출력

연습문제 5-4

무슨 학교 다니세요?

태어난 연도를 계산하여 학교 종류를 맞추는 프로그램

```
당신이 태어난 년도를 입력하세요.
2000
학생이 아닙니다.
```

- 나이는 현재년도-태어난 년도+1로 계산
- 26세 이하 20세 이상이면 “대학생“, 20세 미만 17세 이상이면 “고등학생”, 17세 미만 14세 이상이면 “중학생“, 14세 미만 8세 이상이면 “초등학생”
- 그 외의 경우는 “학생이 아닙니다 " 출력

연습문제 5-5

<평균과 과락을 함께 판별하기>

변수는 kor, eng, math에 세 과목이 저장되어 있다.

다음 두 조건을 만족하면 "합격", 아니면 "불합격“

- 평균: 70점 이상, 모든 과목이 40점 이상

kor = 80

eng = 75

math = 60

조건문 작성

연습문제 5-6

다음 dictionary에서 나이가 20 이상이고 학생이면 “대학생 또는 성인 학생“ ,
20 미만이고 학생이면 “미성년 학생“ 학생이 아니면 “일반인”을 출력하세요.

person = {"name": "Yuna", "age": 19, "is_student": True}

반복문

- 정해진 동작을 반복적으로 수행하게 하는 명령문
- 프로그램 예시 in 생활
  - 100명의 학생에 성적을 산출할 때
  - 검색 결과를 출력해 줄 때
  - 워드에서 단어 바꾸기 명령 실행
- 반복문은 반복 시작 조건, 종료 조건, 수행 명령으로 구성됨
- 반복문 역시 반복 구문은 들여쓰기와 block으로 구분됨
- 파이썬은 반복문으로 for, while 등의 명령 키워드를 사용함

반복문 - while

- 조건문이 참인 동안 반복해서 문장을 수행함

```
[ ]  while 조건 :
         수행할 문장1
         수행할 문장2
```

```
[ ]  # 10이하의 짝수 프린트하기
     i = 1                          -> 조건문에 쓰일 변수

     while i <= 10 :                -> 조건문
         if i % 2 == 0 :
             print(i)
         i += 1                     -> 변수를 증감
```

```
2
4
6
8
10
```

반복문 - while

- break

  반복문에서 빠져나오고 싶다면?

```
[ ]  # 100번째 방문자 찾기
     i = 90

     while i :   -> 변수는 0이 아니라면 항상 참(무한 반복)
         i += 1
         if i == 100 :
             print("축하합니다. %d번째 방문자입니다." % i)
             break
     print("감사합니다. 이벤트가 종료되었습니다.")   -> 반복문이 종료되었으므로 수행됨
```

```
축하합니다. 100번째 방문자입니다.
감사합니다. 이벤트가 종료되었습니다.
```

break를 사용하면 반복문이 더 이상 작동하지 않고 멈춤

반복문 - while

- break

  반복문의 첫 부분으로 돌아가고 싶다면?

```
[ ]  i = 0

     while i < 11 :
         i += 1
         if i == 6 :
             continue
         if i % 2 == 0 :
             print(i)
```

-> 6일 때는 continue를 사용해 반복문의 가장 처음으로 가도록 함

-> 6일 때는 짝수임에도 불구하고 프린트가 안됨

```
2
4
8
10
```

반복문 - for

```
[ ]  for 변수 in range(변수가 속한 자료형, 혹은 변수의 범위) :
         수행해야할 문장1
         수행해야할 문장2
```

주의
콜론(:)과 들여쓰기에
주의할 것.

```
[ ]  for x in range(0,5) :
         print(x)
```

```
0
1
2
3
4
```

반복문 - for

```
[ ]  for x in range(0,5) :
         print(x)
```

```
0
1
2
3
4
```

변수의 범위 지정하기

1) 숫자로 범위주기

2) 자료형으로 범위주기

반복문 - for

- 변수의 범위 지정하기

  1) 숫자로 범위 : range( )

range(시작, 끝, 간격)

- 시작부터 끝 -1 까지
- 간격이 1이라면 생략가능
- 0부터 시작하여 1씩 증가한다면 시작과 간격 생략가능

반복문 - for

- 변수의 범위 지정하기

  1) 숫자로 범위 : range( )

  연습문제) 5에서 0까지 카운트 다운

```
[ ]  # 카운트 다운
     for count in range(5, -1, -1):
         print(count)
```

-> 하나씩 작아지므로 간격은 -1

-> 0까지 출력해야 하므로 -1이 끝 인덱스가 되어야 함

```
5
4
3
2
1
0
```

반복문 - for

- 변수의 범위 지정하기

  2) 자료형

```
[ ] word = 'Hello!'

    for w in word:
        print(w)
```

```
H
e
l
l
o
!
```

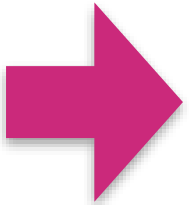
```
[ ] for a, b in [(2,1), (2,2), (2,3), (2,4)] :
        print(a*b)
```

```
2
4
6
8
```

반복문 - for

- 이중 반복문

```
[ ]  # 구구단 2단과 3단을 출력하기

[5]  for i in range(2,4) :
         print('===', i, '단 ===')
         for j in range(1, 10) :
             print(i * j)
```



```
=== 2 단 ===
2
4
6
8
10
12
14
16
18
=== 3 단 ===
3
6
9
12
15
18
21
24
27
```

break, continue

for, while에서 제어흐름을 벗어나기 위해 사용

break

for, while을 완전히 중단

continue

처음으로 돌아가 다음반복 수행

break, continue, pass

for 문에서의 예제

break

```
for i in range(5):
  if (i==3):
    break
  print(i,end = '')
```

012

continue

```
for i in range(5):
  if (i==3):
    continue
  print(i,end = '')
```

0124

```
for i in range(5):
  if (i==3):
    pass
  print(i,end = '')
```

01234

break, continue

while 문에서의 예제

break

```
i=0

while i<30:
  if i==20:
    break
  print(i,end ='')
  i +=1
```

```
012345678910111213141516171819
```

continue

```
i=0

while i<30:
  if i==20:
    continue
  print(i,end ='')
  i +=1
```

```
012345678910111213141516171819
```

Toy example

- 1부터 10까지 합을 구하시오. (range 함수를 이용하시오)

연습문제 1

(1) 사용자로부터 정수를 입력받아, 해당 정수만큼 “안녕”을 출력하세요.

(2) 사용자로부터 정수를 입력받아, 입력된 정수 만큼 별 찍기

(1) [출력결과]

```
정수를 입력해주세요: 5
안녕
안녕
안녕
안녕
안녕
```

(2) [출력결과]

```
정수를 입력해주세요: 5
*
**
***
****
*****
```

연습문제 2

(1) 계단형, n을 입력받아, n 만큼 줄을 만들고 계단 형태로 별찍기

(2) 삼각형, 왼쪽 아래가 직각인 n만큼의 높이를 가지는 직각삼각형

(2)-(1), 오른쪽 아래가 직각인 n만큼의 높이를 가지는 직각삼각형

(3) 역삼각형, 왼쪽 위가 직각인 n만큼의 높이를 가지는 직각삼각형

(3)-(1), 오른쪽 위의 직각인 n만큼의 높이를 가지는 직각삼각형

(4) 피라미드, n만큼의 높이를 가지는 홀수개의 별 피라미드

(1) [출력결과]

```
n:5
*****
 *****
  *****
   *****
    *****
```

(2) [출력결과]

```
n : 5
*
**
***
****
*****
```

(2-1) [출력결과]

```
n : 5
    *
   **
  ***
 ****
*****
```

(3) [출력결과]

```
n : 5
*****
****
***
**
*
```

(3-1) [출력결과]

```
n : 5
*****
 ****
  ***
   **
    *
```

(4) [출력결과]

```
n : 5
    *
   ***
  *****
 *******
*********
```

연습문제 3

(1) x = [3, 6, 9, 20, -7, 5] 의 값의 모든 요소에 10을 곱한뒤 출력하세요

심화 : 출력과 리스트 x 의 값에도 모두 10을 곱해주세요

(2) y = {“math”: 70, “science”: 80, “english”: 20} 의 값의 모든 요소에 10을 더한뒤 출력하세요

심화 : 출력과 딕셔너리 y 의 값에도 모두 10을 더해주세요

(3) 조건에 맞는 값 찾기) score = {"민수": 85, "지영": 92, "철수": 67, "수진": 78}

점수가 80점 이상인 학생의 이름만 출력하세요,

심화 : 가장 점수가 높은 학생의 이름과 점수를 출력하세요.

연습문제 4

(1) 재고확인) stock = {“사과” : 12, “바나나” : 0, “포도“ : 3, “딸기” : 0, “귤” : 25}

재고가 0인 과일은 “품절“, 5개 미만이면 “재고 부족“, 그 외에는 “충분 " 을 출력하세요.

심화 : 품절된 과일 이름만 리스트로 모아서 출력하세요.(결과 예) [‘바나나’, ‘딸기’])

(2) 장바구니 계산

price = {“우유” : 2500, “빵” : 3000, “계란“ : 6000, “치즈” : 4500}

cart = {‘우유” : 2, “빵” : 1, “치즈“ : 3}

장바구니(cart)에 담긴 물건의 총 금액을 계산해 출력하세요.

(가격 x 개수를 모두 더하기)

심화 : 총 금액이 20000원 이상이면 10% 할인된 금액을, 아니면 그대로 출력하세요.

연습문제 5

- 1~100 임의의 숫자를 맞추시오.

```
숫자를 맞춰보세요(1~100)
50
숫자가 너무 큽니다.
30
숫자가 너무 큽니다.
10
숫자가 너무 큽니다.
1
숫자가 너무 작습니다.
4
숫자가 너무 작습니다.
5
숫자가 너무 작습니다.
정답입니다. 입력한 숫자는 5입니다.
```

연습문제 6

(1) word = [“school”, “game”, “piano”, “science”, “hotel”, “mountain”] 중 글자수가 6글자 이상인 문자를 모아 새로운 리스트를 생성하세요

(2) 구구단을 1단부터 9단까지 출력하세요

연습문제 7

1-100 까지 숫자중

3과 5의 공배수일경우 “3과 5의 공배수”

나머지 숫자중 3의배수일경우 “3의배수”

나머지 숫자중 5의배수일경우 “5의배수”

모두 해당되지 않을경우 그냥숫자

를 출력하세요

심화 : 1-입력한 숫자까지의 숫자 중

연습문제 8

사용자로부터 숫자를 계속 입력받다가

s or S 를 입력하면 합계출력

(단, 다른 문자는 받을 수 없다.)

값을 입력해주세요30

값을 입력해주세요20

값을 입력해주세요50

값을 입력해주세요40

값을 입력해주세요30

값을 입력해주세요s

합계는 ? 170

연습문제 9

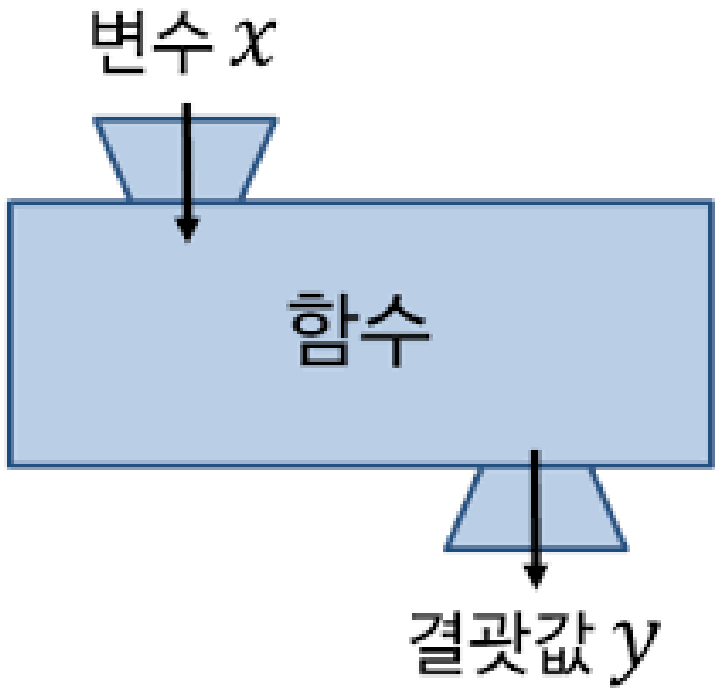
사람 별 평균을 구하라.

- kor_score = [39,69,20,100,80]
- math_score = [32,59,85,30,90]
- eng_score = [49,70,48,60,100]
- midterm_score = [kor_score, math_score, eng_score]

	A	B	C	D	E
국어점수	39	69	20	100	80
수학점수	32	59	85	30	90
영어점수	49	70	48	60	100

함수(Function)

- 함수란?




```
[ ]  def 함수이름(함수인자) :
         수행할 문장1
         수행할 문장2
         return 반환값
```

- 일정한 프로세스를 할 수 있도록 하는 것
- 코드를 논리적인 단위로 분리
- 여러 개의 문장을 하나로 묶어 주는 역할

함수(Function)

```
def 함수이름( 파라미터 ) :
    함수의 내용
    Return 반환값
```

- 함수이름: 사용자가 정의하는 함수 이름, 기존에 사용되는 예약어들을 제외하고 사용
- 파라미터: 함수 안에서 사용할 변수들 (생략 가능)
- return: 함수 안에서 모든 연산을 마친 후 반환할 값(생략 가능)

함수(Function)

pop quiz! 두 개의 input를 받으면 더하기를 하여 값을 돌려주는 함수를 만들어 보자.

```
[ ]  def add(a, b) :
         result = a + b
         return result
```

→ 함수 정의

```
[ ]  c = add(3, 5)
     c
```

```
8
```

→ 함수 호출

함수(Function)

1. 매개변수와 return값이 모두 있는 함수

```
[ ]  def add(a, b) :
         result = a + b
         return result
```

```
[ ]  c = add(3, 5)
     c
```

```
8
```

```
def add(a, b):
    """두 수의 합을 반환"""
    return a + b

def greet(name="student"):
    """기본 인자, 키워드 인자 예시"""
    return f"Hello, {name}"

print(add(2, 3))
print(greet())
print(greet(name="youngpyo"))
```

```
5
Hello, student
Hello, youngpyo
```

2. return값이 비어있는 함수

```
[ ]  def sub(a, b) :
         print('뺄셈의 결과는 %d입니다.' %(a-b))
         return
```

```
[ ]  d = sub(1, 2)
```

```
뺄셈의 결과는 -1입니다.
```

```
[ ]  print(d)
```

```
None
```

3. return이 없는 함수

```
def show_sum(a, b):
    print("덧셈 결과:", a + b)

result = show_sum(3, 2)
print("함수 반환값:", result)
```

```
덧셈 결과: 5
함수 반환값: None
```

```
def f(x):
  print(x+10)
```

```
f(10)
```

```
20
```

```
c = f(10)
```

```
20
```

```
print(c)
```

```
None
```

4. 매개변수와 return값이 모두 없는 함수

```
[ ]  def start():
         print("Hello World!")
```

```
[ ]  start()
```

```
Hello World!
```

함수(Function)

예제) 구구단. 함수를 사용하지 않을 때 VS 함수를 사용할 때

```
for i in range(1,10):
  print(f'==== {i} 단====')
  for j in range(1,10):
    print(i*j)
```

```
==== 1 단====
1
2
3
4
5
6
7
8
9
```

```
def gugu(num):
    for i in range(1, 10):
        print(f'{num} x {i} = {num * i}')
```

```
gugu(3)
```

```
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15
3 x 6 = 18
3 x 7 = 21
3 x 8 = 24
3 x 9 = 27
```

Python naming rule

- 변수와 Class 명 함수명을 짓는 방식이 존재.
- Snake-case : 단어 사이를 언더바(_)로 이어줌(ex. user_list)

  : 식별자가 마치 뱀처럼 생겼다고 붙혀지 이름
- Camale-case : 단어의 첫글자를 대문자로 사용함. (ex. UserList)

  : 식별자가 모양이 마치 낙타모양 같아서 붙여진 이름

함수의 매개변수 전달 방식

1. 불변(Immutable)
: 함수 안에서 바꿔도 원본은 그대로
-> int, str, tuple

```
def bump(x):
    x = x + 1
    return x

n = 10
print(bump(n))
print(n)
```

```
11
10
```

2. 가변(mutable)
: 함수 안에서 바꾸면 원본도 바뀜
-> list, dict, set

```
def push_one(lst):
    lst.append(1)
    return lst

arr = [0]
print(push_one(arr))
print(arr)
```

```
[0, 1]
[0, 1]
```

3. 안전 패턴(원본 보존)

```
def safe_push_one(lst):
    new_lst = lst.copy() #얕은 복사
    new_lst.append(1)
    return new_lst

arr = [0]
print(safe_push_one(arr))
print(arr) #원본 보존
```

함수(Function)

3가지 전달 방식이 존재함.

값에 의한 호출 (Call by Value)	참조에 의한 호출 (Call by Reference)	객체 참조에 의한 호출 (Call by Object Reference)
함수에 인자를 넘길 때 값만 넘김	함수에 인자를 넘길 때 메모리 주소를 넘김	객체의 주소가 함수로 전달됨.
함수 내에 인자 값 변경시에 호출자에게 영향을 주지 않음	함수 내에 인자 값 변경이 되면 호출자의 값도 변경됨.	전달된 객체를 참조하여 변경 시 호출자에게 영향을 주나, 새로운 객체를 만들 경우 호출자에게 영향을 주지 않음.

함수(Function)

```
def Sandwich(eggs):
    eggs.append(1) # 기존 객체의 주소값에 [1] 추가
    eggs = [2, 3]

ham = [0]
Sandwich(ham)
print(ham) # [0, 1]
print(eggs)
```

```
[0, 1]
---------------------------------------------------------------
NameError                                 Traceback (most r
<ipython-input-4-15a09db8233a> in <cell line: 8>()
      6 Sandwich(ham)
      7 print(ham) # [0, 1]
----> 8 print(eggs)

NameError: name 'eggs' is not defined
```

```
# 1번 함수
def swap_value(x, y):
    temp = x
    x = y
    y = temp

# 2번 함수
def swap_offset(offset_x, offset_y):
    temp = a[offset_x]
    a[offset_x] = a[offset_y]
    a[offset_y] = temp

# 3번 함수
def swap_reference(list, offset_x, offset_y):
    temp = list[offset_x]
    list[offset_x] = list[offset_y]
    list[offset_y] = temp
```

```
#테스트
a = [1,2,3,4,5]
swap_value(a[1],a[2]) #swap이 발생 안함
print(a)
swap_offset(1,2) #swap이 발생
print(a)
swap_reference(a,1,2) #swap 발생
print(a)
```

연습문제 1

함수를 만들어 보자.

- 이 함수는 두 개의 숫자를 input으로 받으면 앞의 숫자를 뒤의 숫자로 나누는 함수이다.
- 나눗셈을 한 후에는 몫과 나머지 순으로 된 튜플 값을 반환한다.

```
def div(a, b):
# 정답을 적어주세요
```

```
div(10, 3)
```

```
(3, 1)
```

```
div(3, 5)
```

```
(0, 3)
```

```
div(1, 10)
```

```
(0, 1)
```

연습문제 2

함수를 만들어보자.

- 이 함수는 두 개의 숫자를 input으로 받으면 작은 수로 큰 수를 나눈 몫과 나머지를 반환하는 함수이다.
- 반환 값은 튜플로 되어 있으며 몫, 나머지 순으로 되어 있다.
- 단, 0으로 나누는 것은 불가하기 때문에 두 수 중에 작은 수가 0이라면 화면에 '0은 사용할 수 없습니다.'를 출력하고 종료되어야한다.

```
div2(3, 5)
```

```
(1, 2)
```

```
div2(0, 5)
```

```
0은 사용할 수 없습니다.
```

```
div2(10, 1)
```

```
(10, 0)
```

함수(Function)

- 더 알아보기

1. 매개변수의 초기값 미리 설정하기

```
[ ]  def function(a, n=2) :
         print("%d의 제곱은 %d 입니다." %(a, a**n))
```

```
[ ]  function(4)
```

```
4의 제곱은 16 입니다.
```

⚠️ 초기값을 설정할 때는 매개변수 순서에 주의.

초기값이 없는 변수 → 초기값이 있는 변수 순으로 배치해야 함.

함수(Function)

- 더 알아보기

2. 매개변수가 몇 개가 필요할지 모를 때

```
[3]  def test(*args) :
       print(args)
```

→ 매개변수에 *args를 붙이면 함수가 받은 input 모두를 tuple으로 묶어줌

```
[4]  test(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)

     (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
```

Quiz! 왜 tuple로 묶어줄까요?

```
[5]  test([1,2],3)

     ([1, 2], 3)
```

*(Asterisk)

- 더 알아보기

  2. 매개변수가 몇 개가 필요할지 모를 때

```
def add(**kwargs):
    print(kwargs)
```

→ 매개변수에 *kwargs를 붙이면 함수가 받은 input 모두를 dict으로 묶어줌

```
add(a =1,b=2, c=3)
```

```
{'a': 1, 'b': 2, 'c': 3}
```

Quiz! 왜 dict로 묶어줄까요?

연습문제 3

어떠한 string을 받으면 일정한 단위로 끊어서 화면에 출력하는 함수를 짜보자.
끊는 단위는 따로 정하지 않으면 2로 설정해보자.

Hint : input을 string 과 unit = 2 로 받고, while을 사용하고, 길이는 len 함수를 사용하도록 하자.

```
: func('테스트를 위한 문장입니다.')
```

```
테스
트를
 위
한
문장
입니
다.
```

```
: func('테스트를 위한 문장입니다.', 4)
```

```
테스트를
 위한
문장입니
다.
```

연습문제 4

add_all 함수를 짜봅시다

```
add_all(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
```

55

Hint : *args를 input으로 받으세요.

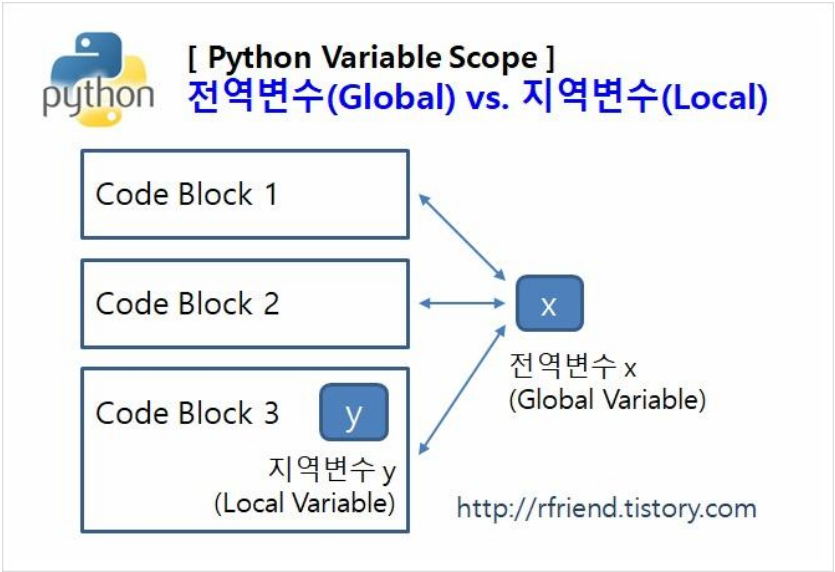
함수(Function)

- 함수와 변수

  Quiz! 함수 안에 선언한 변수를 함수 밖에서도 사용할 수 있을까?

  NO!

  함수 안의 블록은 함수 고유의 영역임.
  따라서, 함수 안에서 선언한 변수는 함수 밖에서 사용할 수 없음.



함수(Function)

- 함수와 변수

  예시 1)

```
[ ]  def myfunction() :
         in_val = '함수 안의 변수'
```

```
[ ]  myfunction()
```

```
[ ]  in_val
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-30-a23adeea90d3> in <module>
----> 1 in_val

NameError: name 'in_val' is not defined
```

함수(Function)

- 함수와 변수

  예시 2)

```
[ ]  a = 1
```

```
[ ]  def myfunction2(a) :
         a += 1
         print(a)
```

```
[ ]  myfunction2(a)
```

```
2
```

```
[ ]  a
```

```
1
```

함수 실행 시 출력값은 무엇일까요?

a는 어떤 값일까요?

함수(Function)

함수 안의 블록은 함수 고유의 영역임.

따라서, 함수 안에서 선언한 변수는 함수 밖에서 사용할 수 없음.

```
[ ]  def 함수이름(함수인자) :
         수행할 문장1
         수행할 문장2
         return 반환값
```

2. global 사용하기 ← 권장하지 않음

- 함수와 변수

- 함수 안에 있는 변수를 밖에서도 쓰고 싶다면?

1. return 사용하기
2. global 사용하기

함수 안에 있는 변수를 밖에서도 쓰고 싶다면?

1. return 사용하기

```
a = 1

def myfunction3(a):
    a += 1
    print(a)
    return a  # a 값을 반환

a = myfunction3(a)  # 반환된 값을 밖의 변수 a에 할당
print("함수 밖의 a:", a)
```

꼭 값을 할당해야 함

함수(Function)

함수 안에 있는 변수를 밖에서도 쓰고 싶다면?

2. global 사용하기(권장하지 않음)

```
[ ] a = 1

[ ] def myfunction4():
        global a
        a += 1
        print(a)

[ ] myfunction4()

    2

[ ] a

    2
```

```
a=1
print('처음에 있는 전역변수',a)
def myfunction():
    global a
    a+=1
    print('안에 있는 a',a)
myfunction()
print('밖에 있는 a',a)
```

```
처음에 있는 전역변수 1
안에 있는 a 2
밖에 있는 a 2
```

함수 안에 있는 변수를 밖에서도 쓰고 싶다면?

3. 딕셔너리나 리스트를 사용해서 값을 변경하기

```
data = {"a": 1}

def myfunction3(data):
    data["a"] += 1
    print(data["a"])

myfunction3(data)
print("함수 밖의 a:", data["a"])
```

연습문제

- 팩토리얼(Factorial)을 구하시오.

1부터 시작하여 어떤 범위에 있는 모든 정수를 곱하는 것.

Ex) 5! = 120

Function annotation

```
def func_name(params):
  return 0
```

```
def func_name(param1 : int, param2 : str) -> int:
  return 0
```

- 파이썬 3.5부터 추가된 기능(PEP 484 기반)
- 함수의 매개변수와 반환값에 **자료형을 주석처럼 표시**할 수 있음
- 실행 결과에는 영향을 주지 않고, **코드를 읽는 사람과 도구**를 위한 설명

```
def greet(name: str) -> None:
    print("안녕하세요,", name)

greet("철수")   # 정상 실행
greet(123)      # 실행은 되지만, 타입상 권장 X
```

```
안녕하세요, 철수
안녕하세요, 123
```

```
def add(a: int, b: int) -> int:
    return a + b

print(add(2, 3))      # 5
print(add("a", "b"))  # 실행은 되지만 타입상 맞지 않음
```

```
5
ab
```

- PEP 484: 파이썬 3.5부터 적용된 “타입 힌트 문법”을 정의한 공식 문서

Match Case문

- 파이썬 3.10 버전부터 Match와 Case문이 도입이 됨.
- Match문 : 패턴 매칭을 통해 값을 비교하고 실행 코드를 선택하는데 사용.
- Case문 : Match문 내에서 특정 패턴에 대한 처리를 정의하는데 사용.

```
match value:
    pattern_1:
        # pattern_1에 일치하는 경우 실행할 코드
    pattern_2:
        # pattern_2에 일치하는 경우 실행할 코드
    ...
    pattern_n:
        # pattern_n에 일치하는 경우 실행할 코드
    _:
        # 위의 패턴에 일치하지 않는 경우 실행할 코드
```

```
def classify_number(n):
    match n:
        case 0:
            print("Zero")
        case 1 | 2 | 3:
            print("Small number")
        case 4 | 5 | 6 | 7 | 8 | 9:
            print("Big number")
        case _:
            print("Other number")

classify_number(0)  # Zero
classify_number(2)  # Small number
classify_number(7)  # Big number
classify_number(10) # Other number
```

Thank you.

스마트제조_1일차
ryp1662@gmail.com

Youngpyo Ryu. All rights reserved.
Unauthorized reproduction or distribution is prohibited.